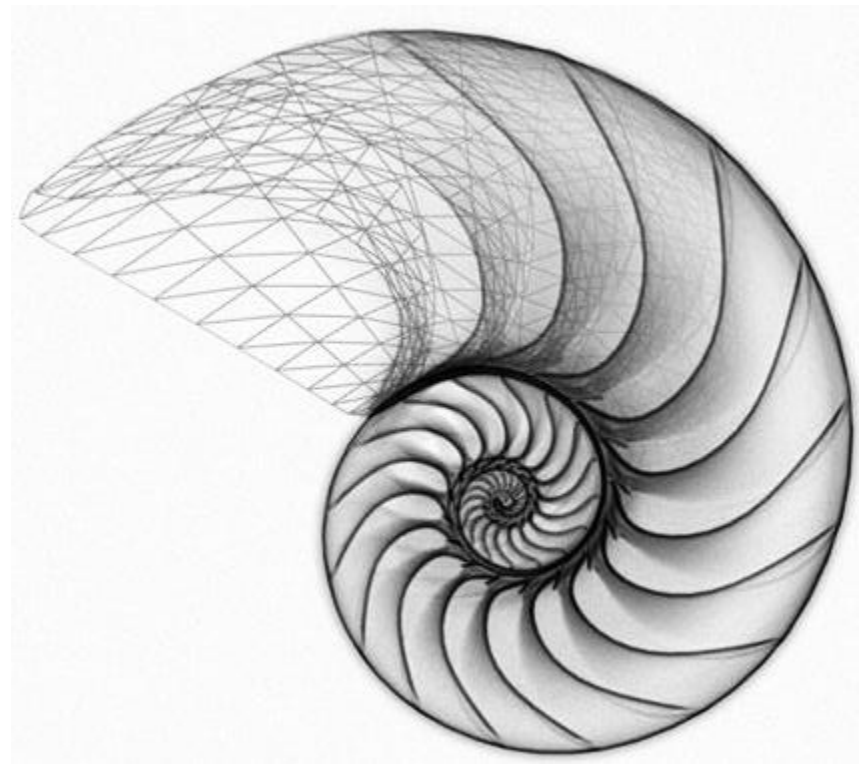




Lecture 11

Recursion

CSE225: Data Structures and Algorithms

Another Example:

n choose r (combinations)

- Given n things, how many different sets of size r can be chosen?

$$\binom{n}{r} = \frac{n!}{r! \times (n-r)!}, 1 < r < n \text{ (closed form)}$$

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}, 1 < r < n \text{ (recursive)}$$

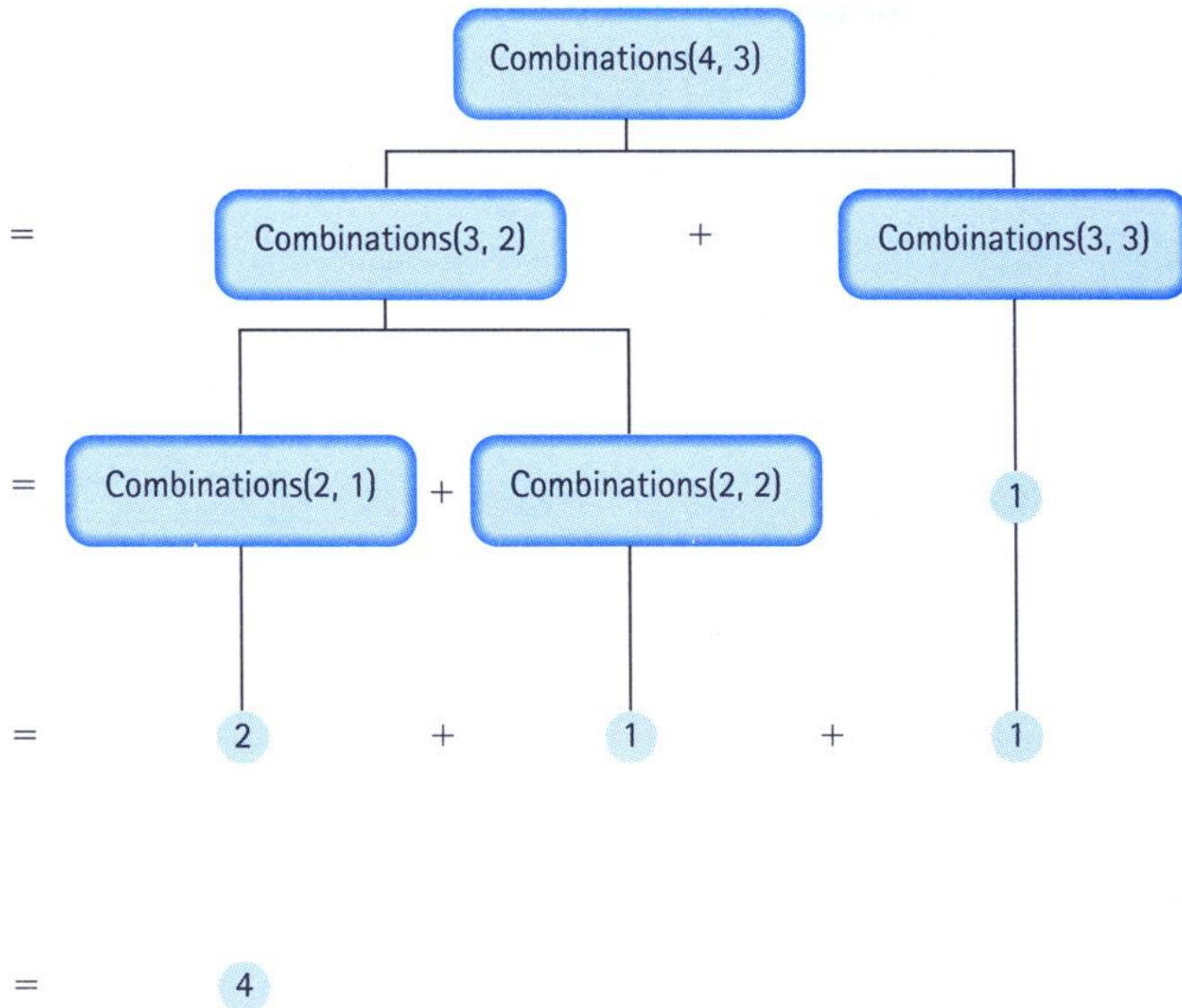
with base cases:

$$\binom{n}{1} = n \text{ and } \binom{n}{n} = 1$$

n choose r (Combinations)

```
int Combinations(int n, int r)
{
    if(r == 1)    // base case 1
        return n;
    else if (n == r)    // base case 2
        return 1;
    else //general case
        return(Combinations(n-1, r-1) + Combinations(n-1, r));
}
```

Tracing Combinations(4,3)



Binary Search: The Recursive Way

- **Base case:**

1. Array is empty: **item not found**
2. Array is non-empty and item is the middle element:
item is found

- **General case:**

- Search either the right half or the left half of the array

Binary Search: The Recursive Way

```
template<class ItemType>
bool BinarySearch(ItemType info[], ItemType item, int fromLocation,
int toLocation)
{
    if (fromLocation > toLocation) // Base case 1
        return false;
    else
    {
        int midPoint;
        midPoint = (fromLocation + toLocation) / 2;
        if (item < info[midPoint])
            return BinarySearch(info, item, fromLocation, midPoint - 1);
        else if (item == info[midPoint]) // Base case 2
            return true;
        else
            return BinarySearch(info, item, midPoint + 1, toLocation);
    }
}
```

A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

A

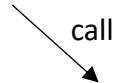
index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

Find 9:

A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

Find 9:

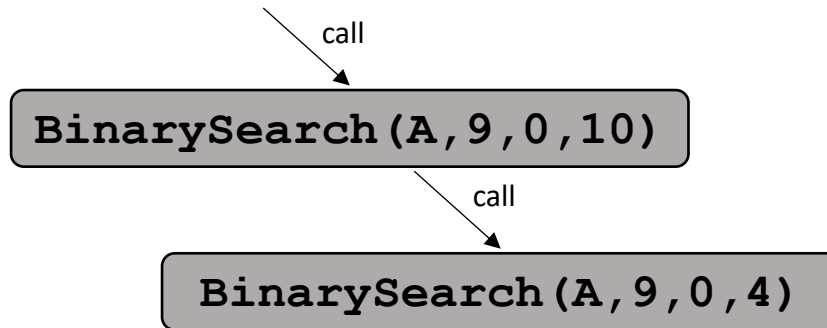


```
BinarySearch(A, 9, 0, 10)
```

A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

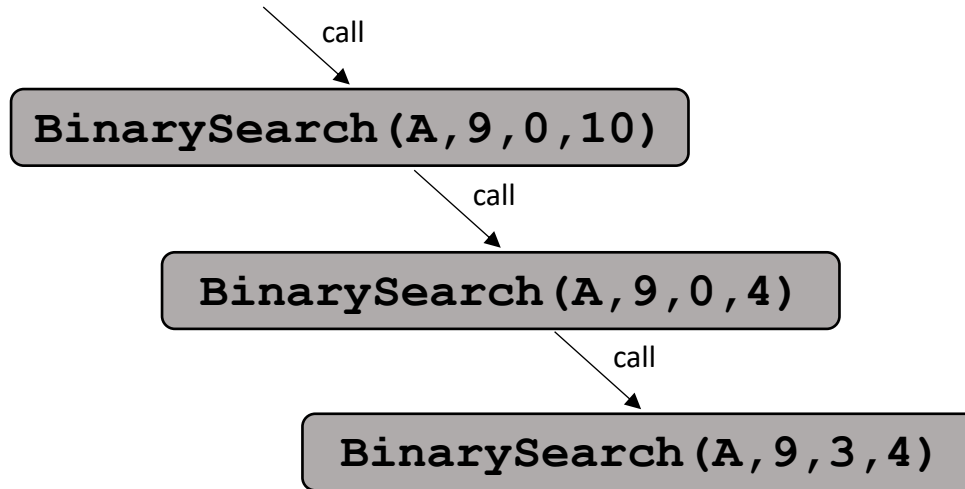
Find 9:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

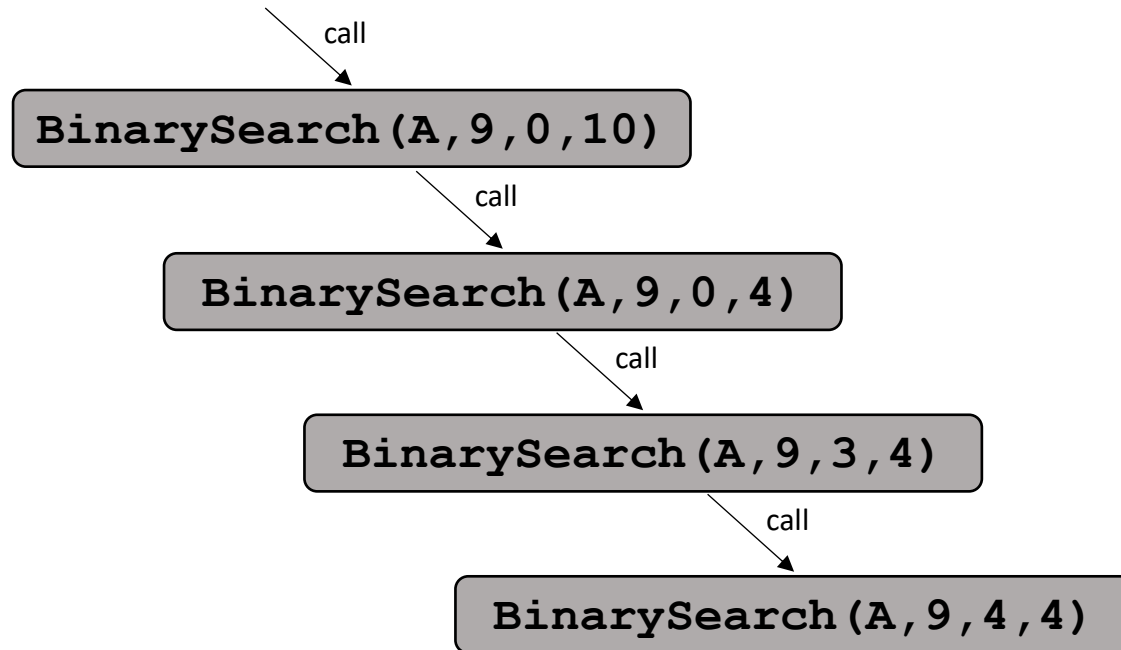
Find 9:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

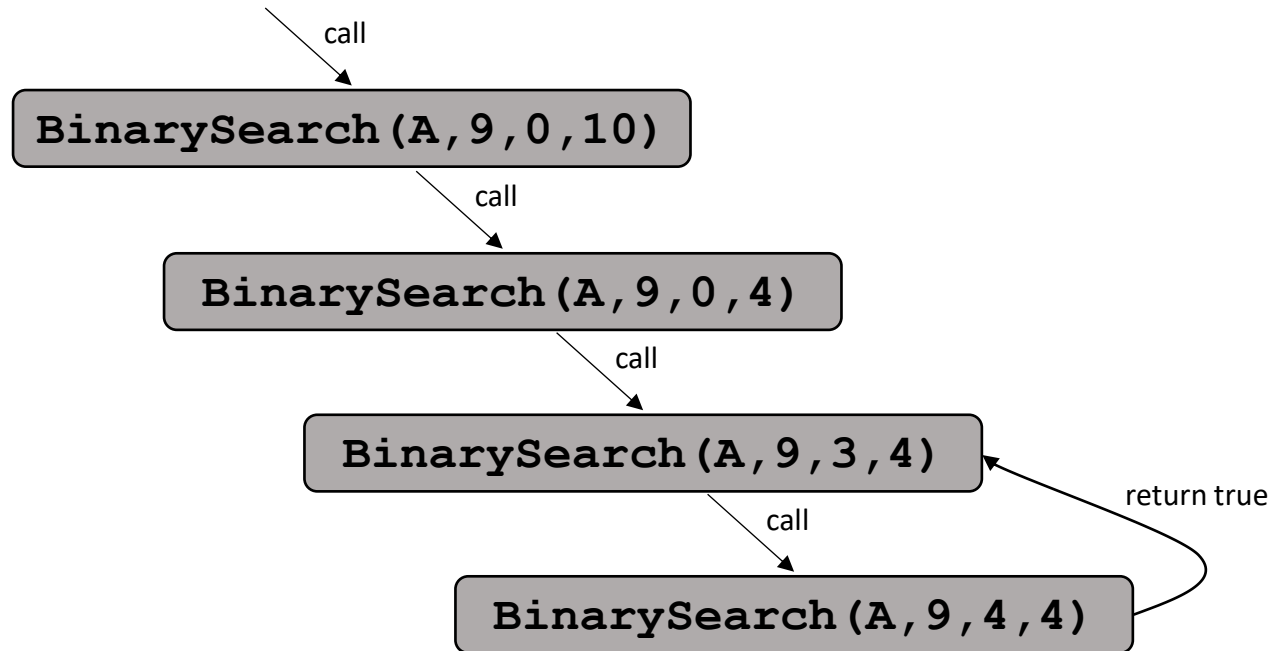
Find 9:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

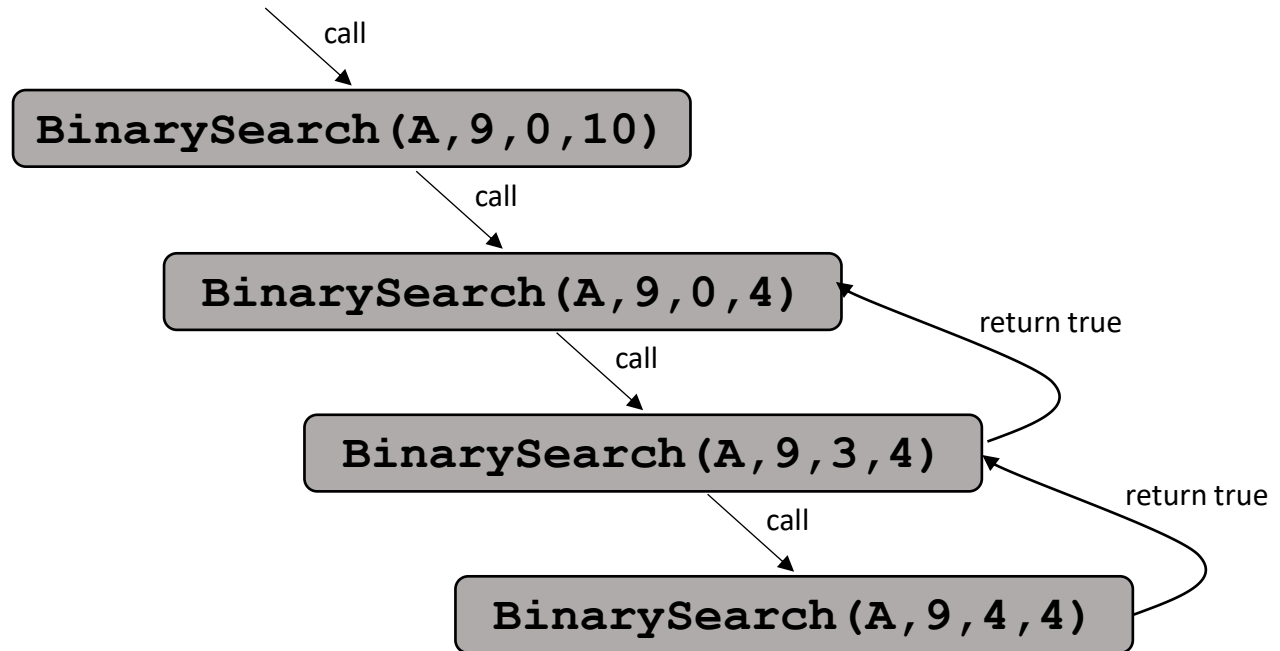
Find 9:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

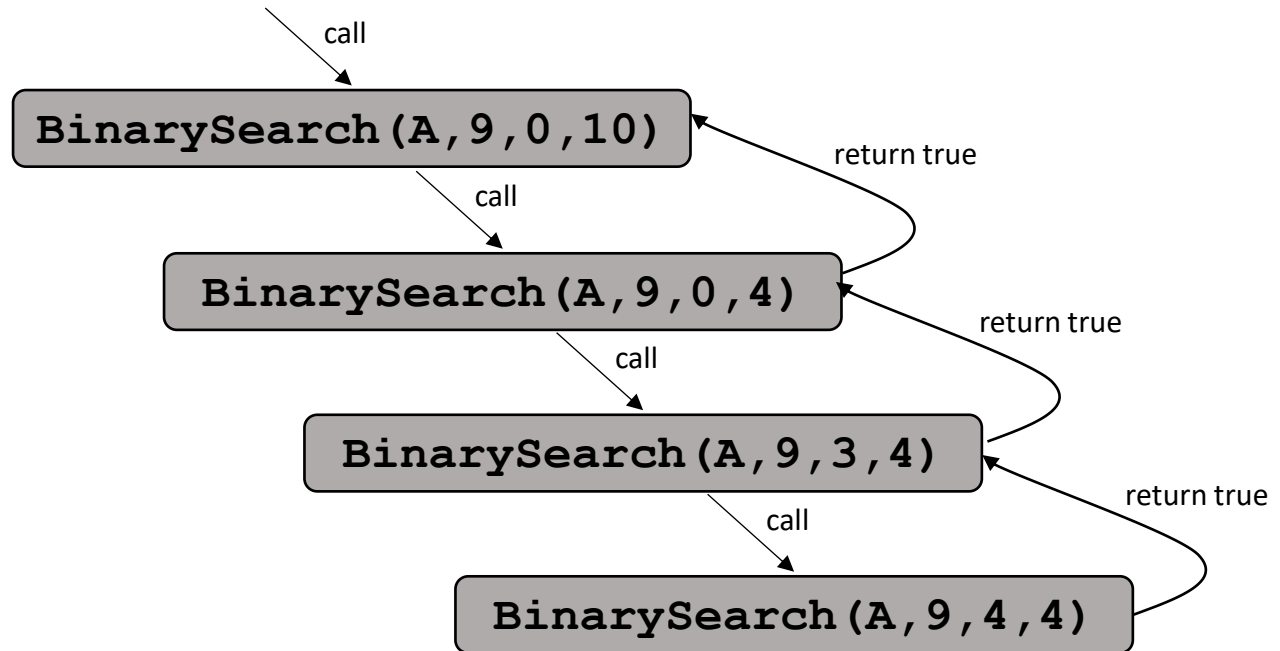
Find 9:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

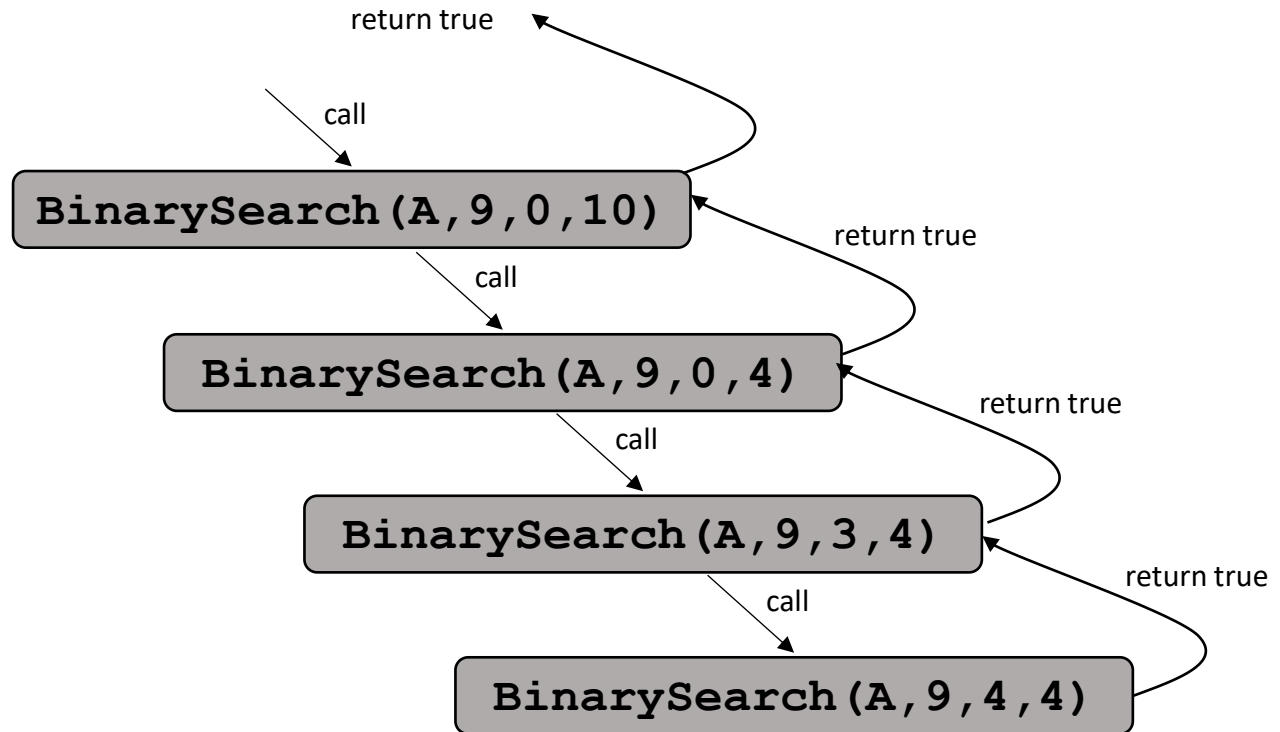
Find 9:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

Find 9:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

A

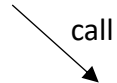
index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

Find 20:

A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

Find 20:

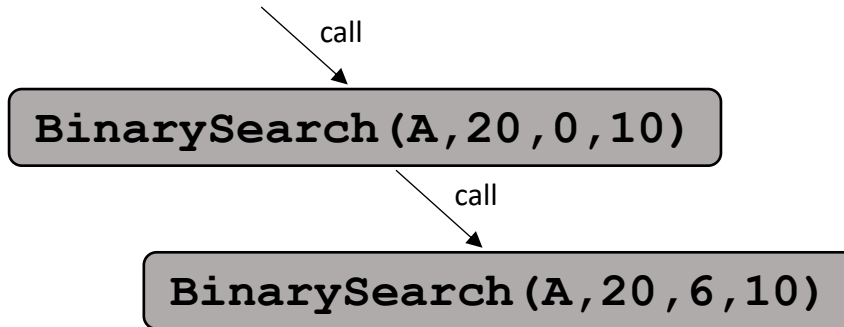


BinarySearch(A, 20, 0, 10)

A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

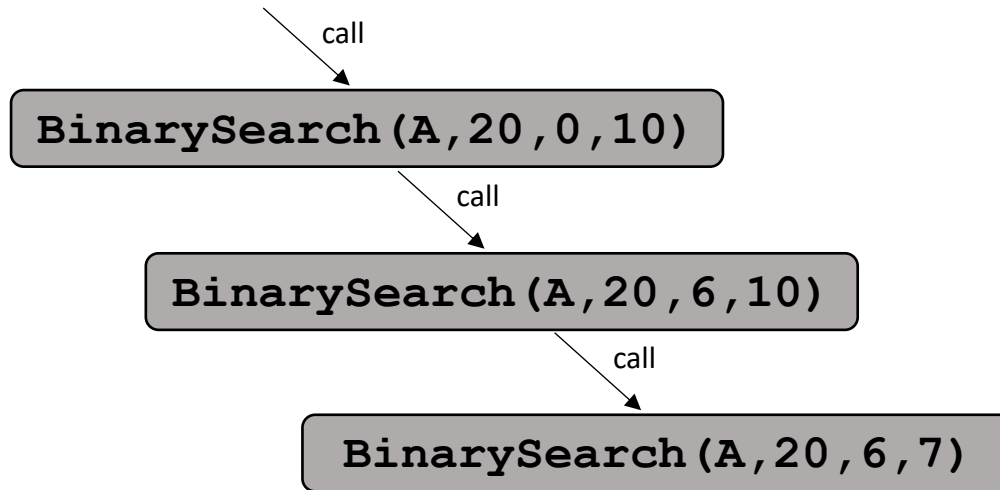
Find 20:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

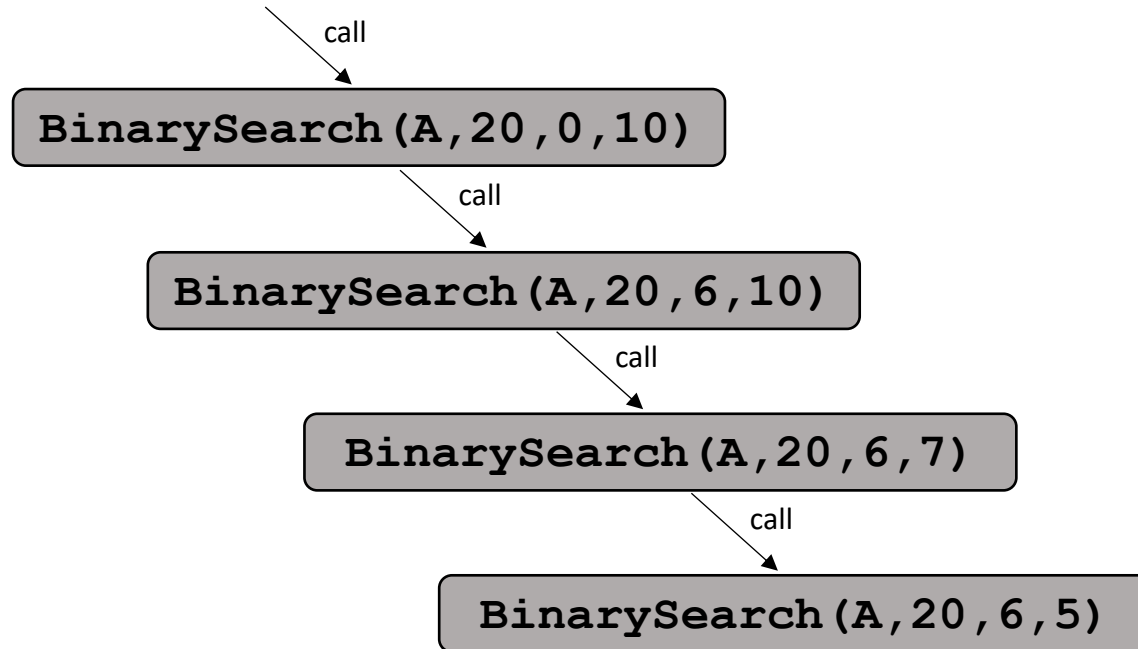
Find 20:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

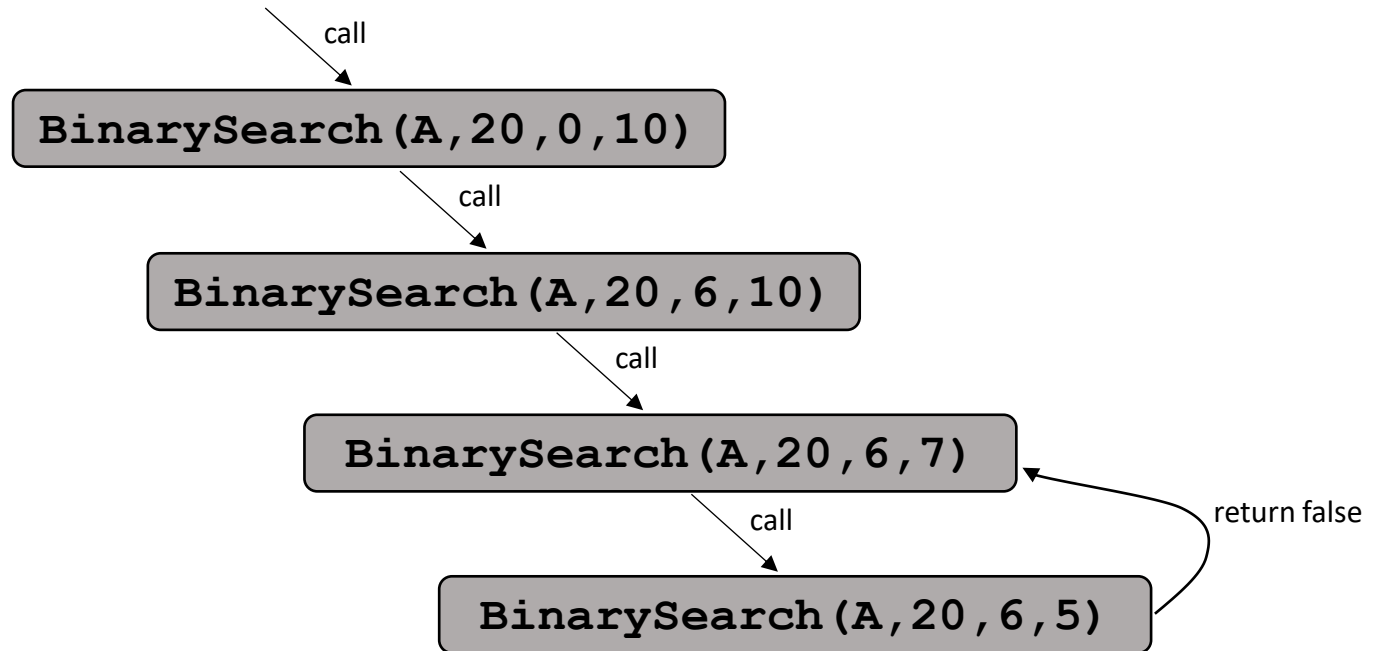
Find 20:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

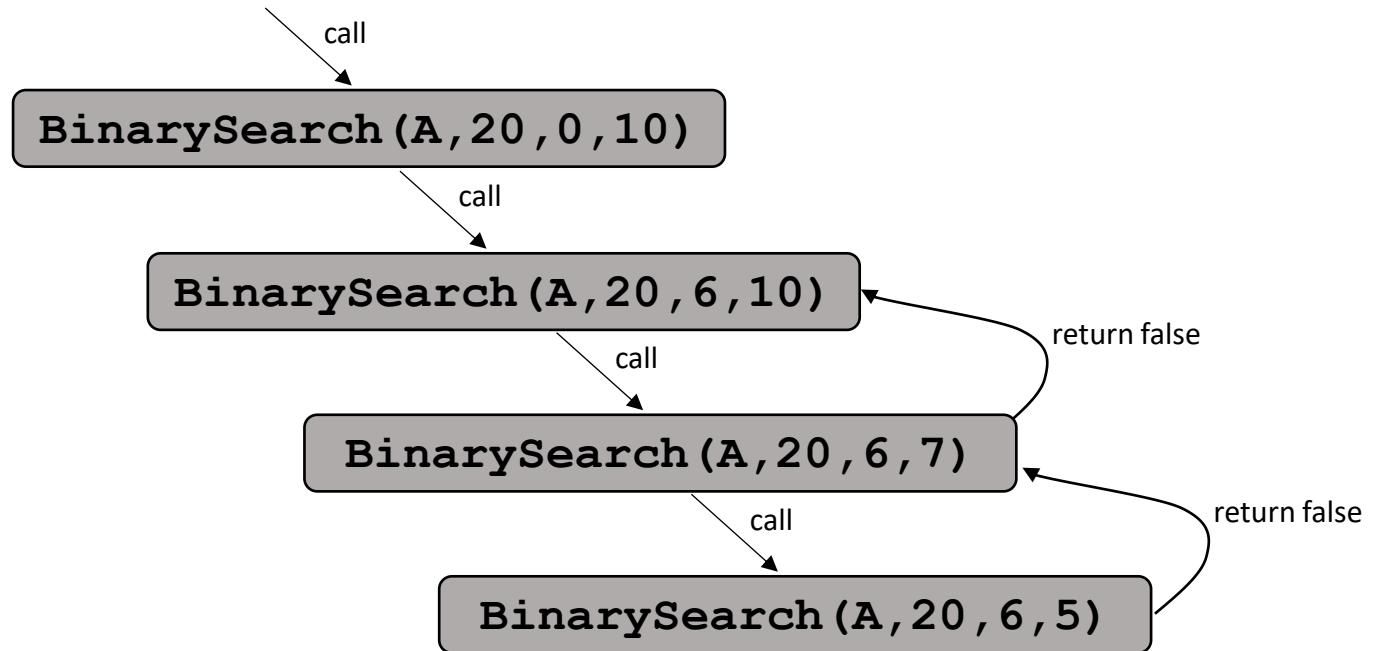
Find 20:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

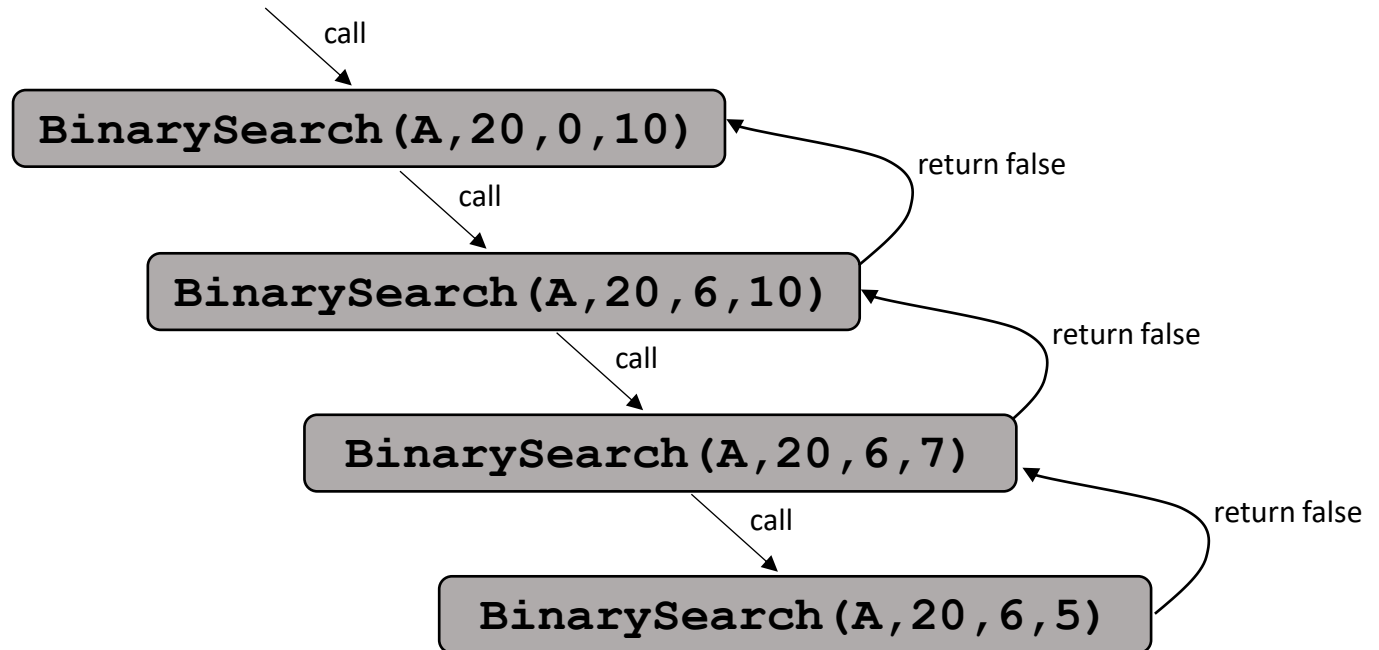
Find 20:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

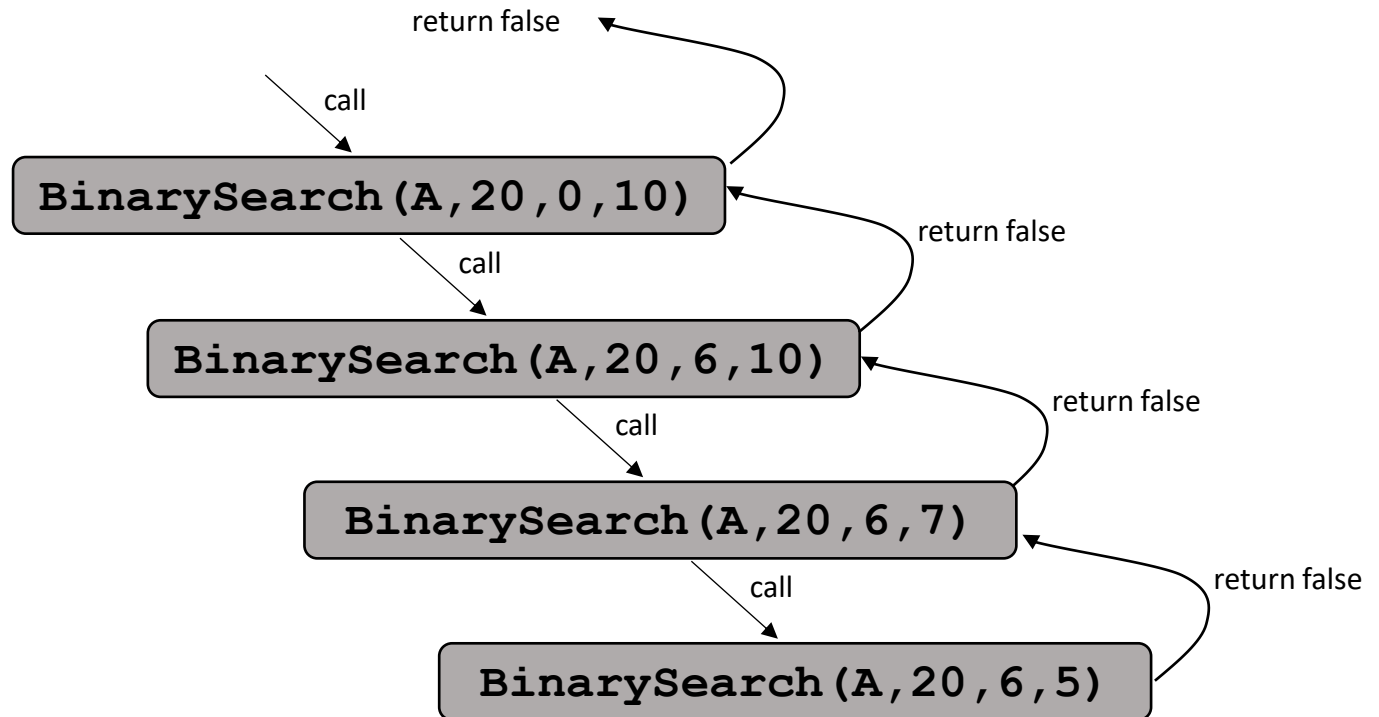
Find 20:



A

index	0	1	2	3	4	5	6	7	8	9	10
value	1	3	5	8	9	17	32	36	42	49	53

Find 20:





The Puzzle

- Tower of Hanoi is a mathematical puzzle invented by a French Mathematician Edouard Lucas in 1883.
- The game starts by having few discs stacked in increasing order of size. The number of discs can vary, but there are only three pegs.



The Puzzle

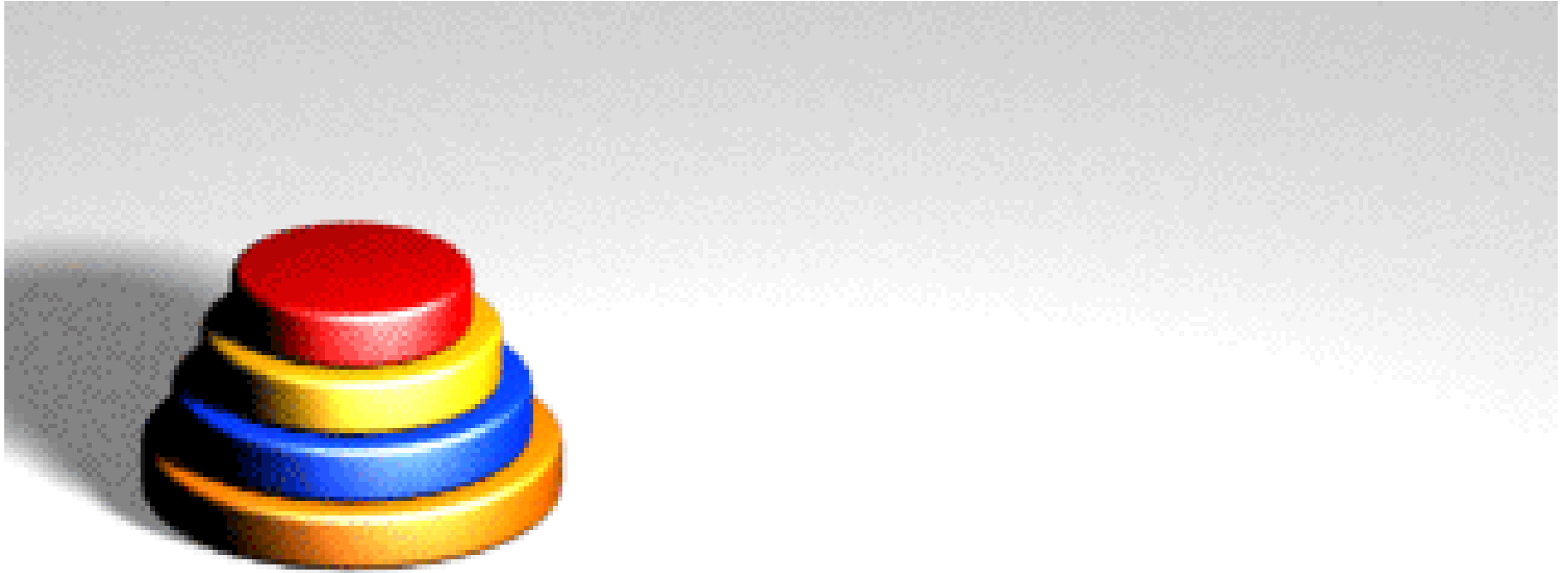
- The Objective is to transfer the entire tower to one of the other pegs. However you can only move one disk at a time and you can never stack a larger disk onto a smaller disk. Try to solve it in fewest possible moves.



Solve Hanoi with 4 Discs

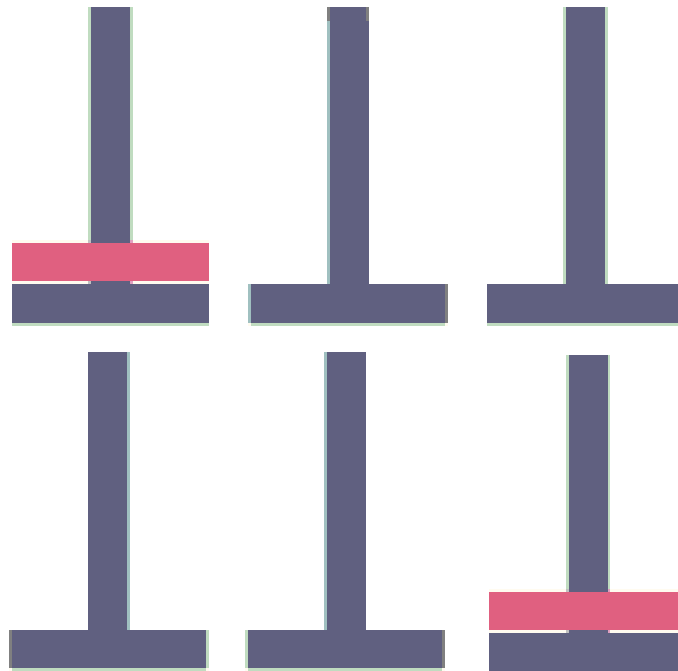


Solve Hanoi with 4 Discs



Recursive Solution

- Problem: Move **N** discs from **source** peg to **destination** peg using **auxiliary** peg as via
- Base case:
 - Move disc from source peg to destination peg ($N = 1$)



Recursive Solution

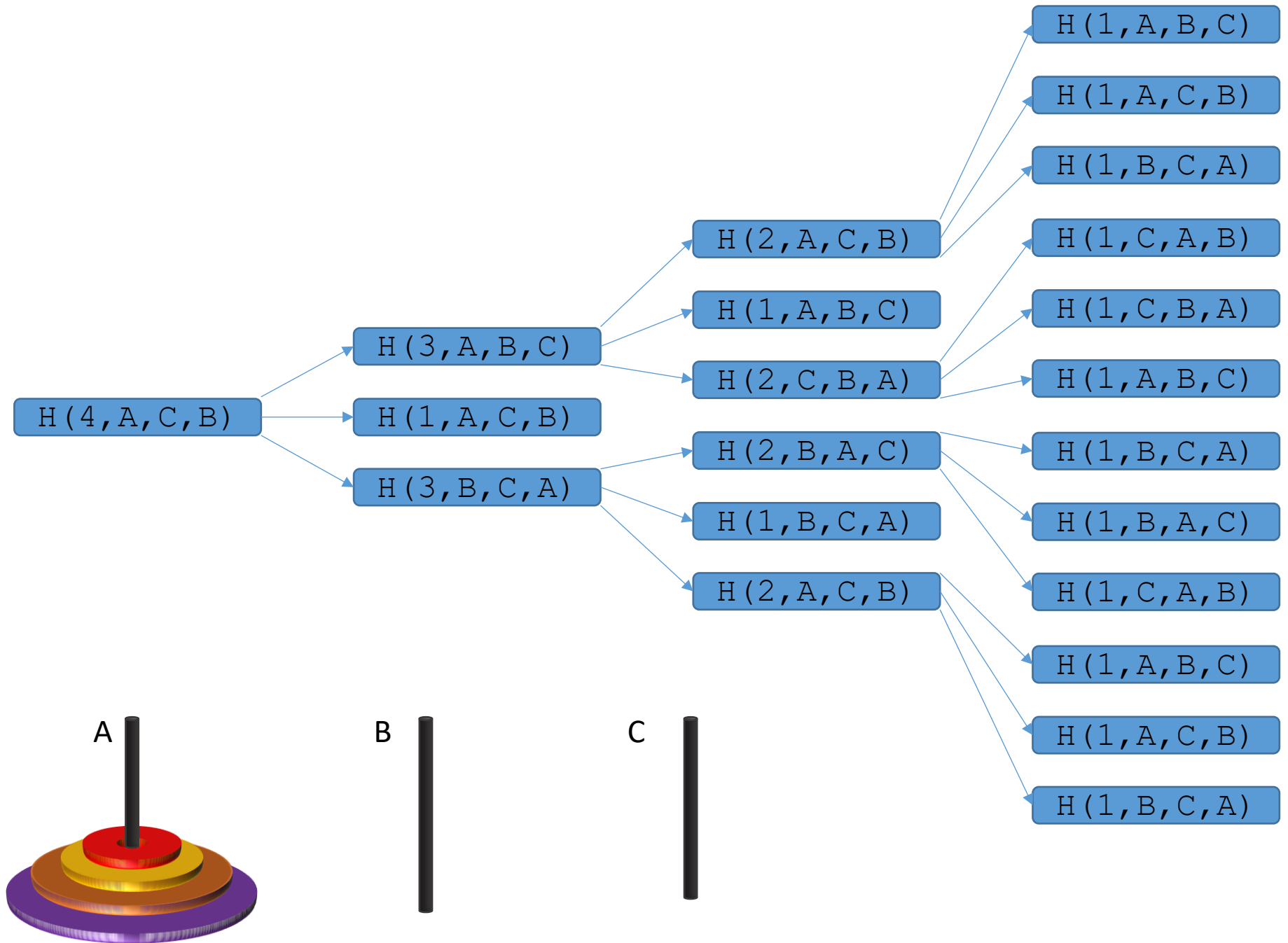
- Problem: Move **N** discs from **source** peg to **destination** peg using **auxiliary** peg as via
- General case:
 - Move $N-1$ discs from source peg to auxiliary peg via destination peg
 - Move disc from source peg to destination peg
 - Move $N-1$ discs from auxiliary peg to destination peg via source peg

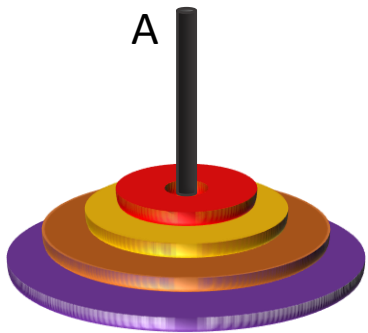
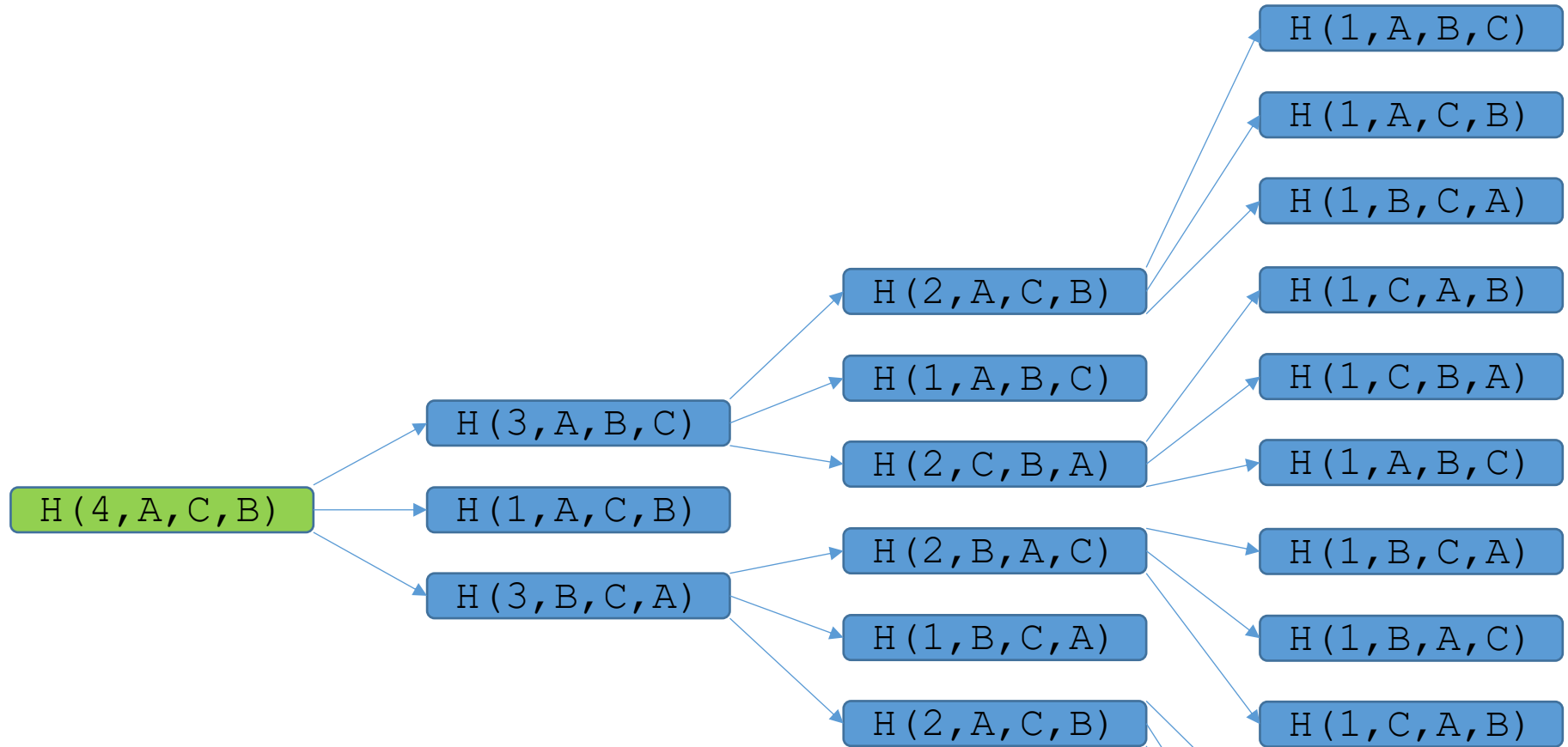


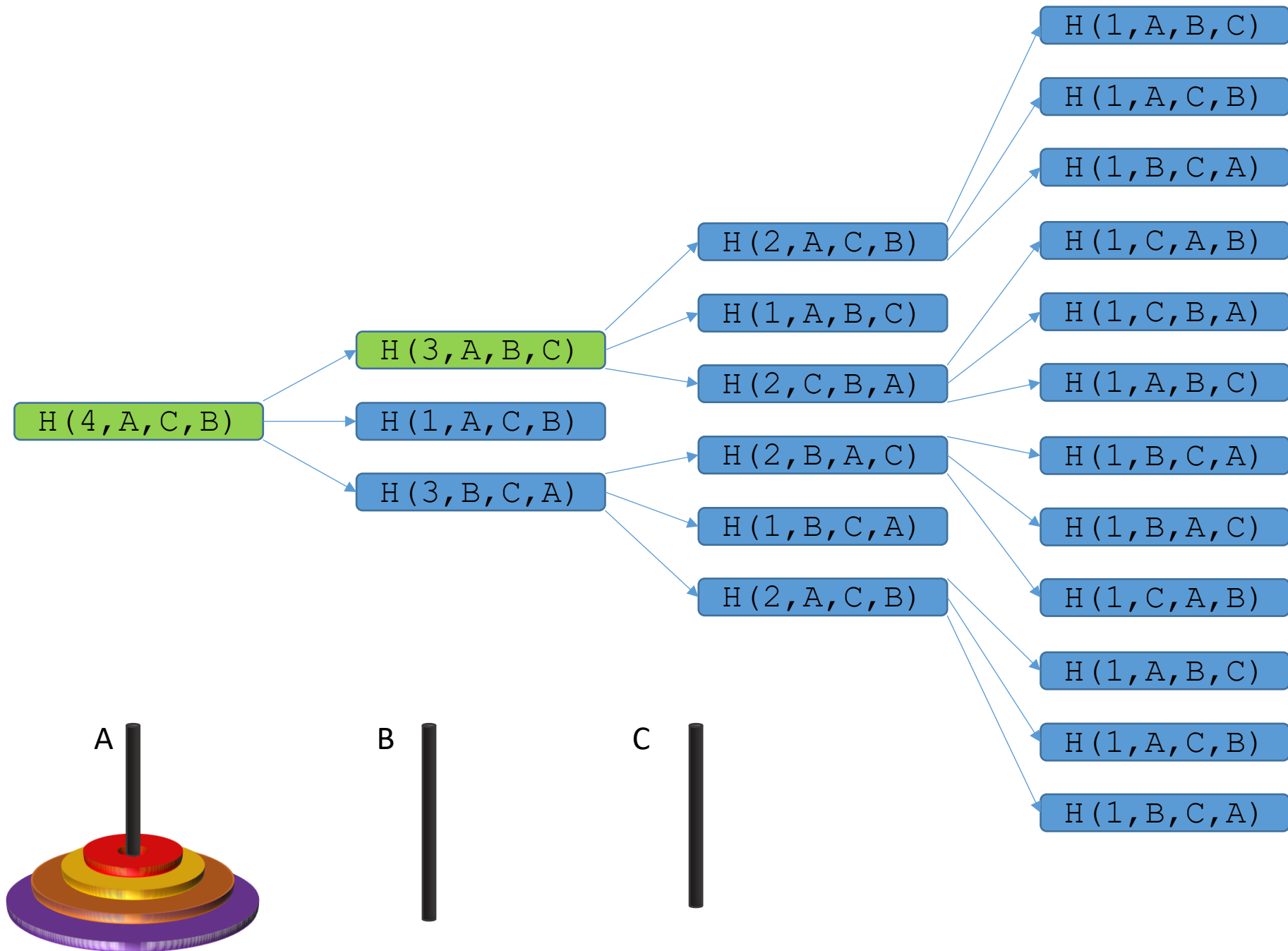
Recursive Function for Solving Hanoi

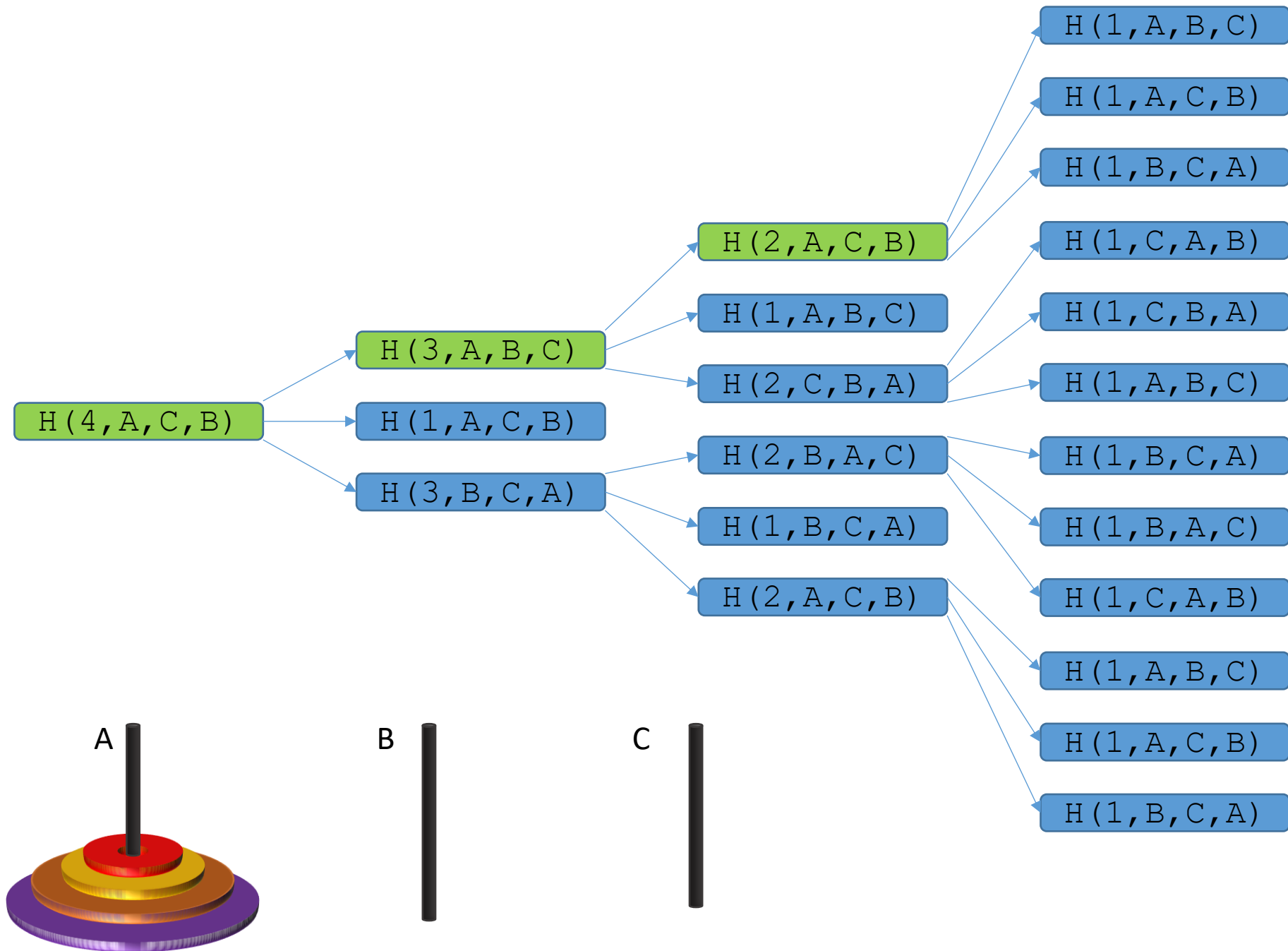
```
#include <stdio.h>
void hanoi(int n, char src, char dst, char aux)
{
    if(n == 1)
        printf("Move disc from peg %c to peg
%c\n", src, dst);
    else
    {
        hanoi(n-1, src, aux, dst);
        hanoi(1, src, dst, aux);
        hanoi(n-1, aux, dst, src);
    }
}

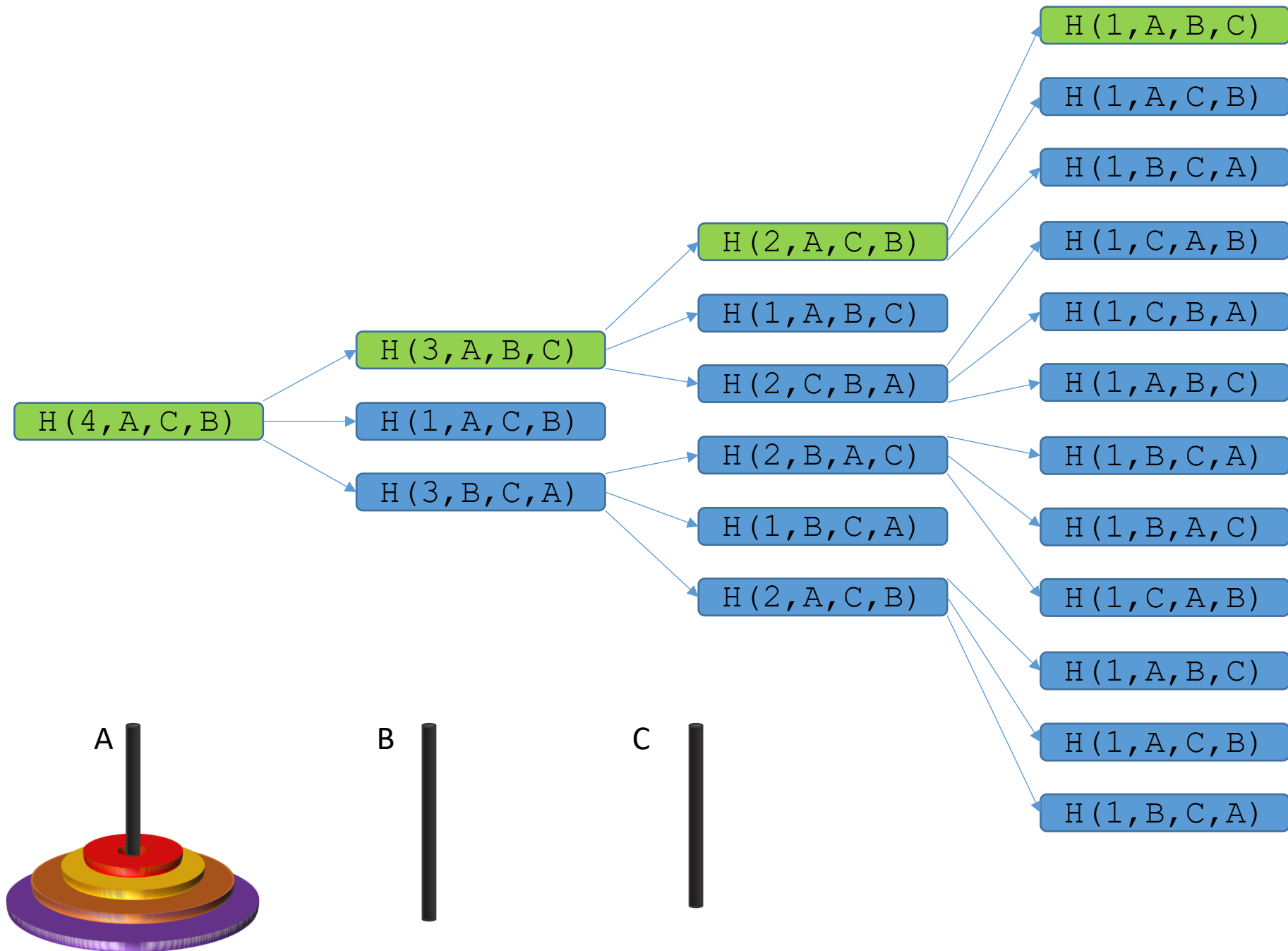
int main()
{
    hanoi(4, 'A', 'C', 'B');
    return 0;
}
```

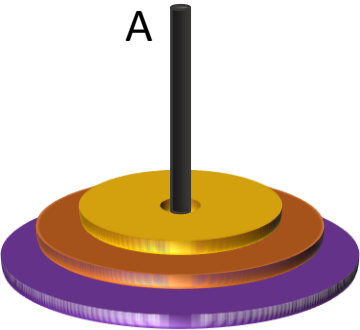
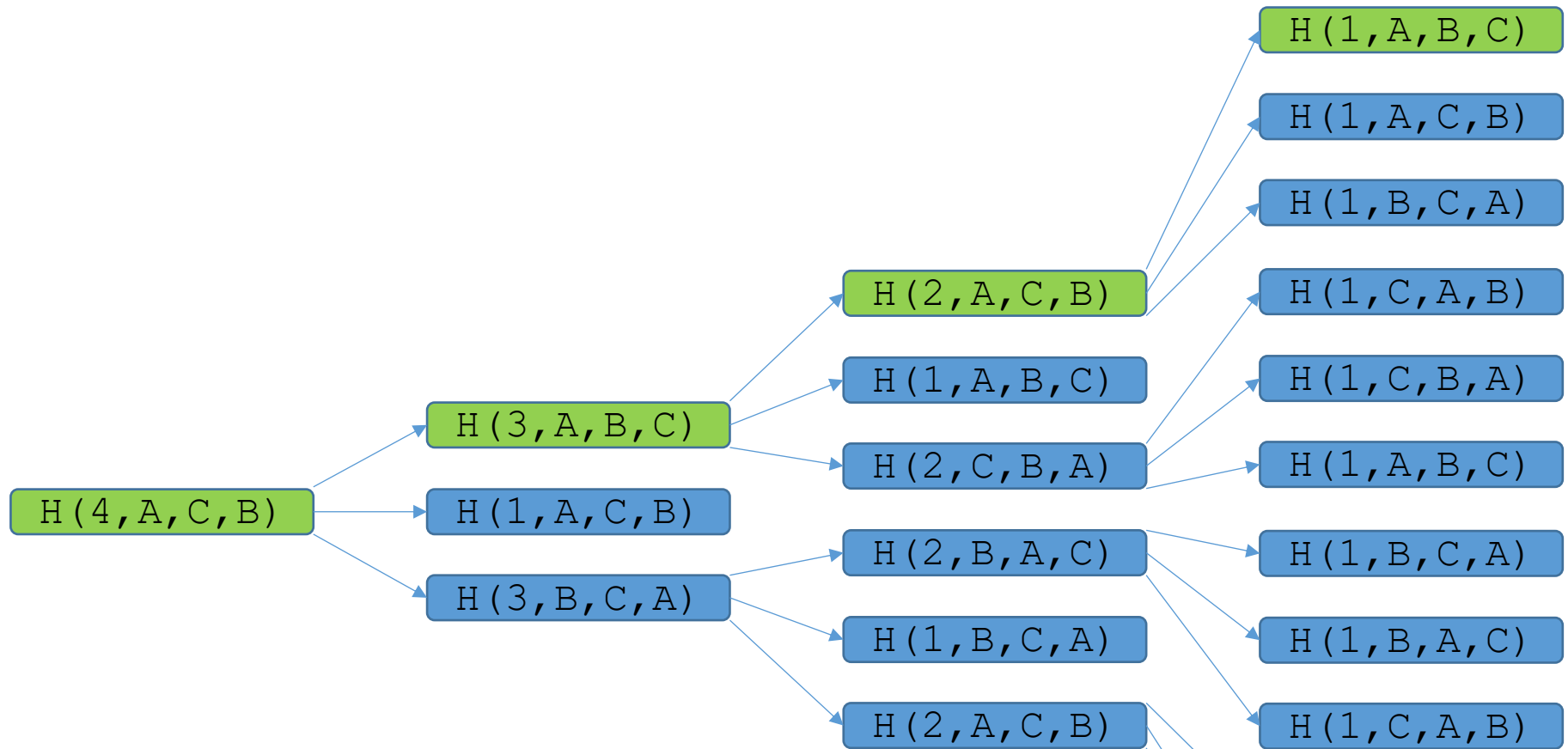


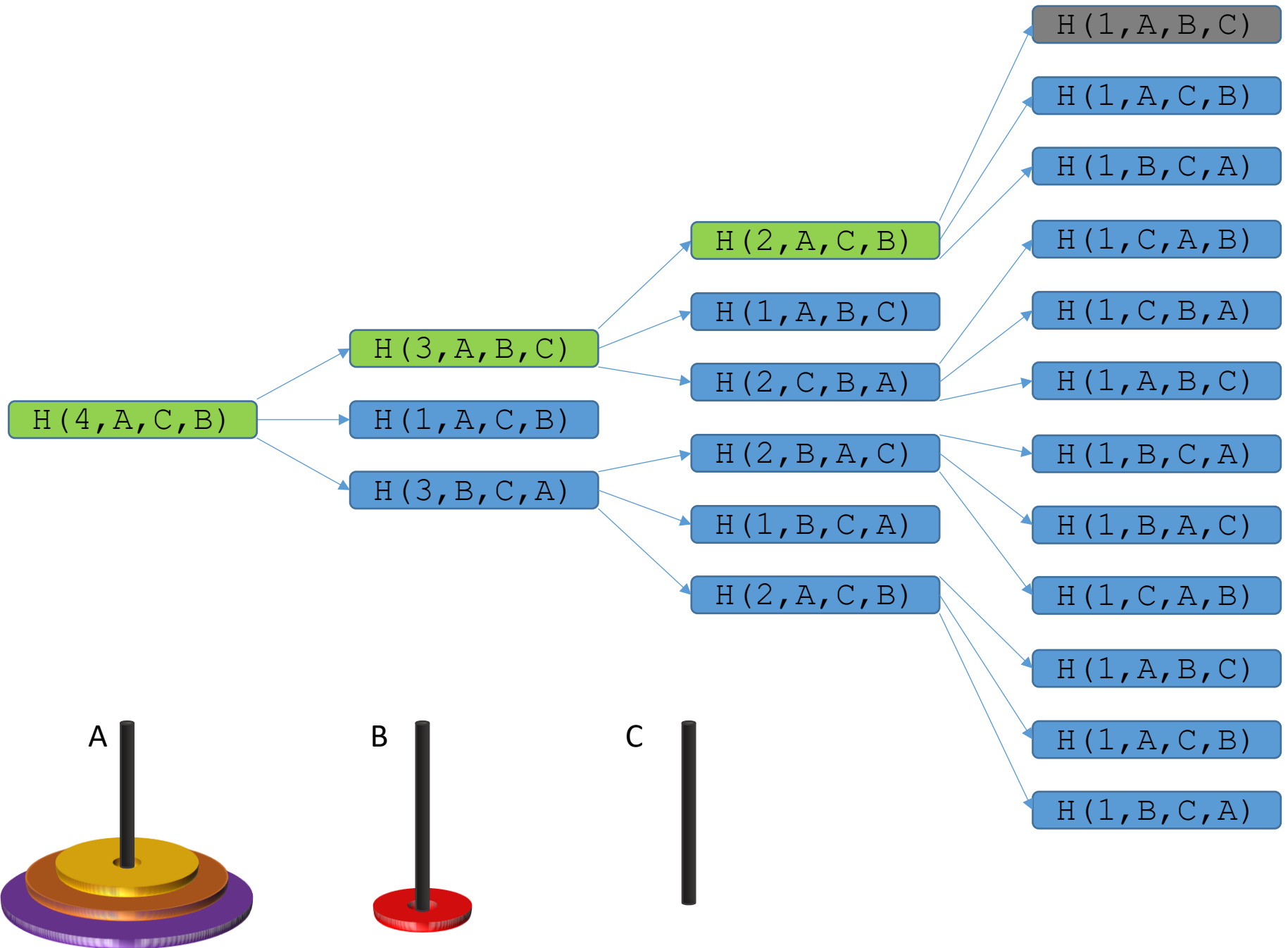


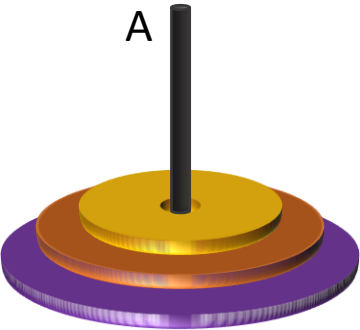
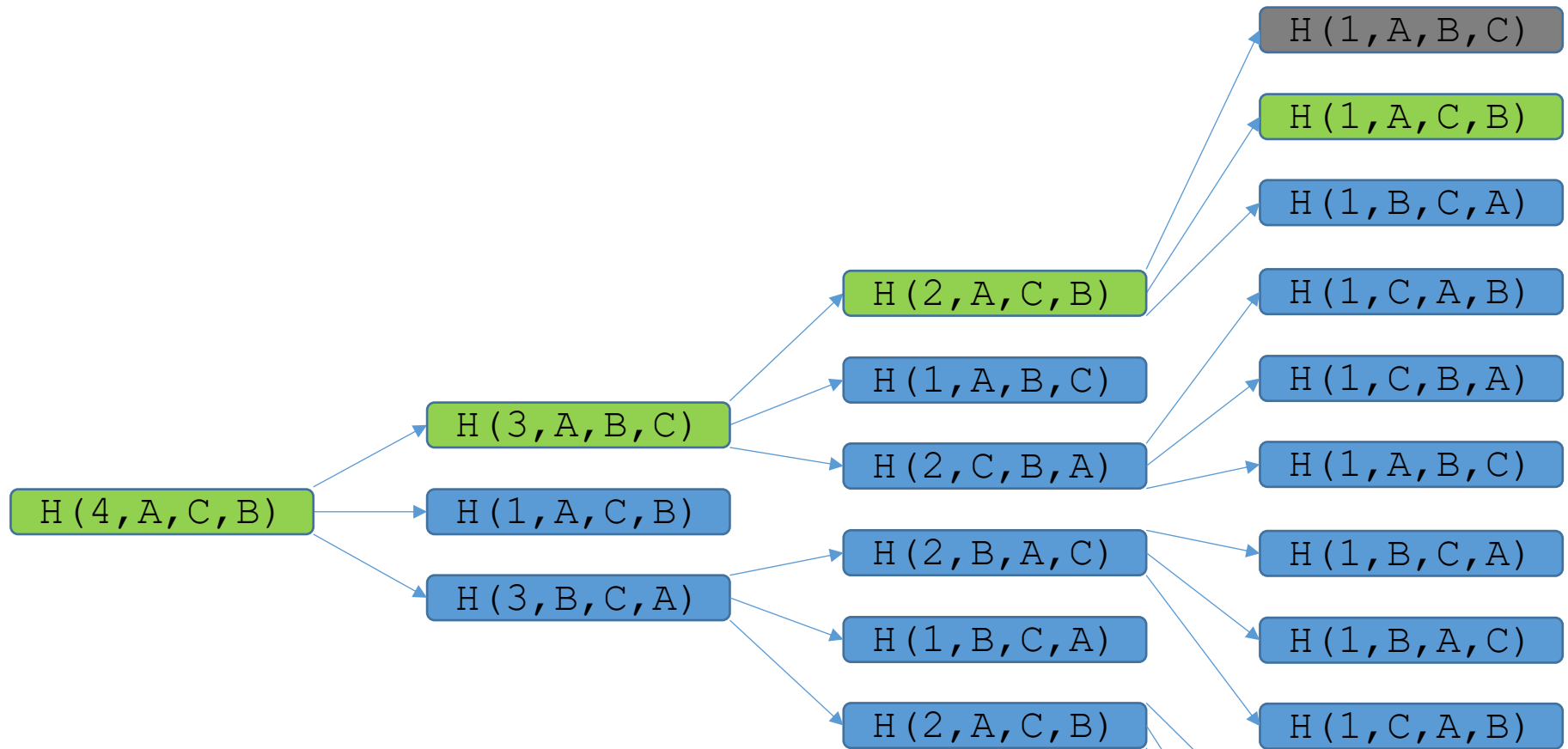


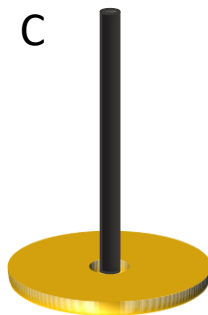
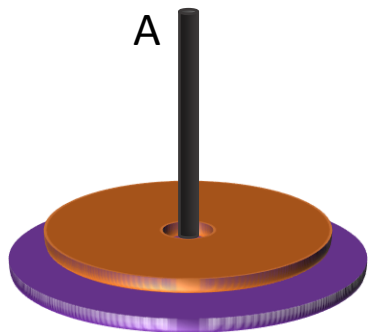
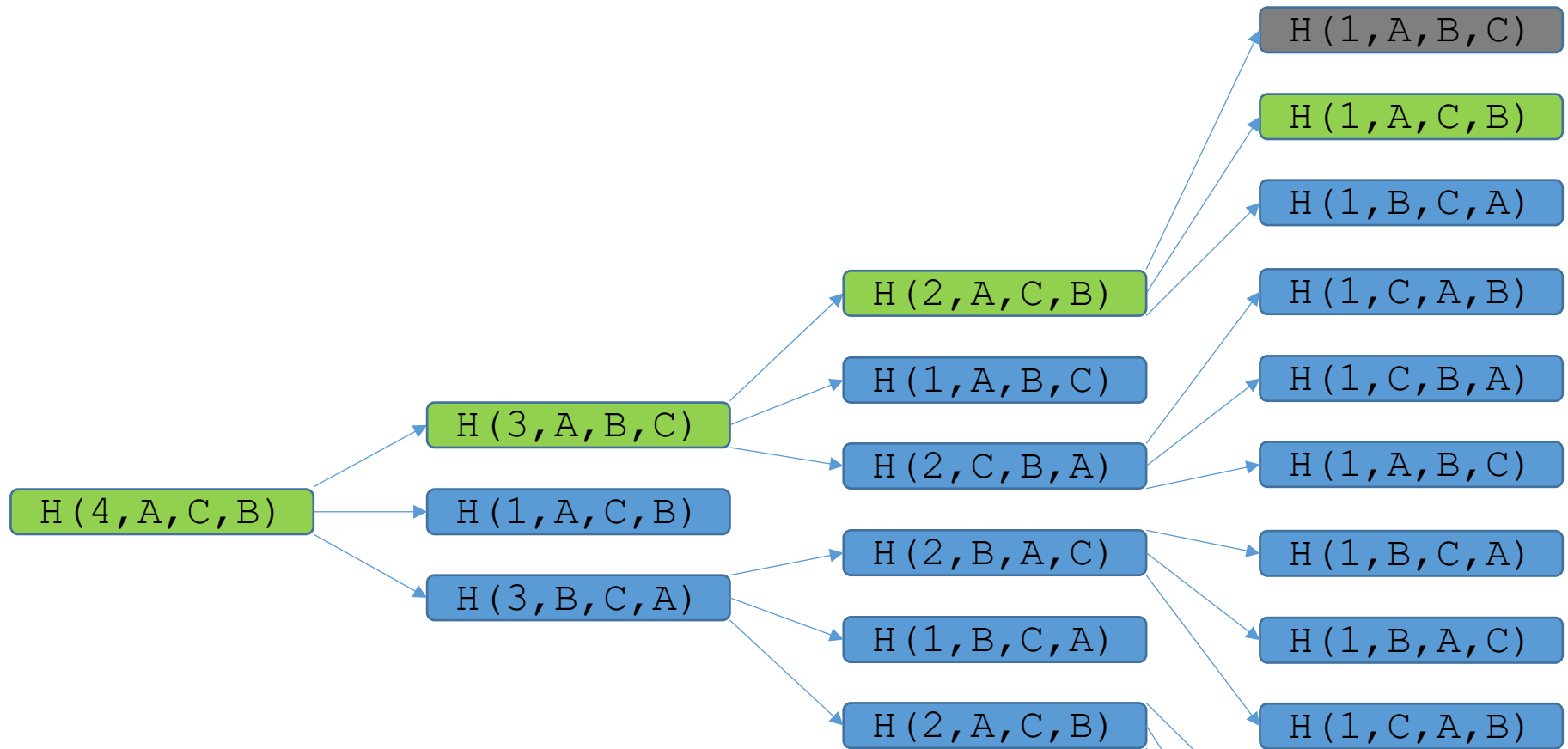


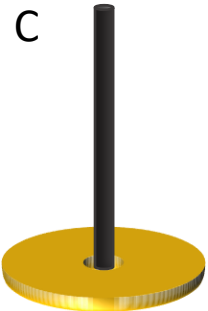
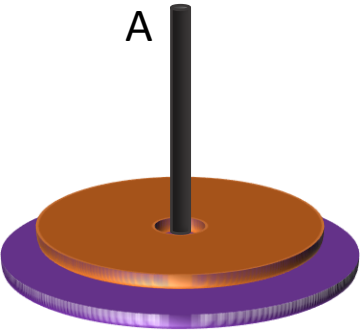
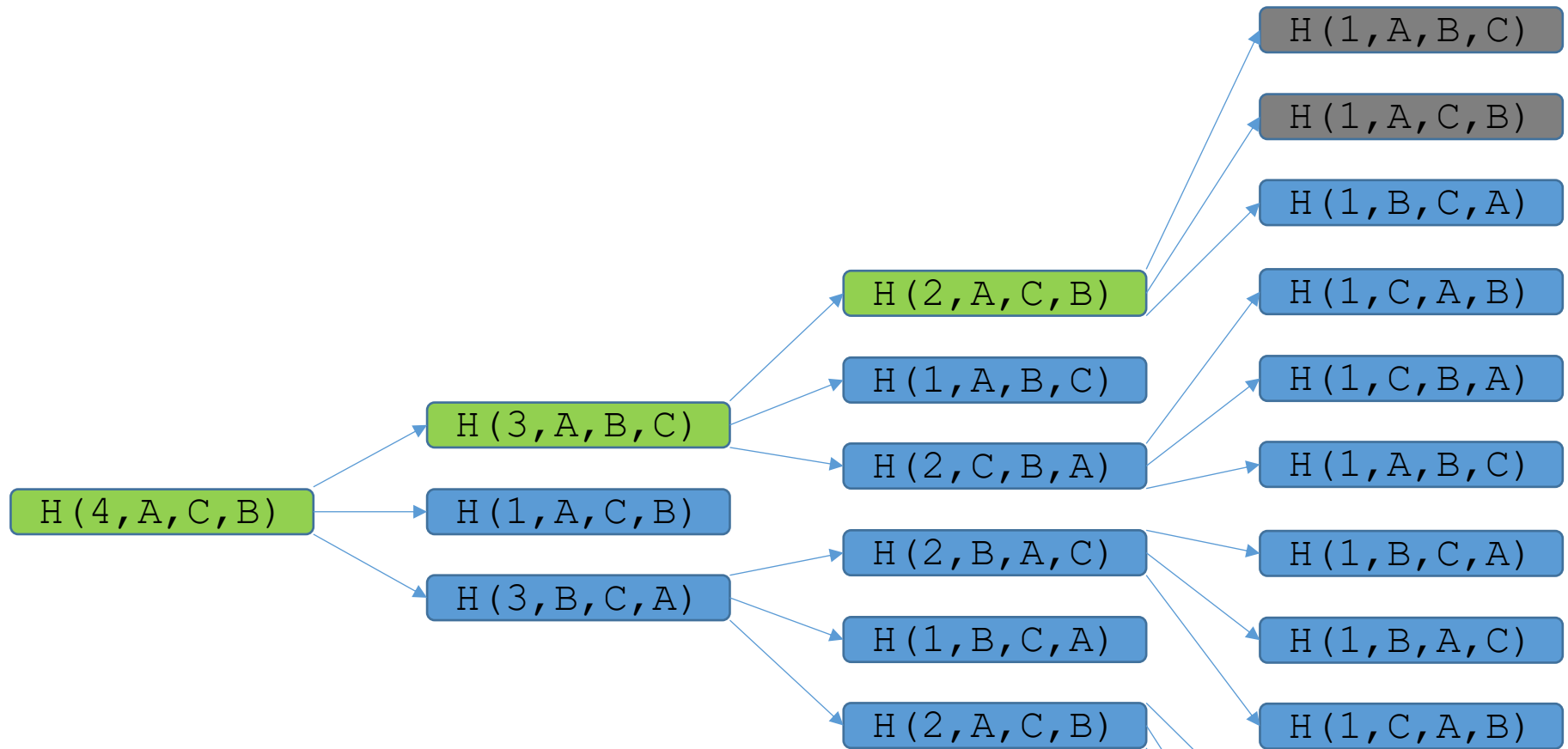


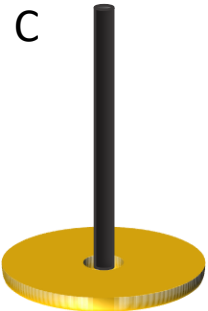
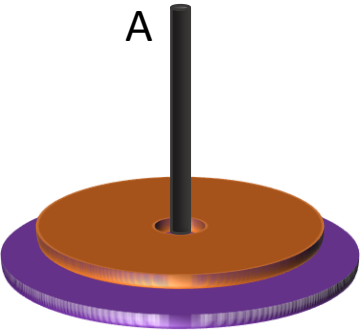
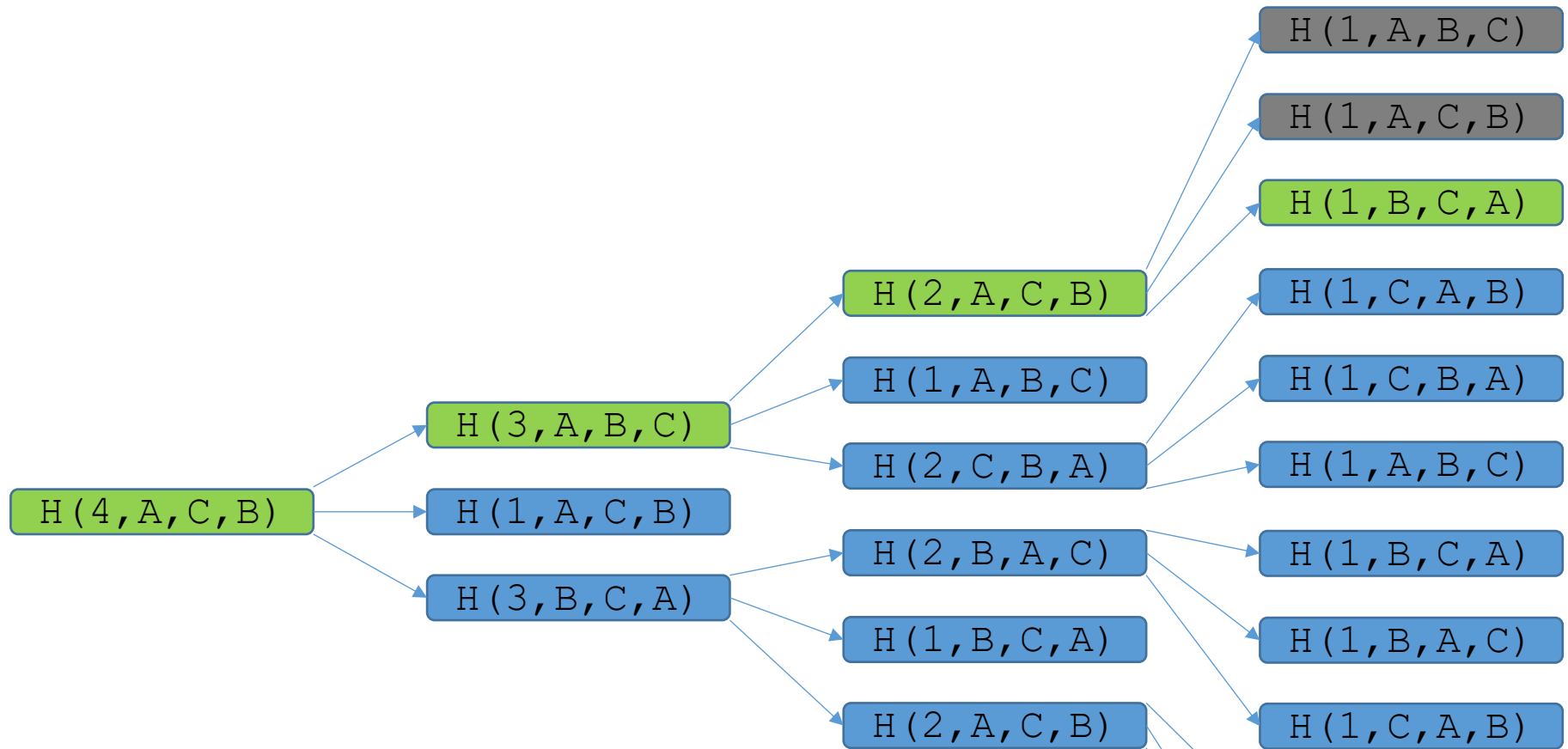


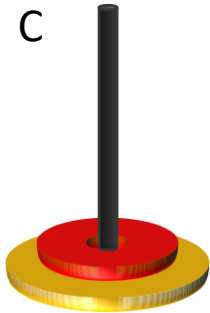
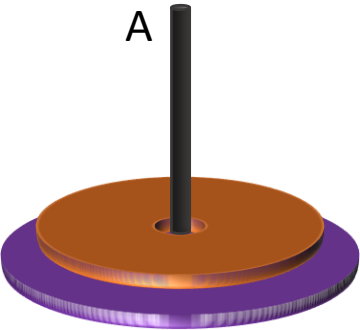
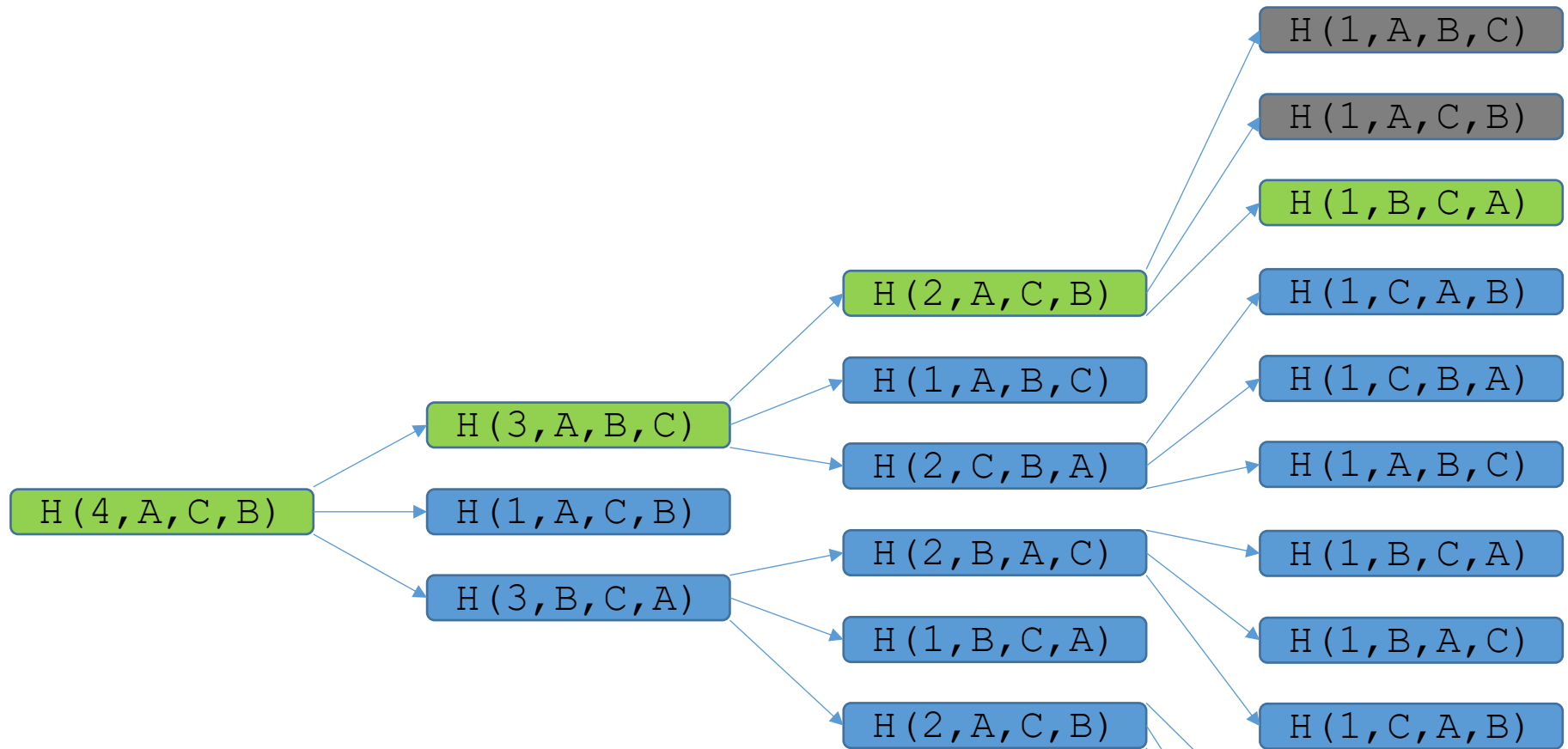


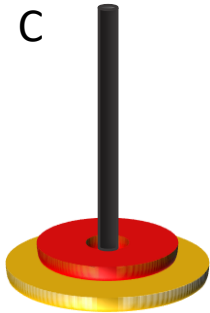
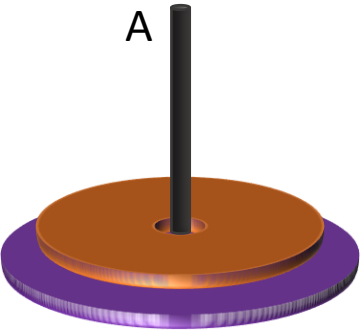
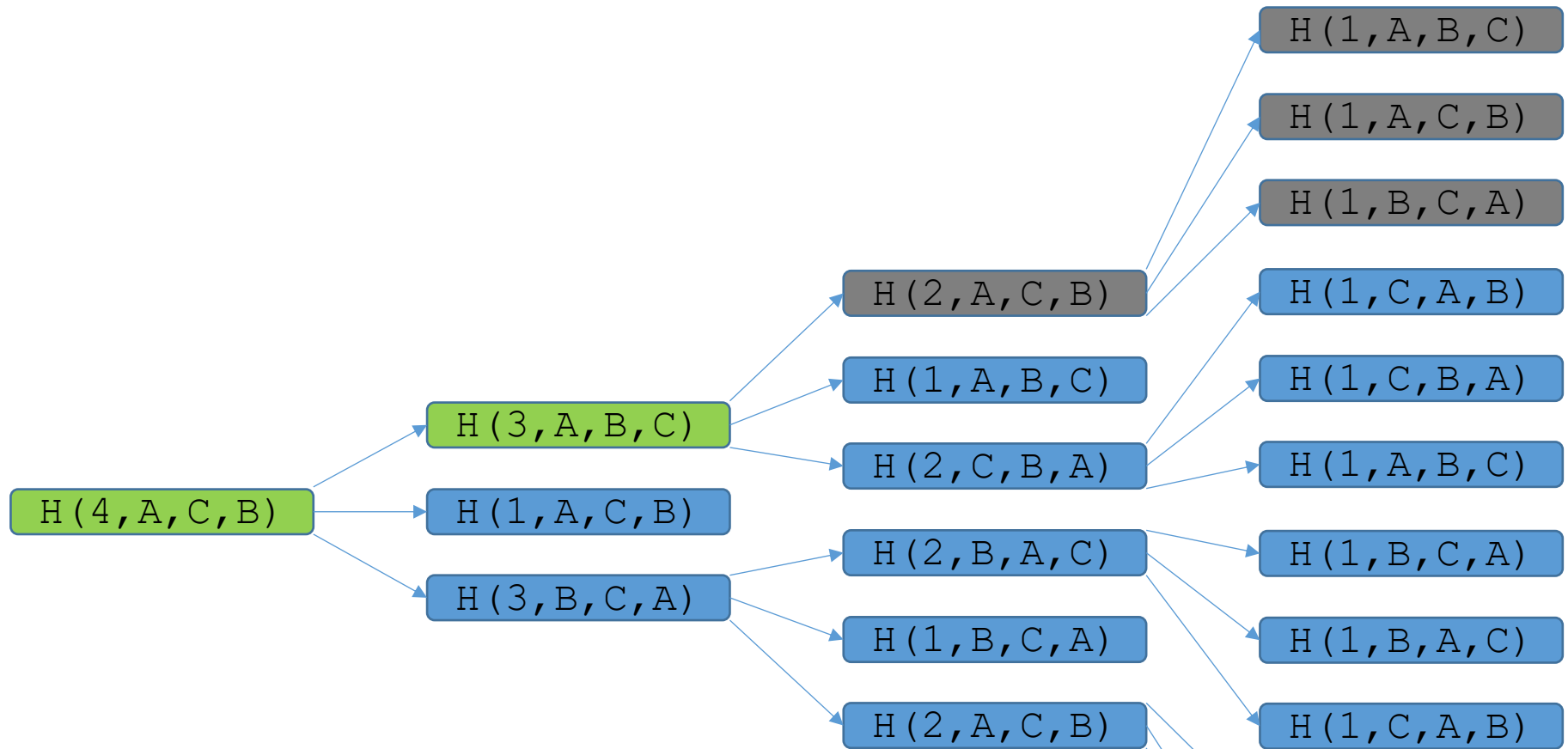


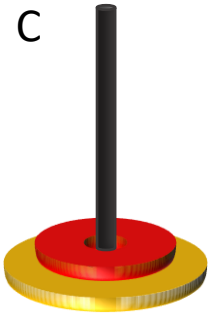
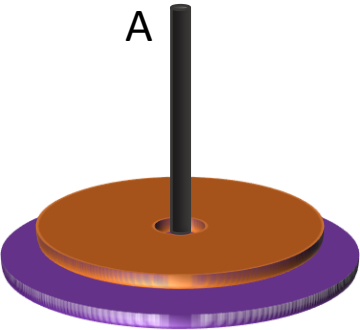
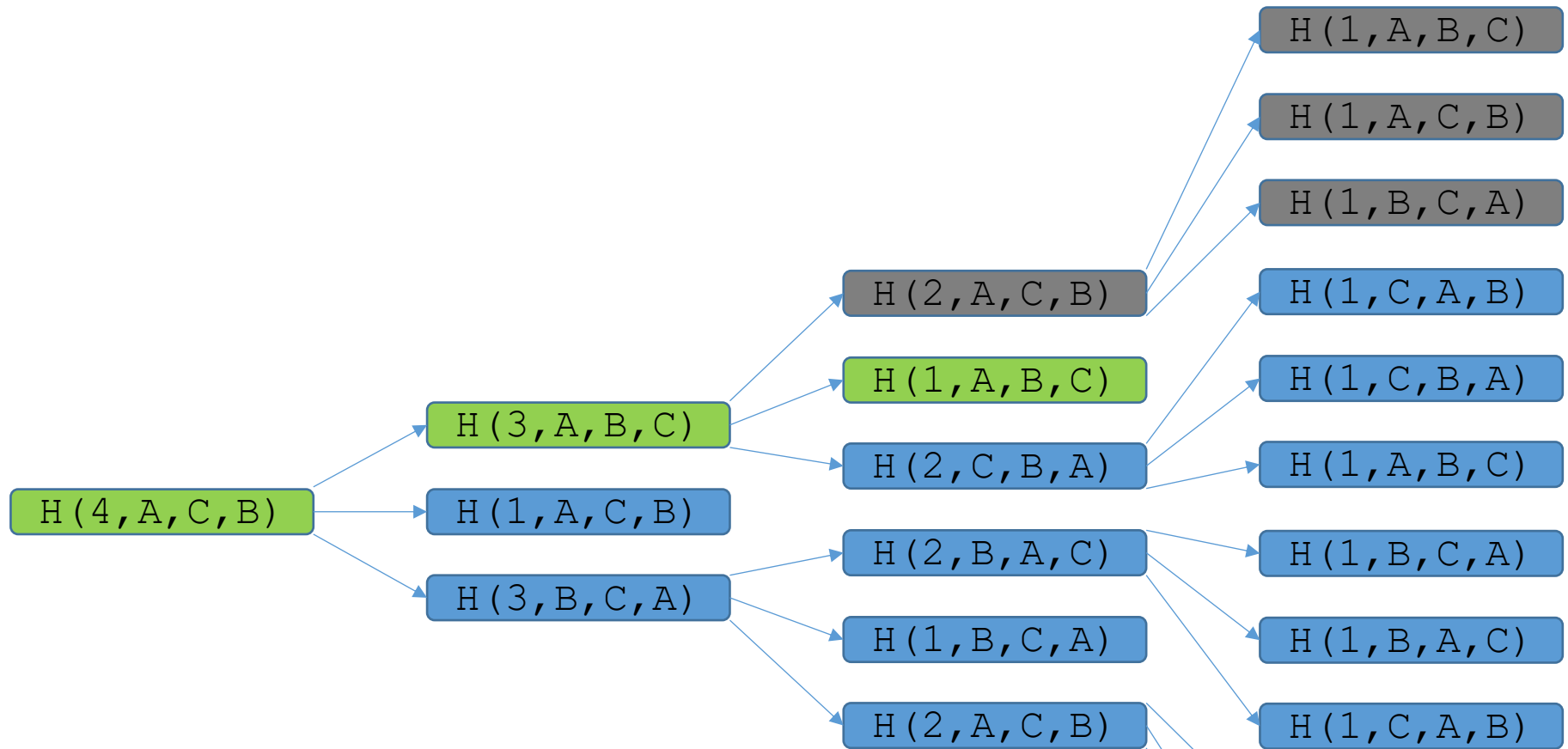


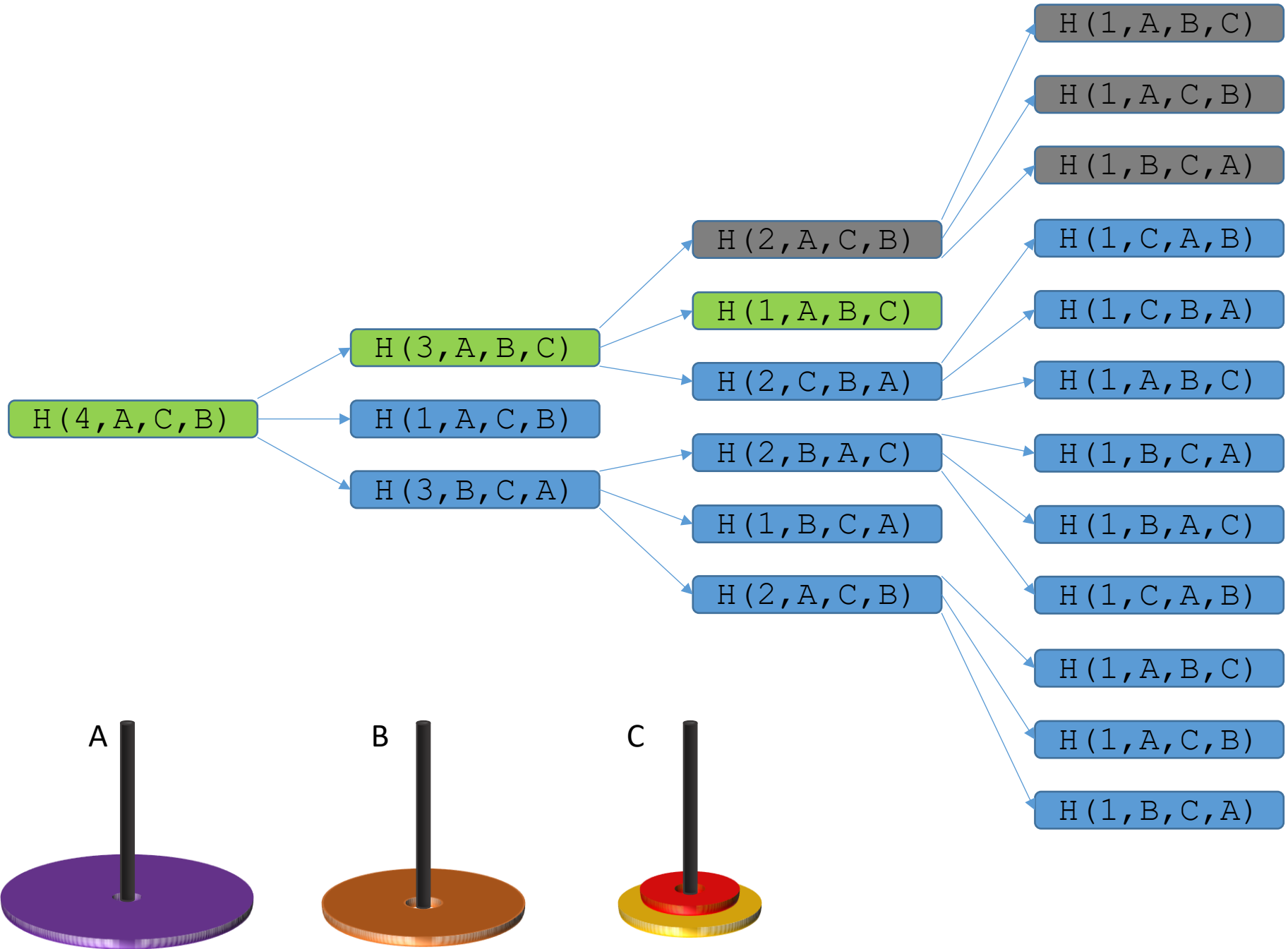


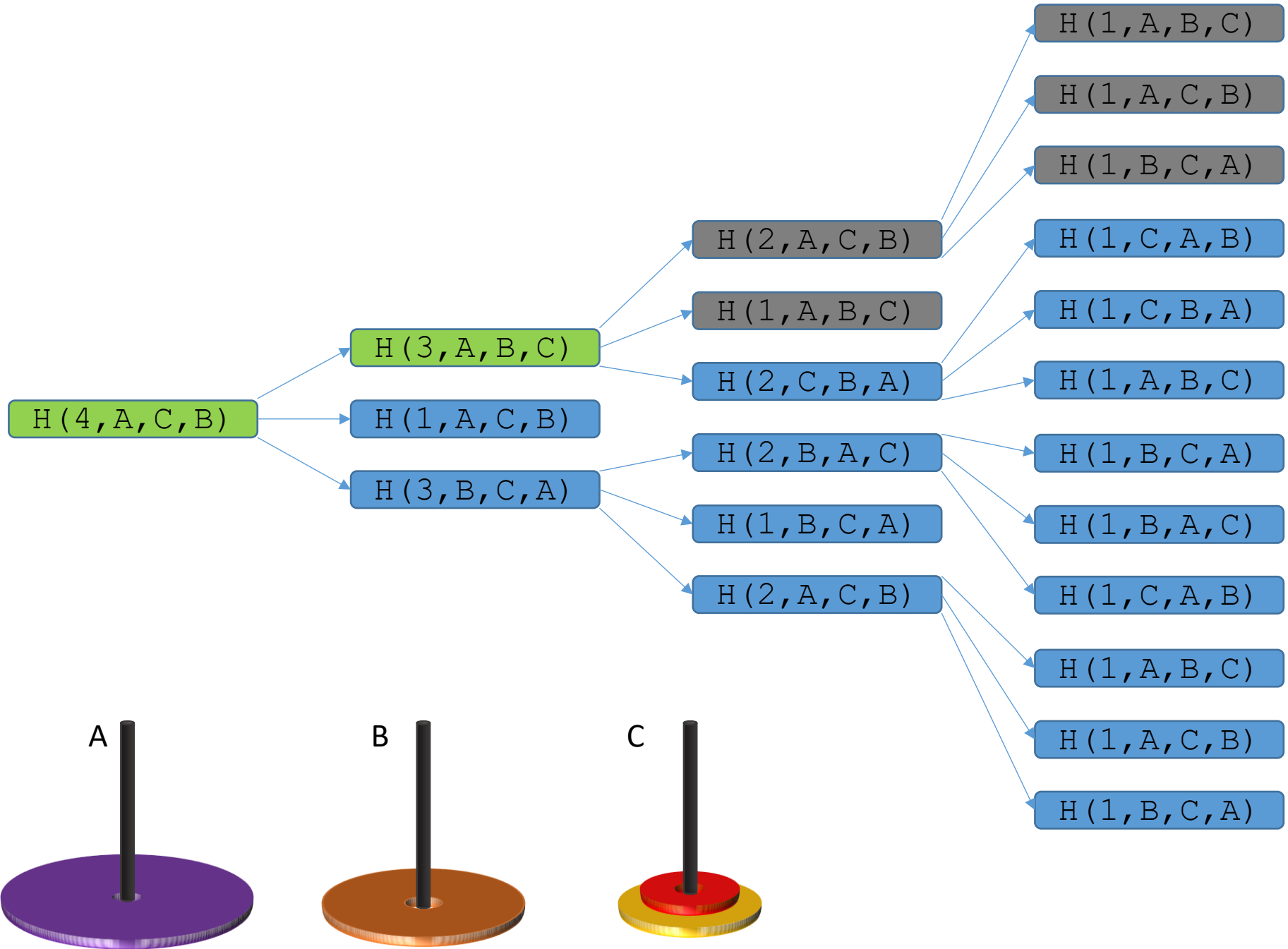


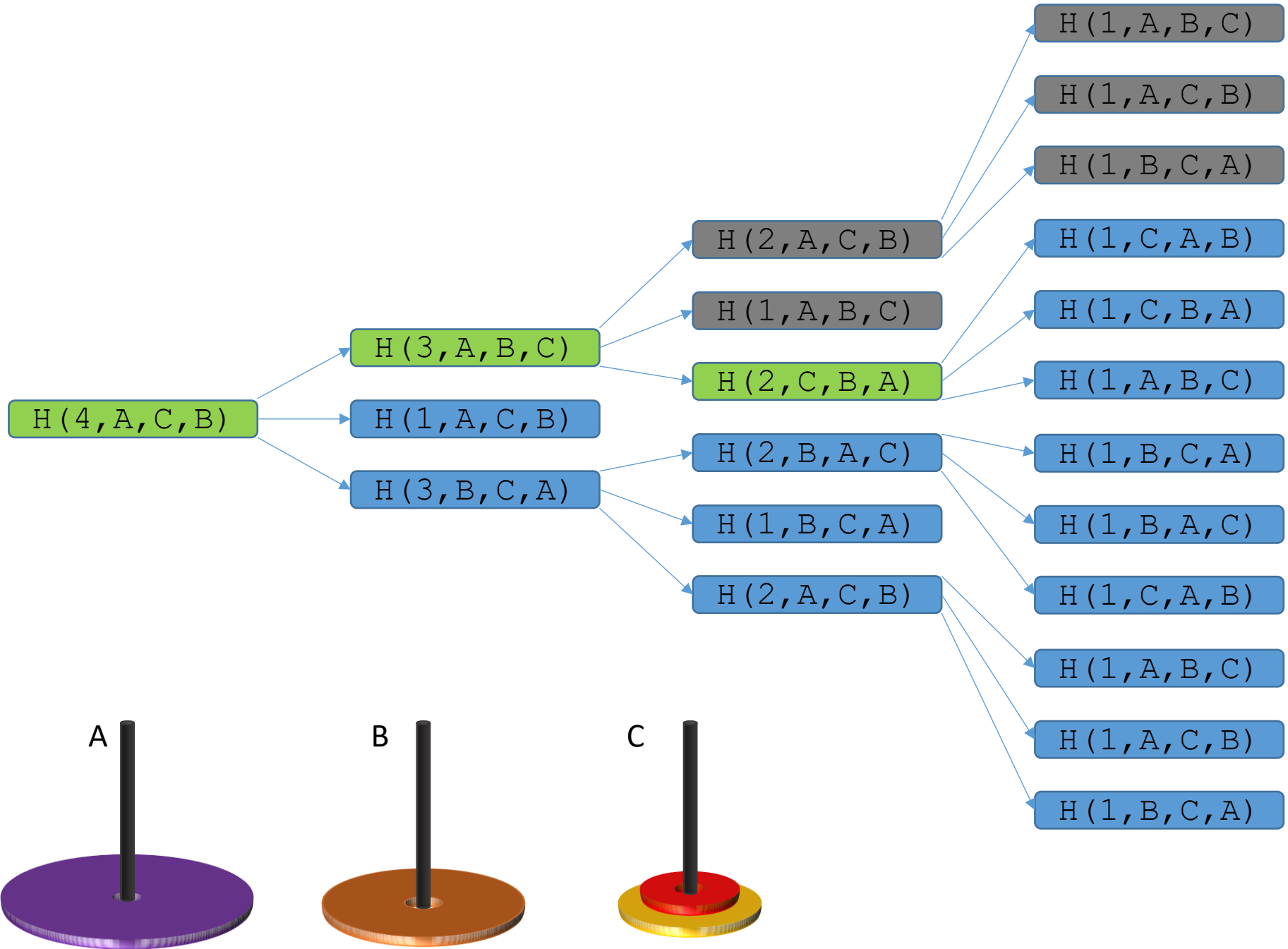


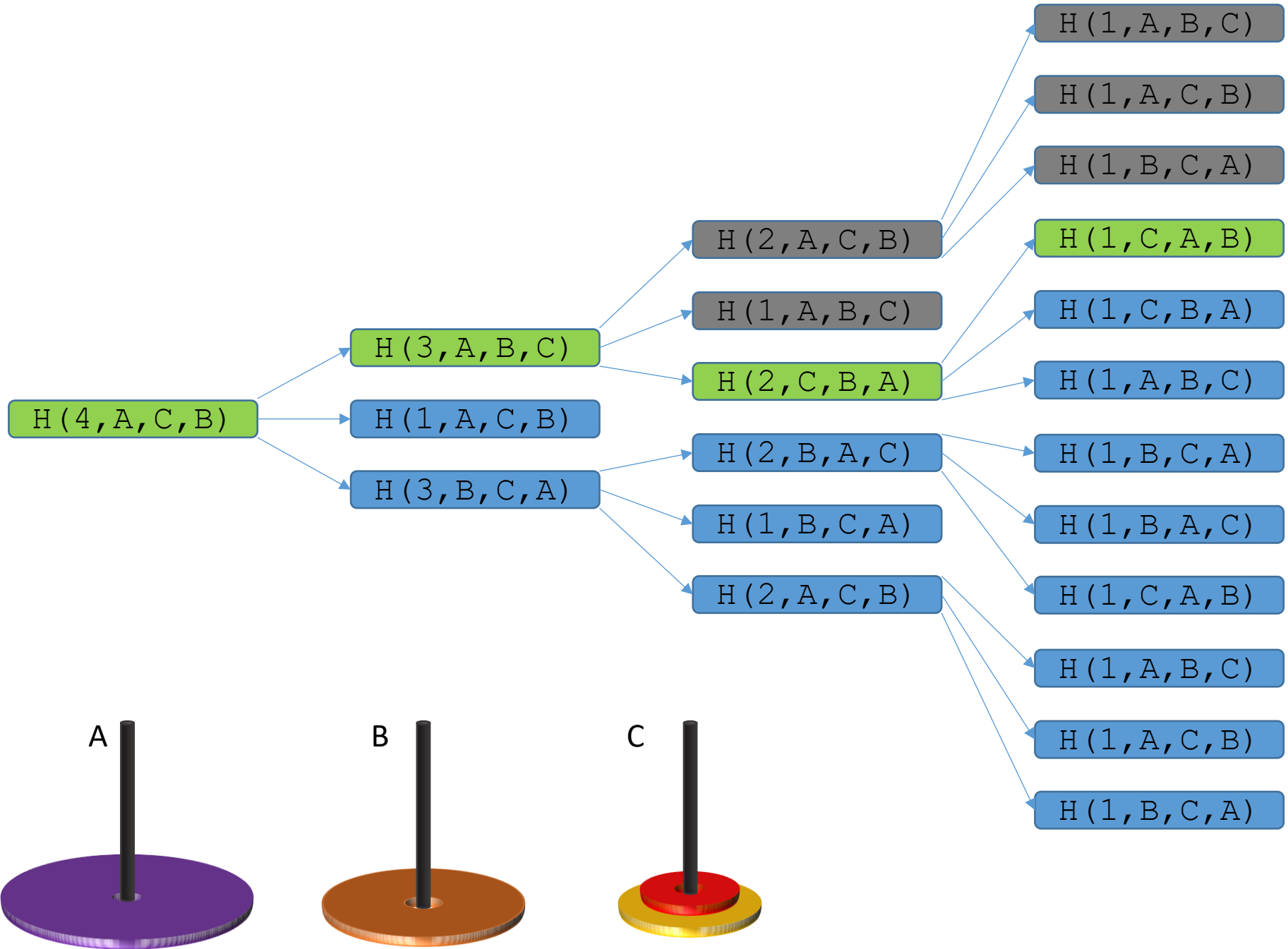


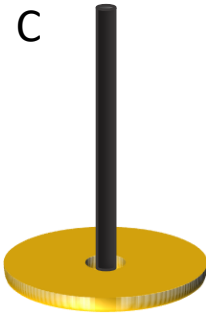
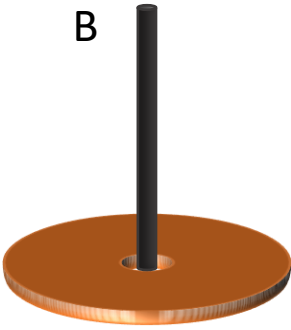
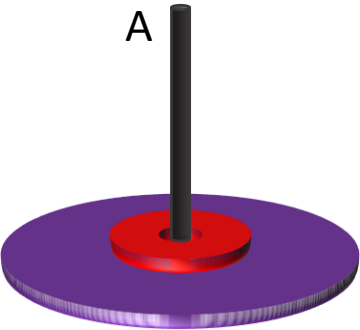
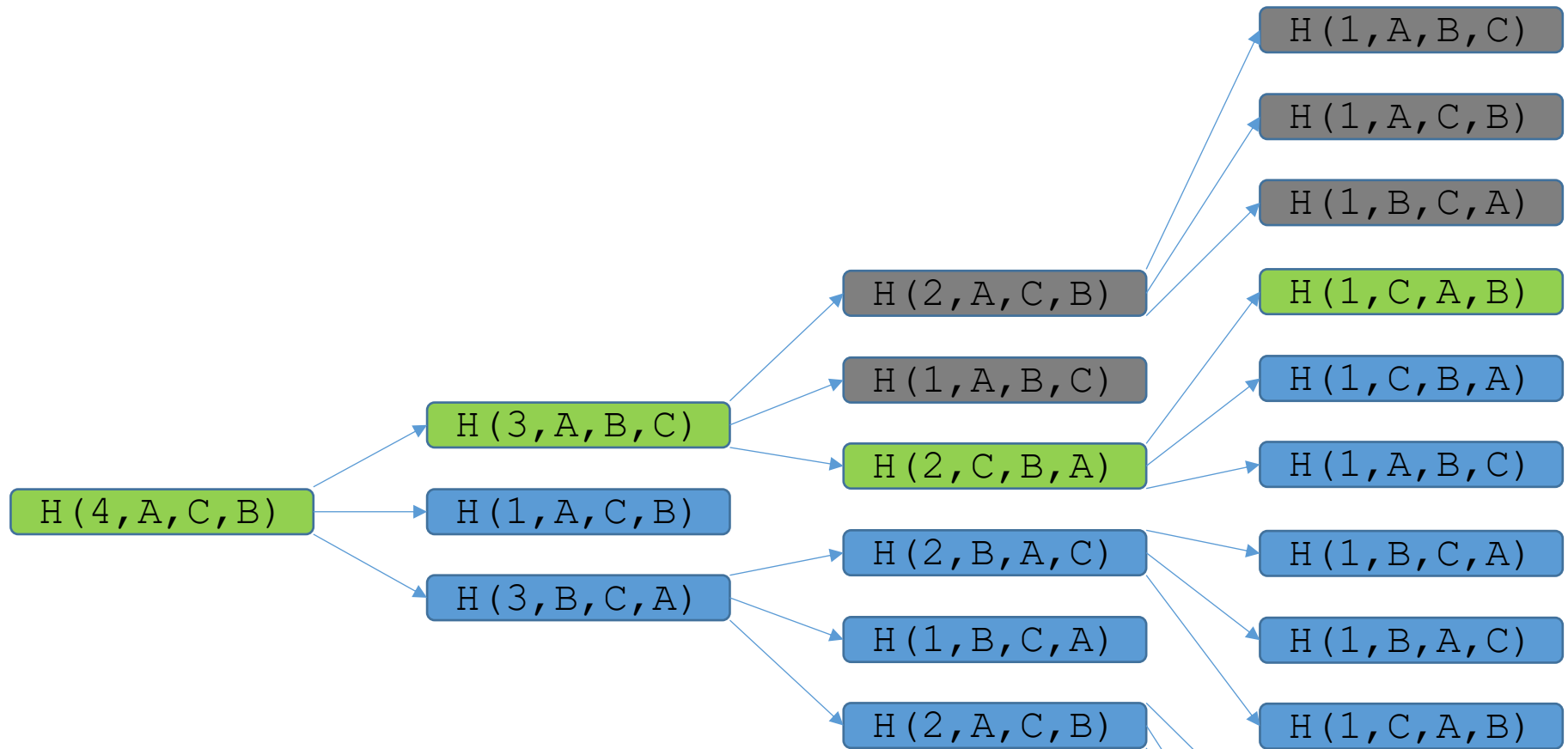


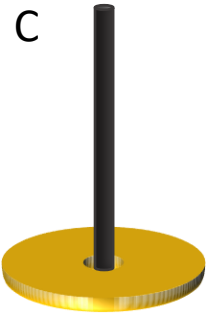
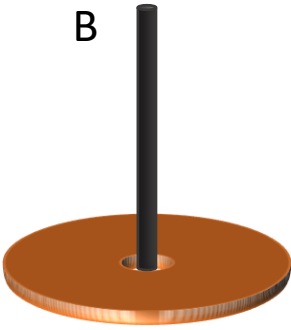
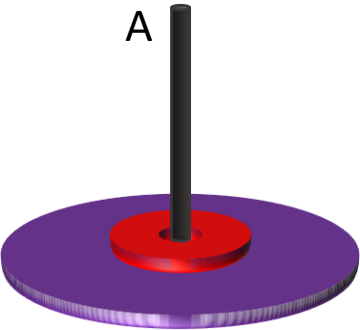
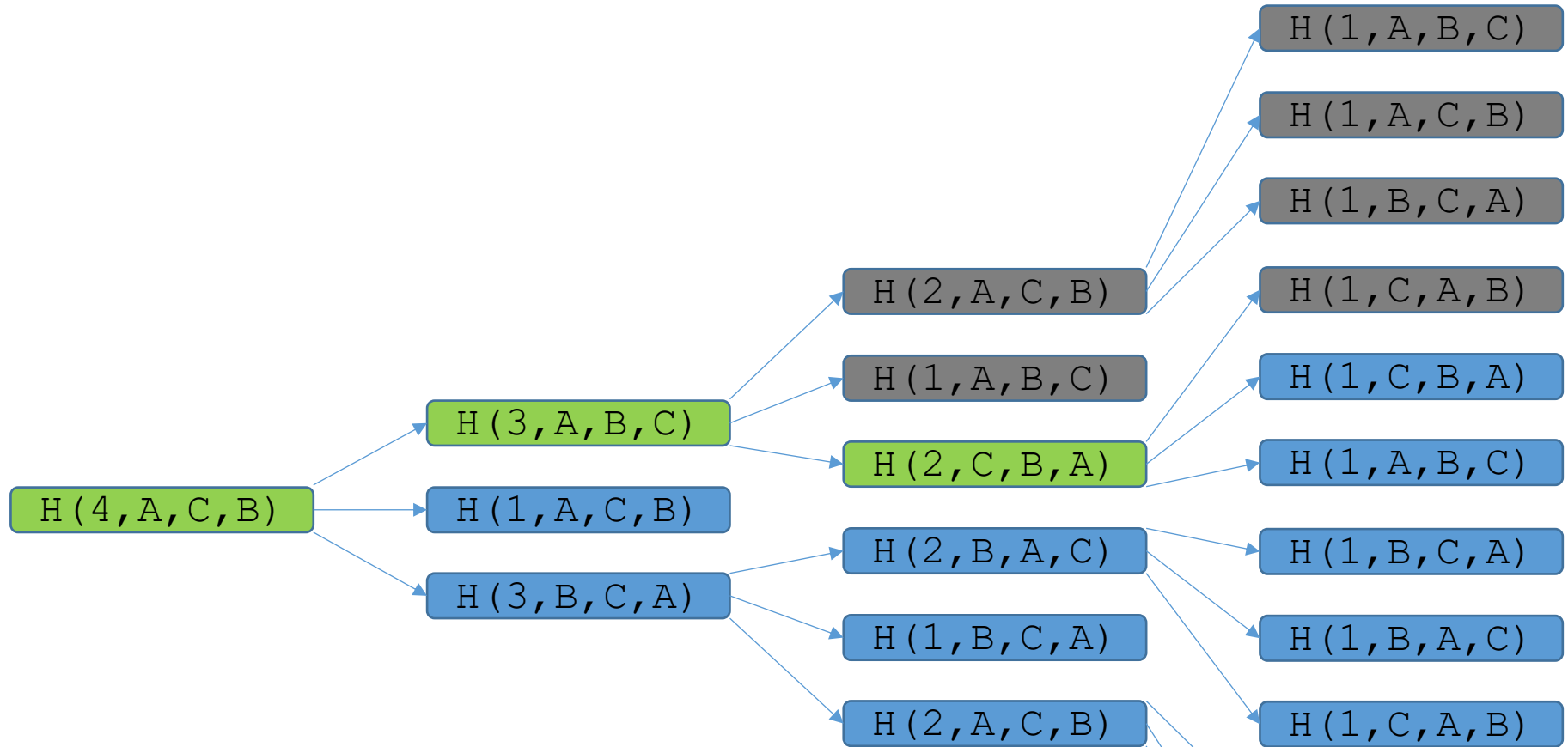


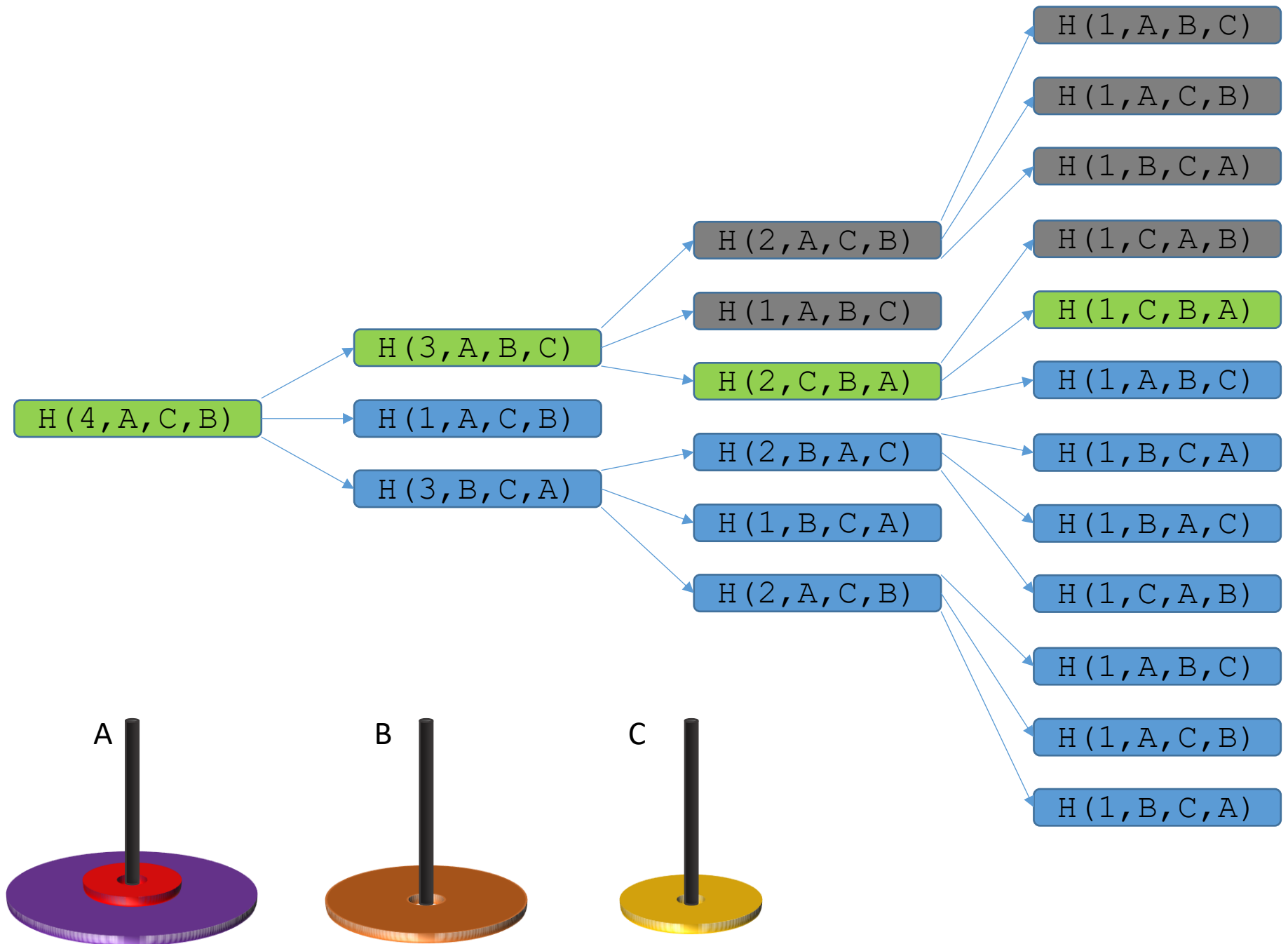


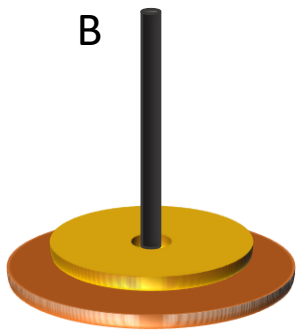
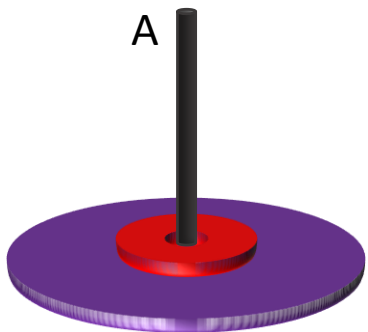
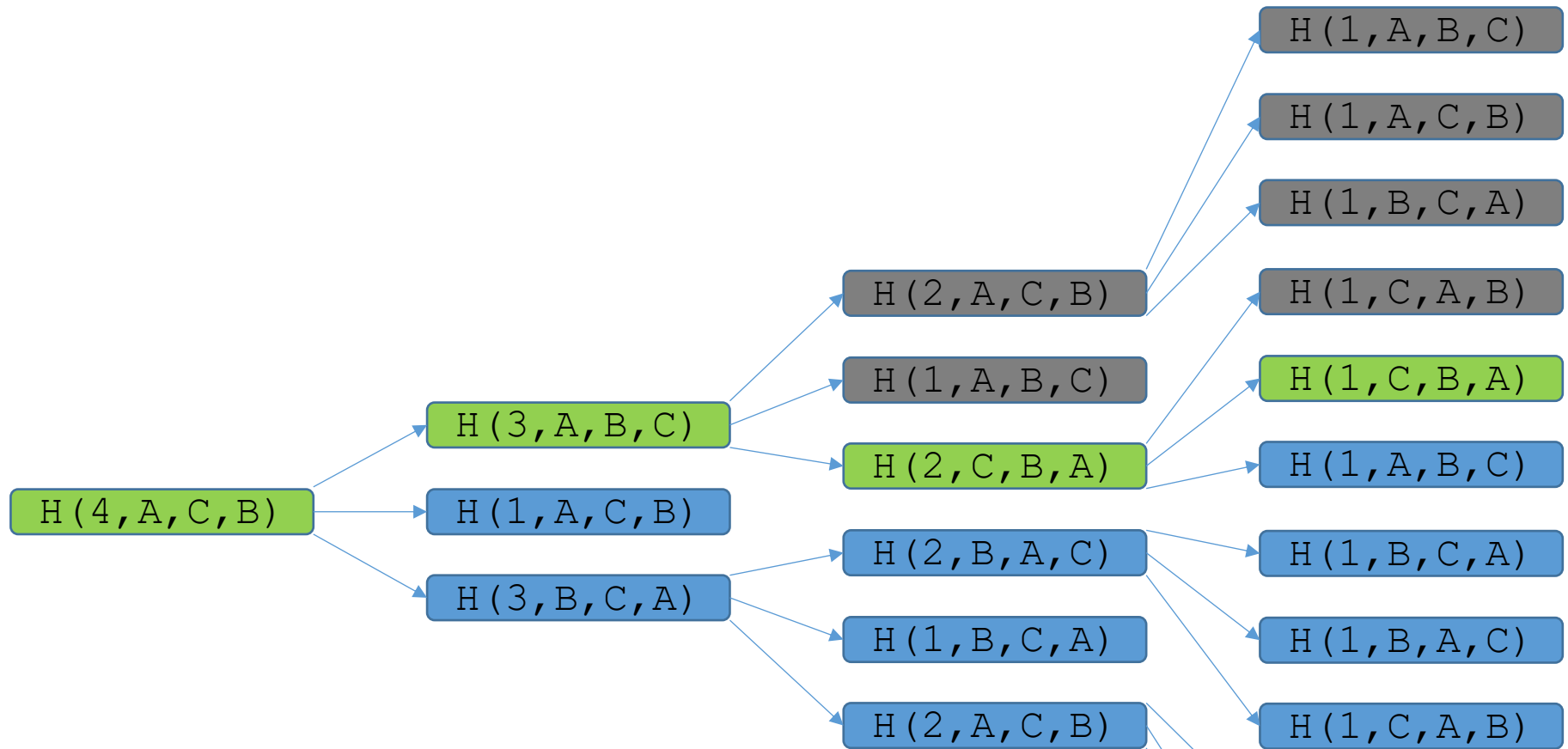


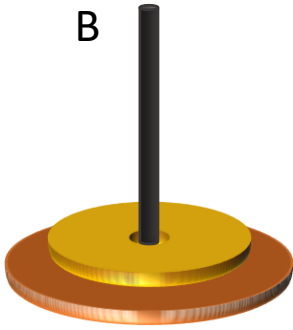
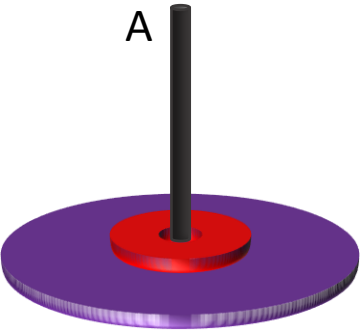
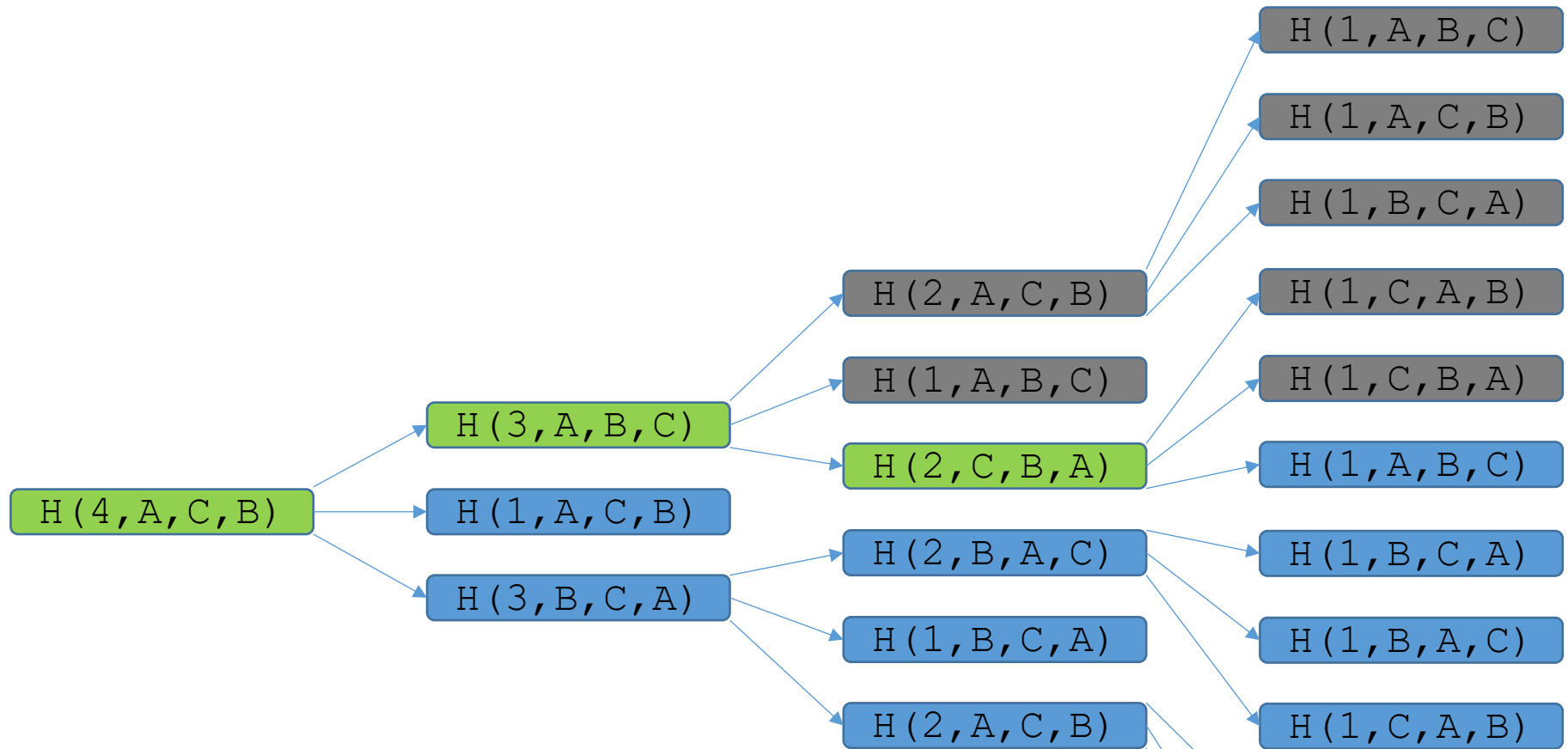


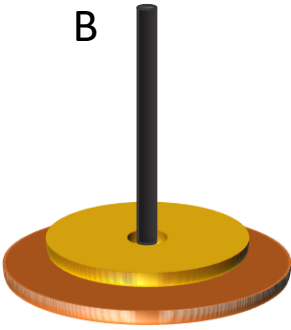
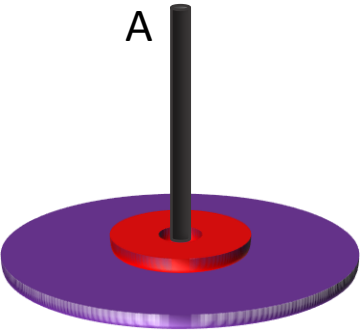
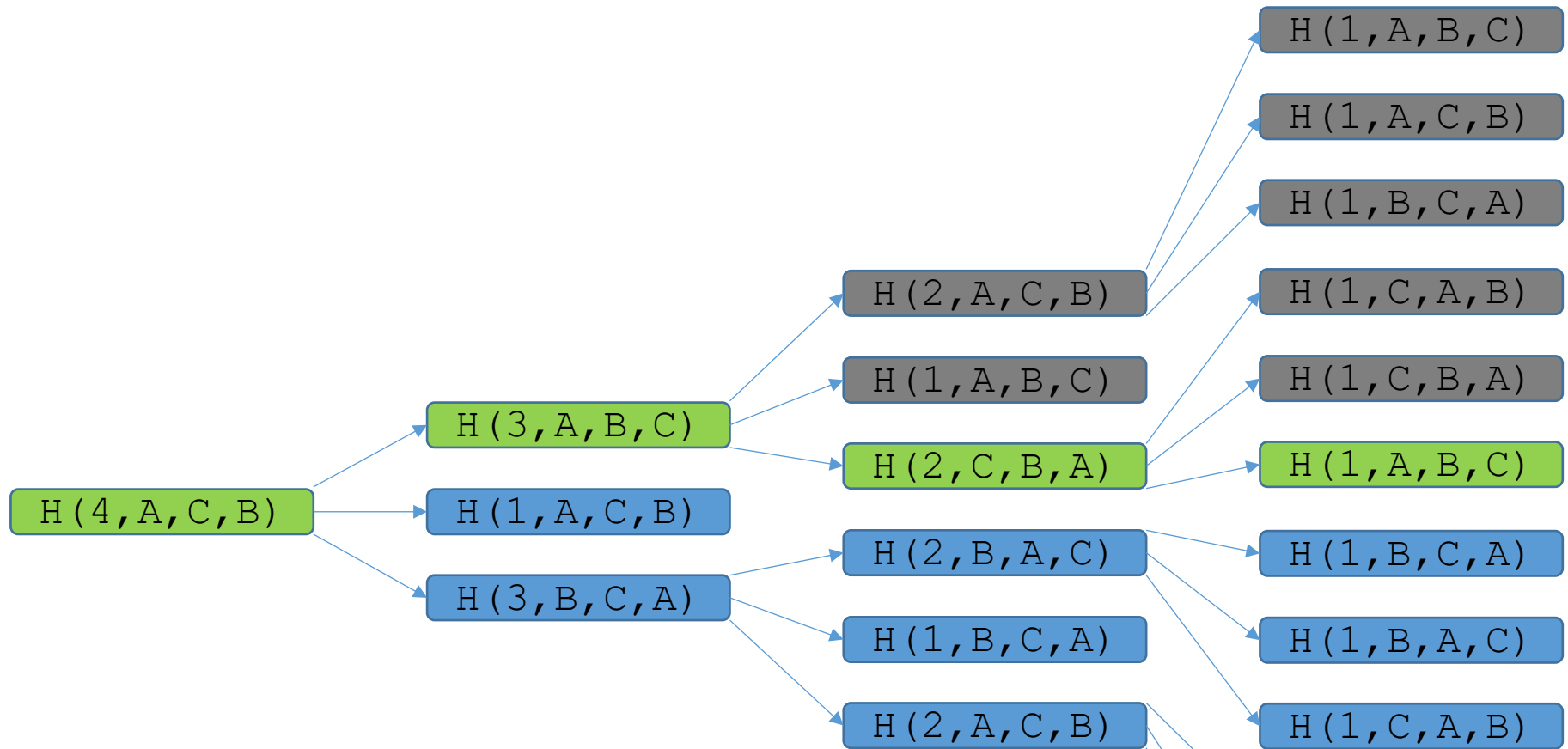


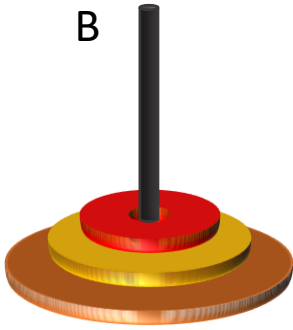
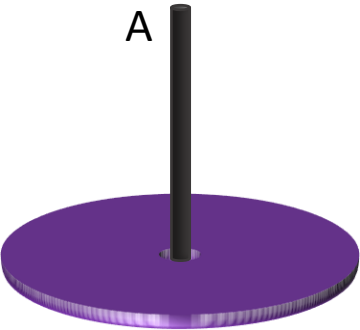
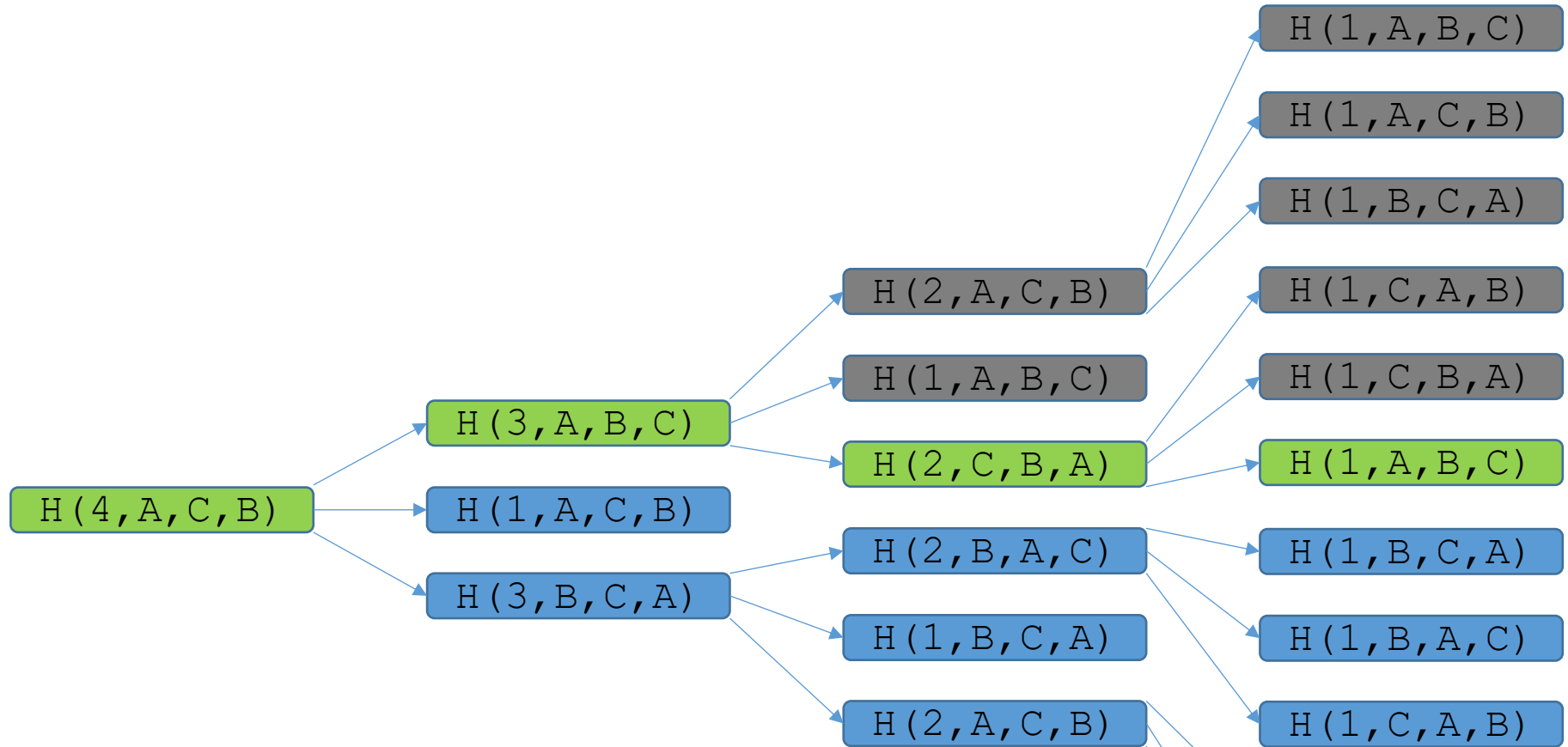


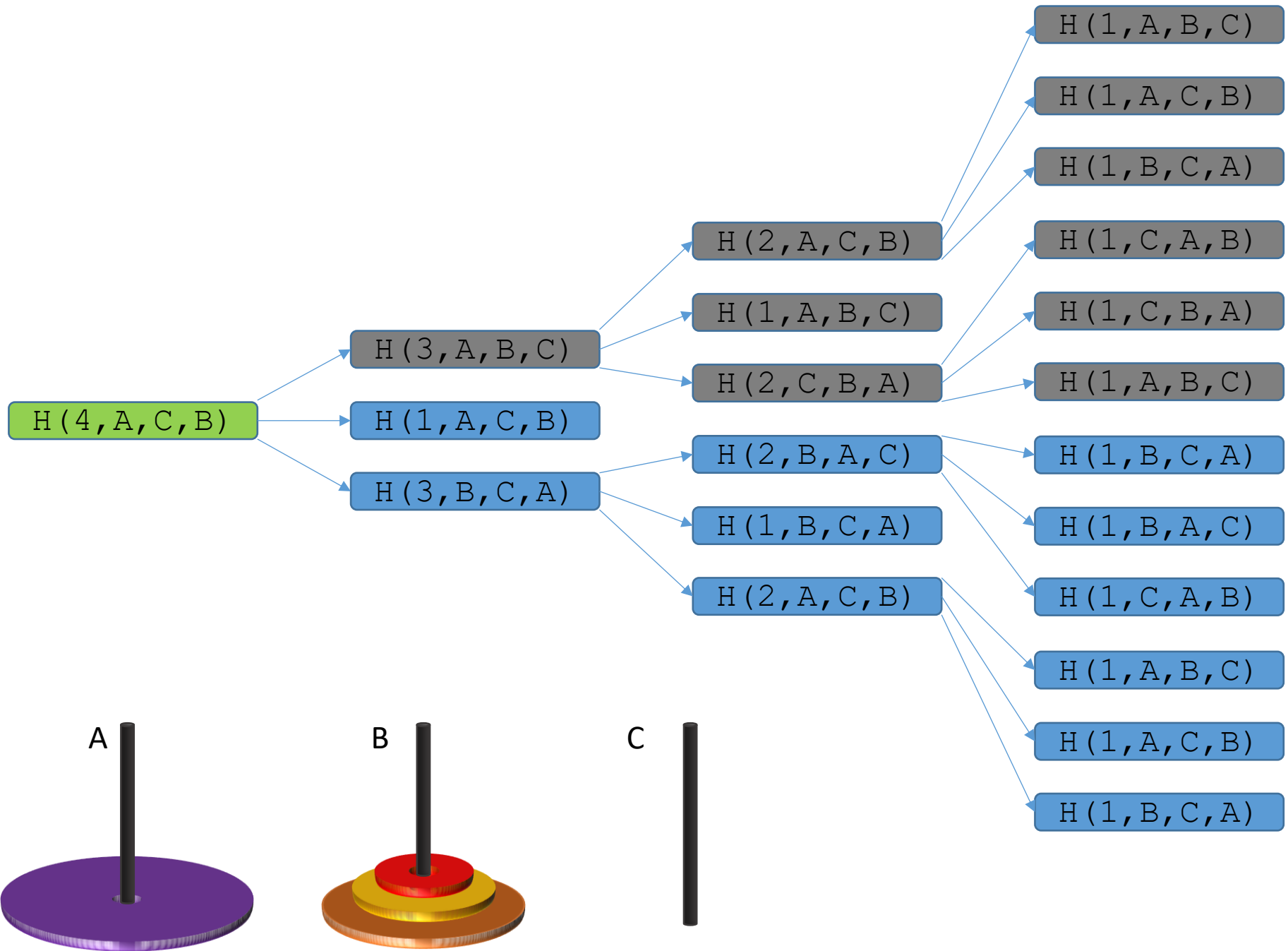


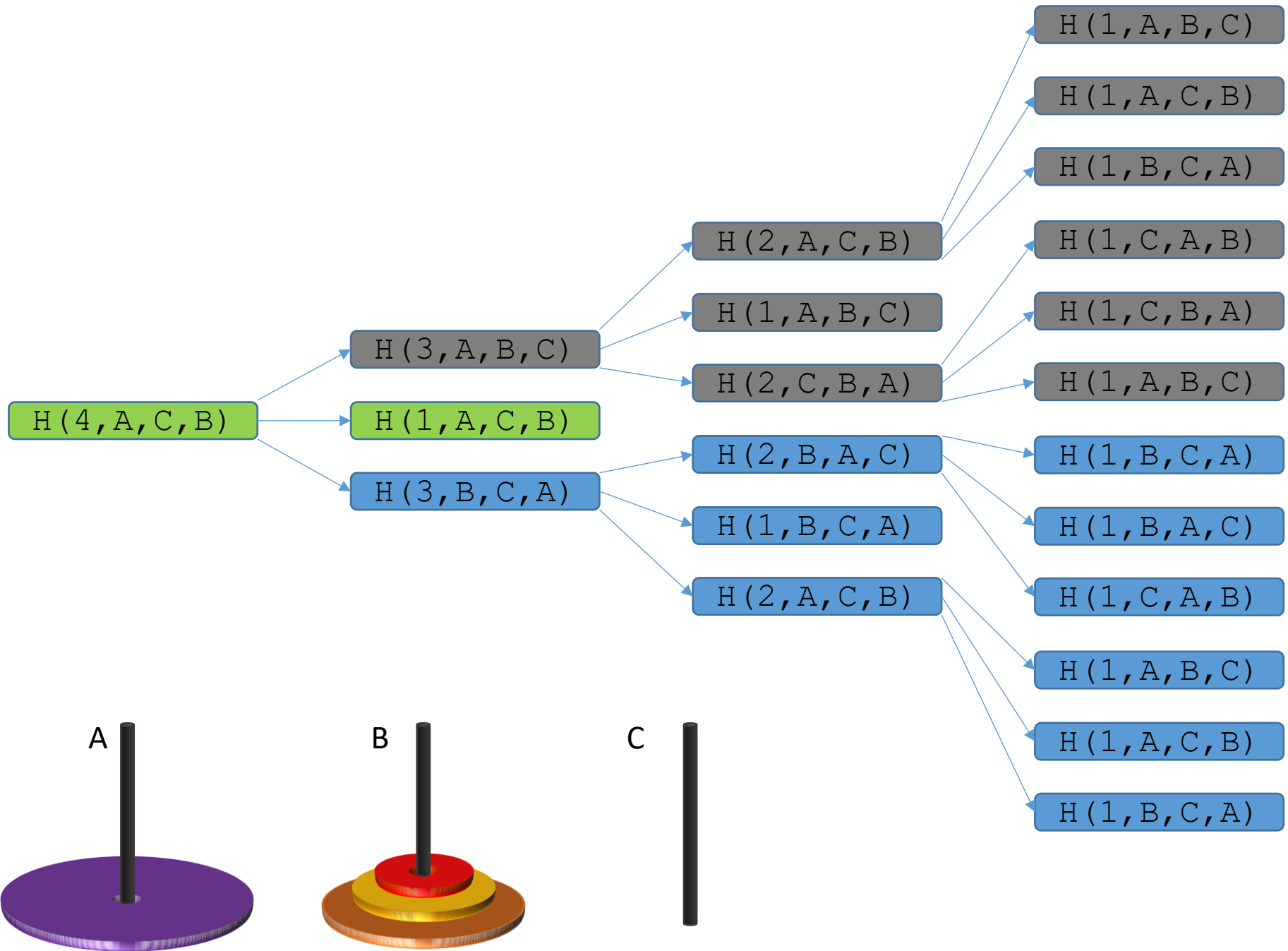


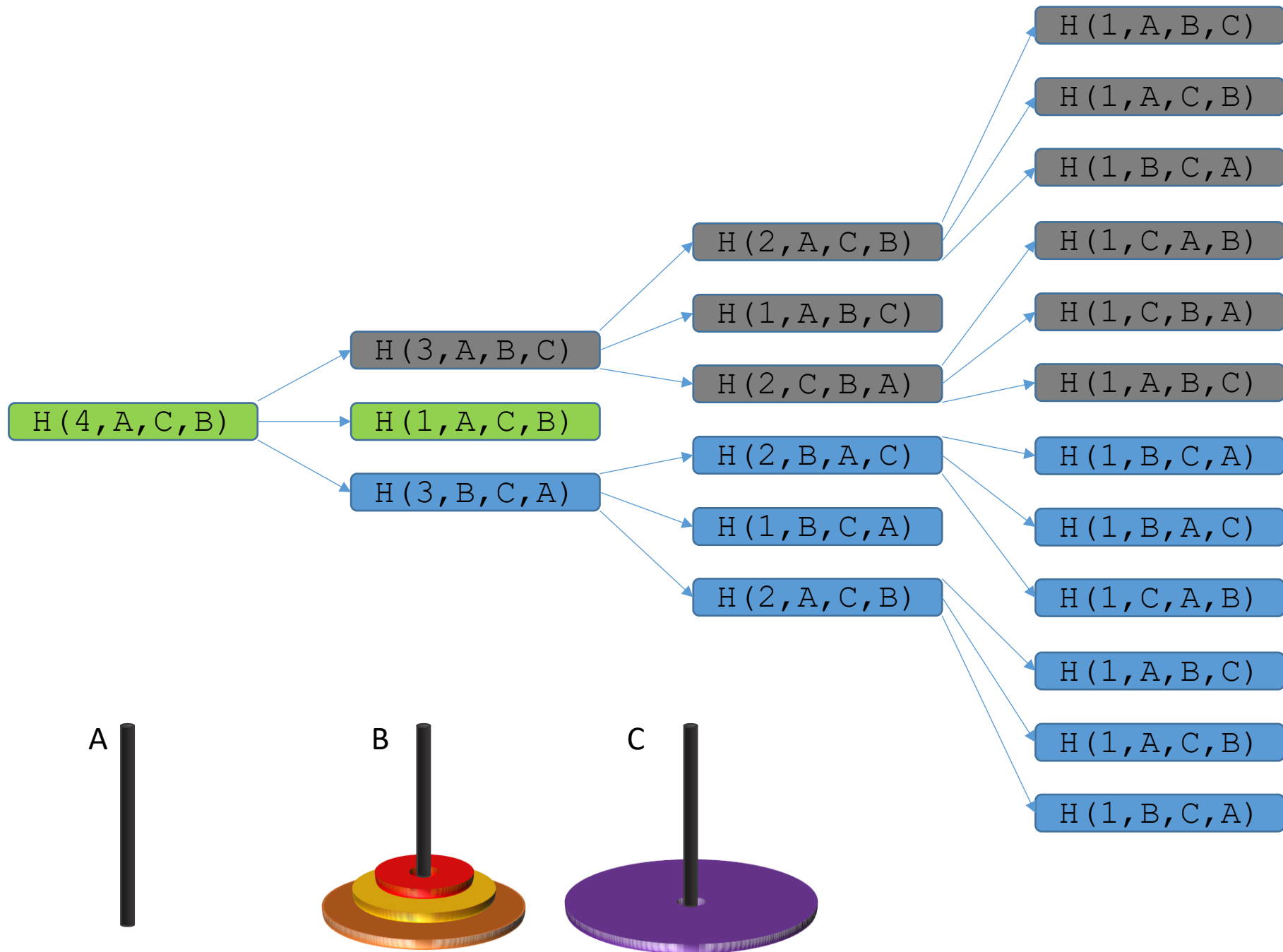


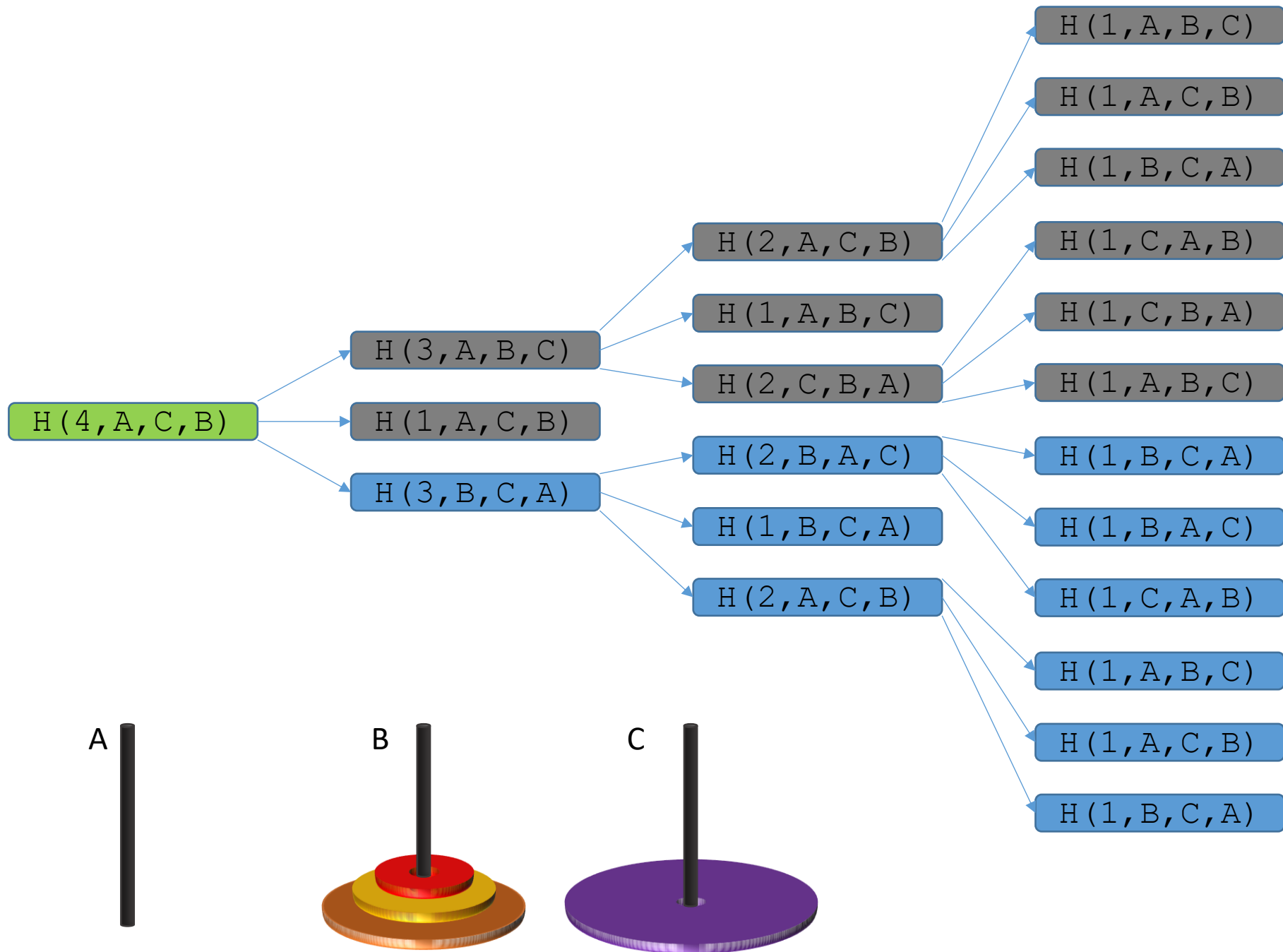


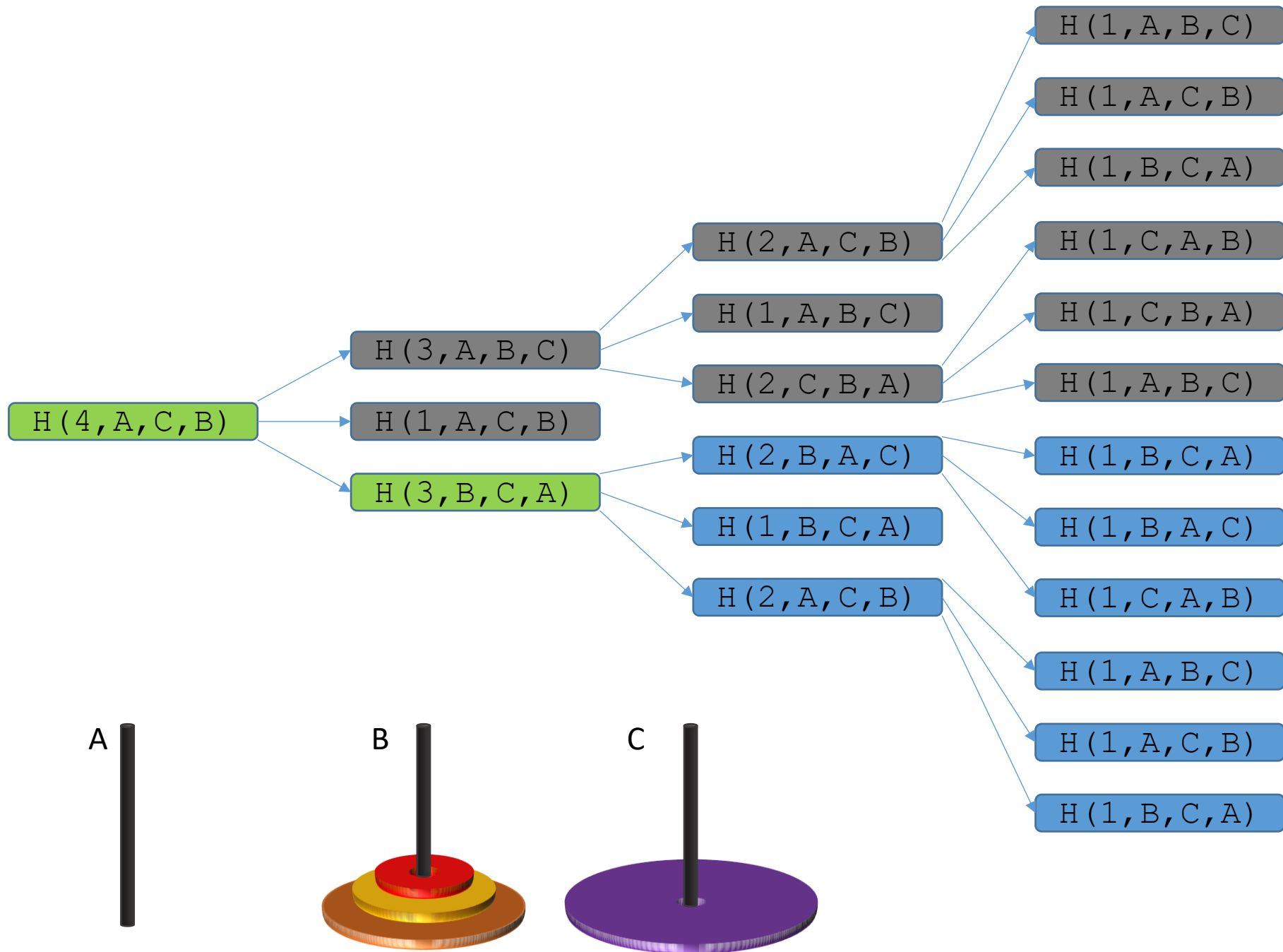


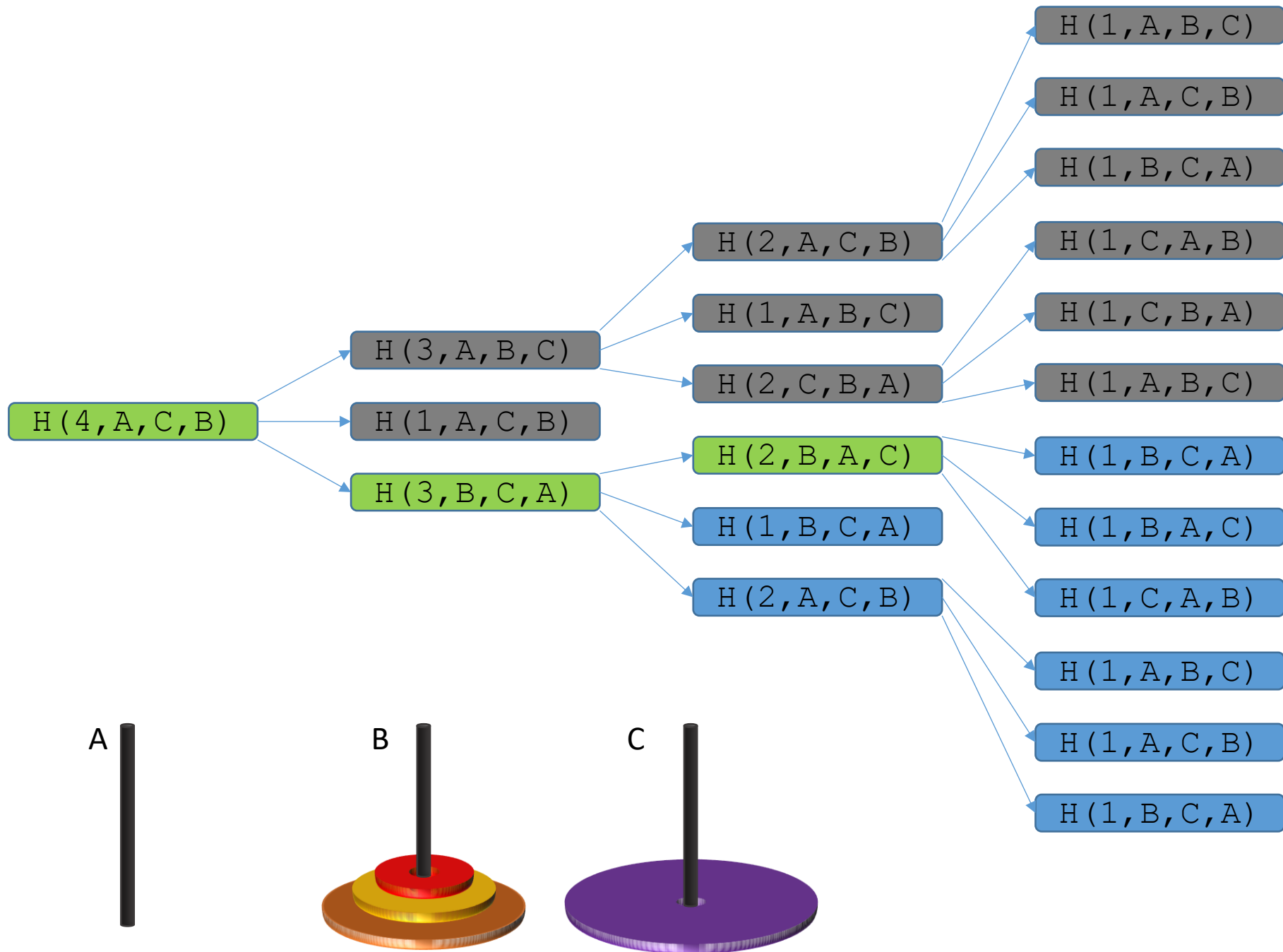


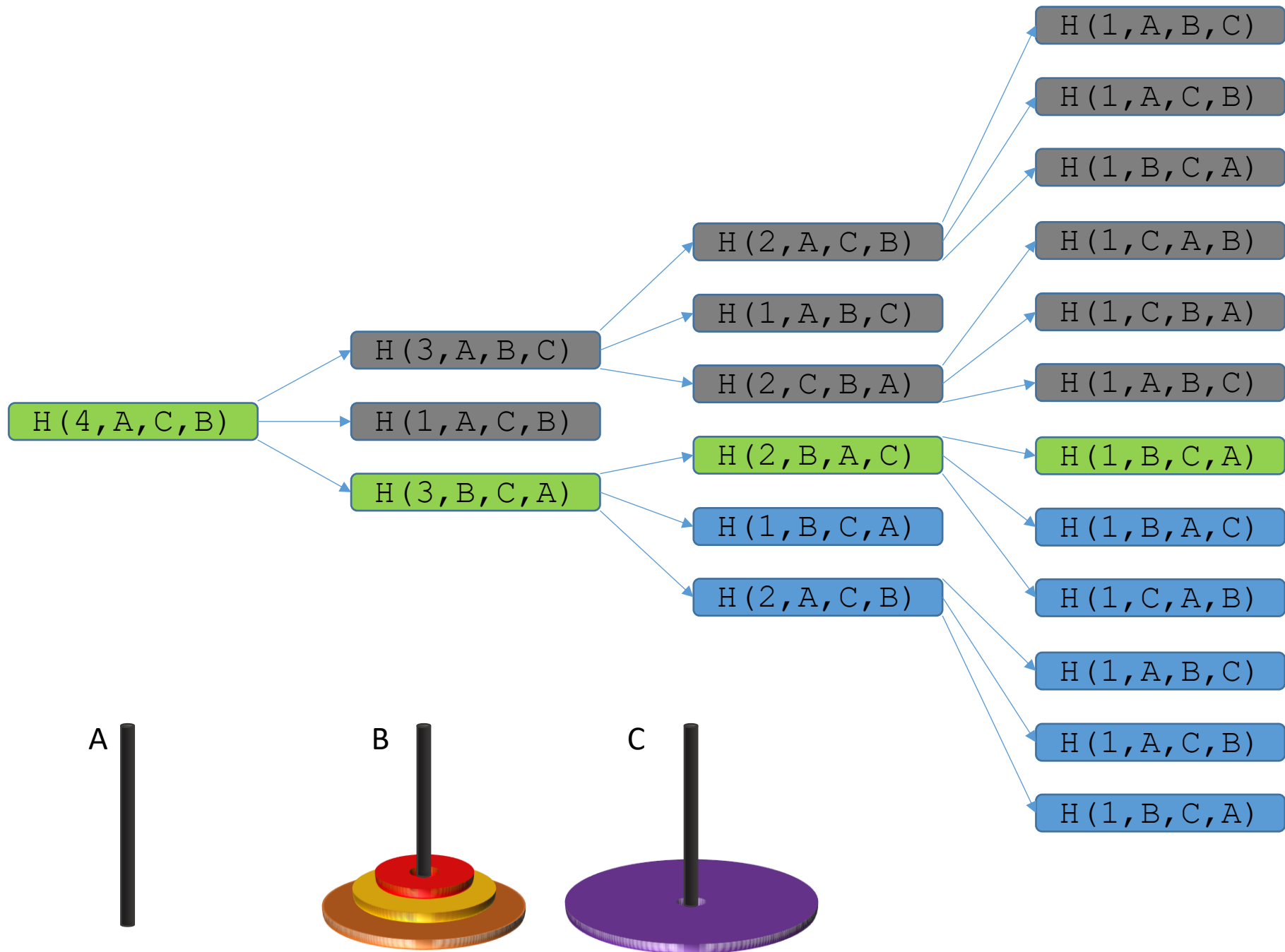


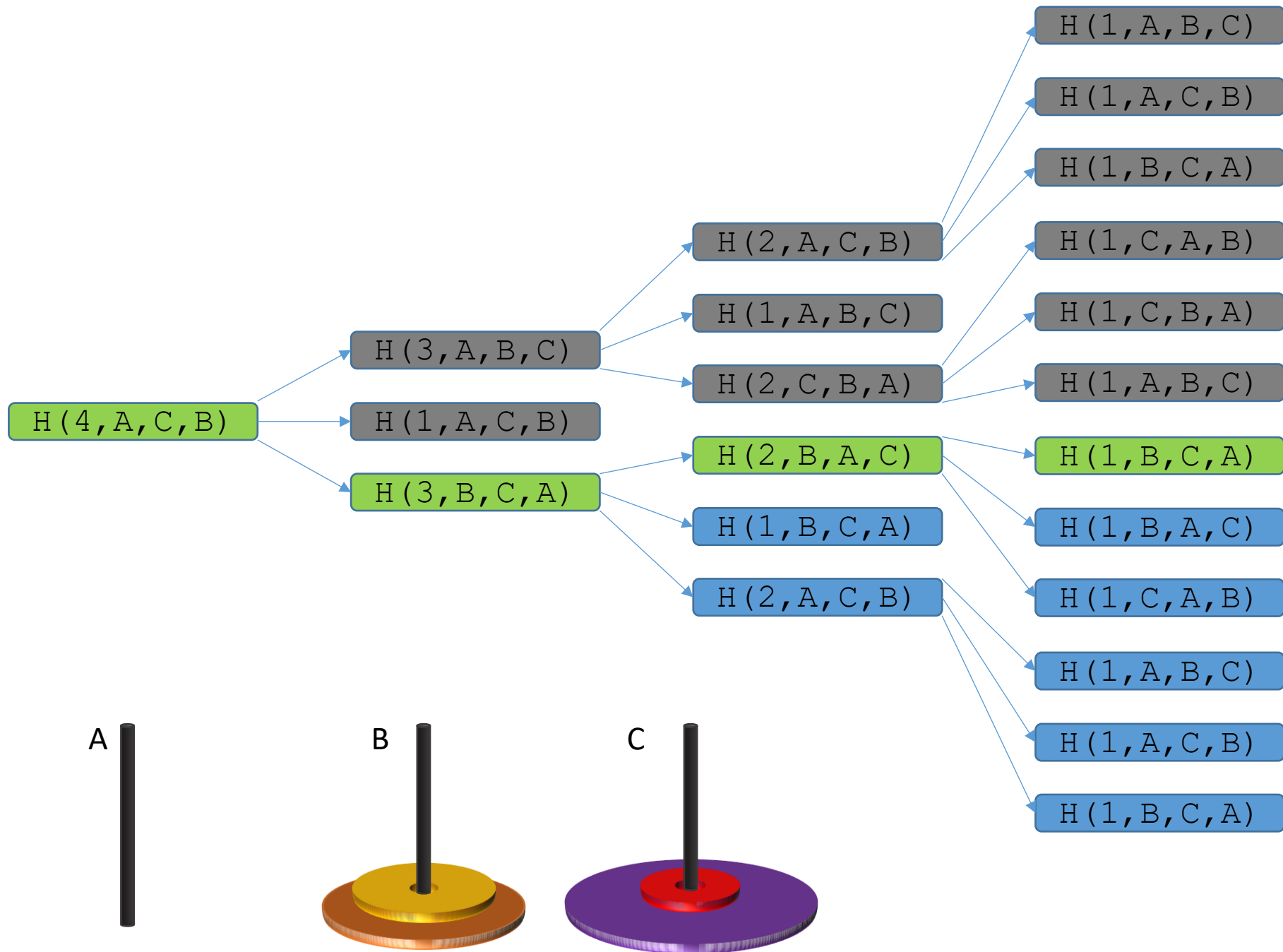


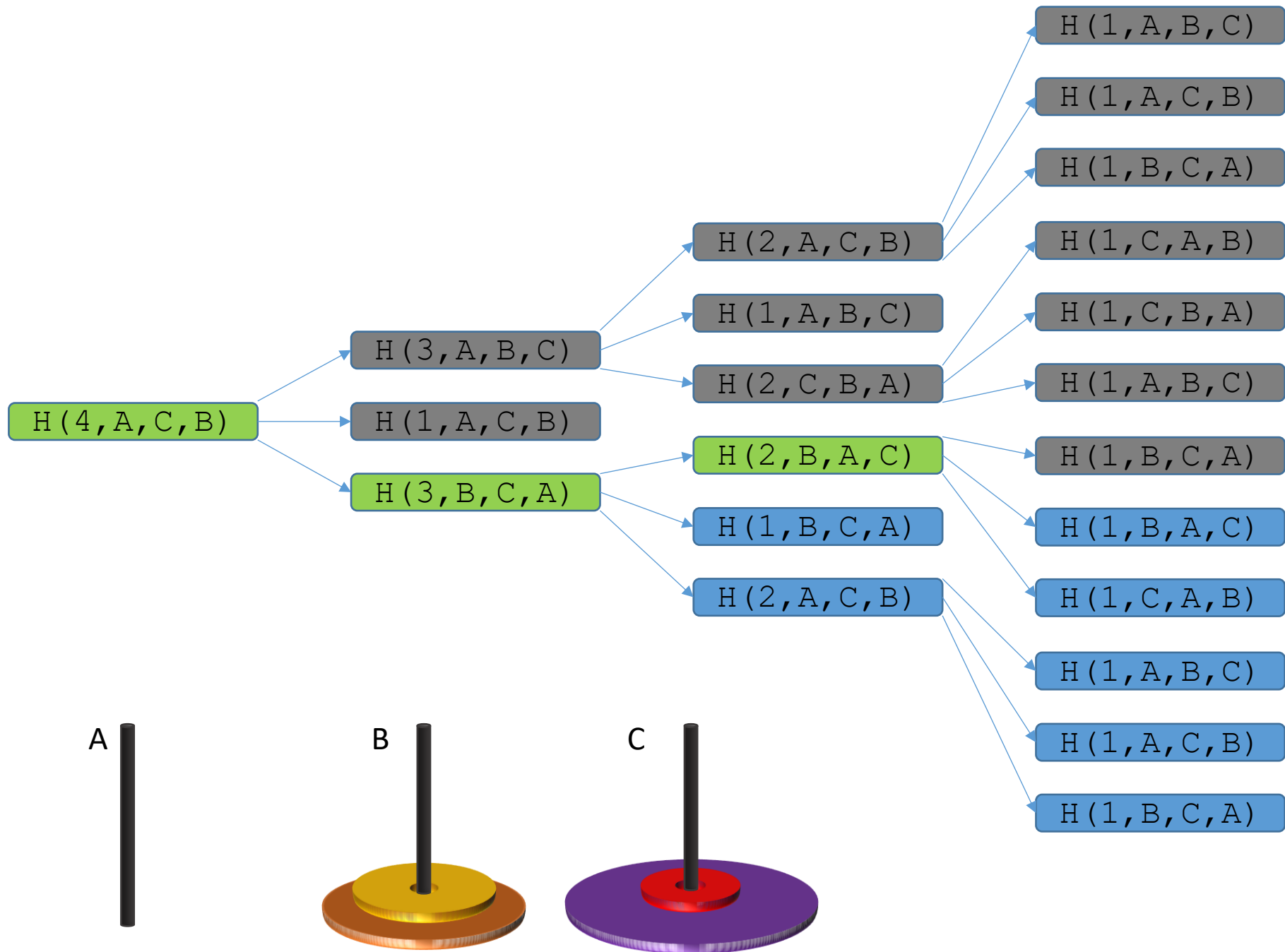


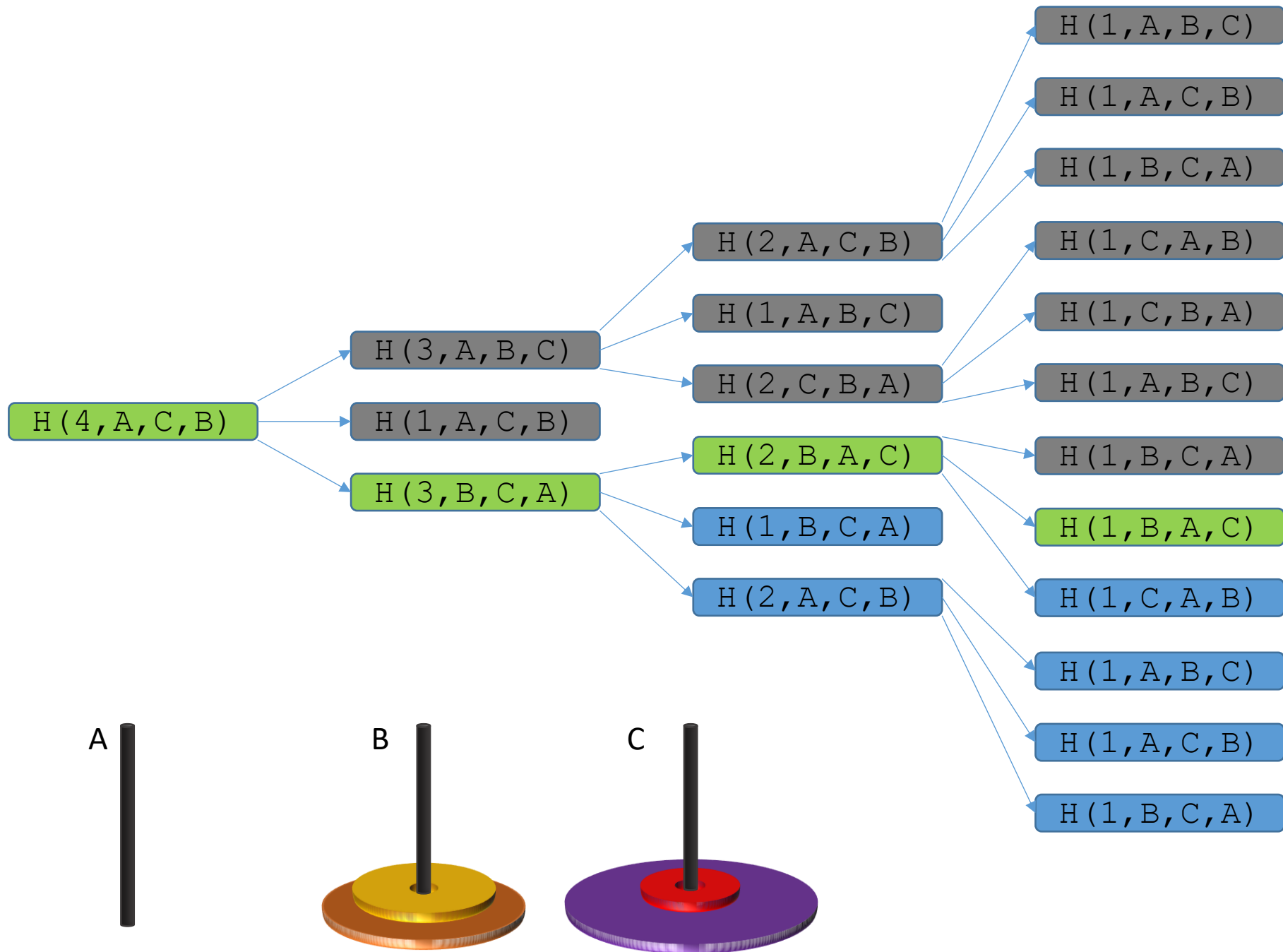


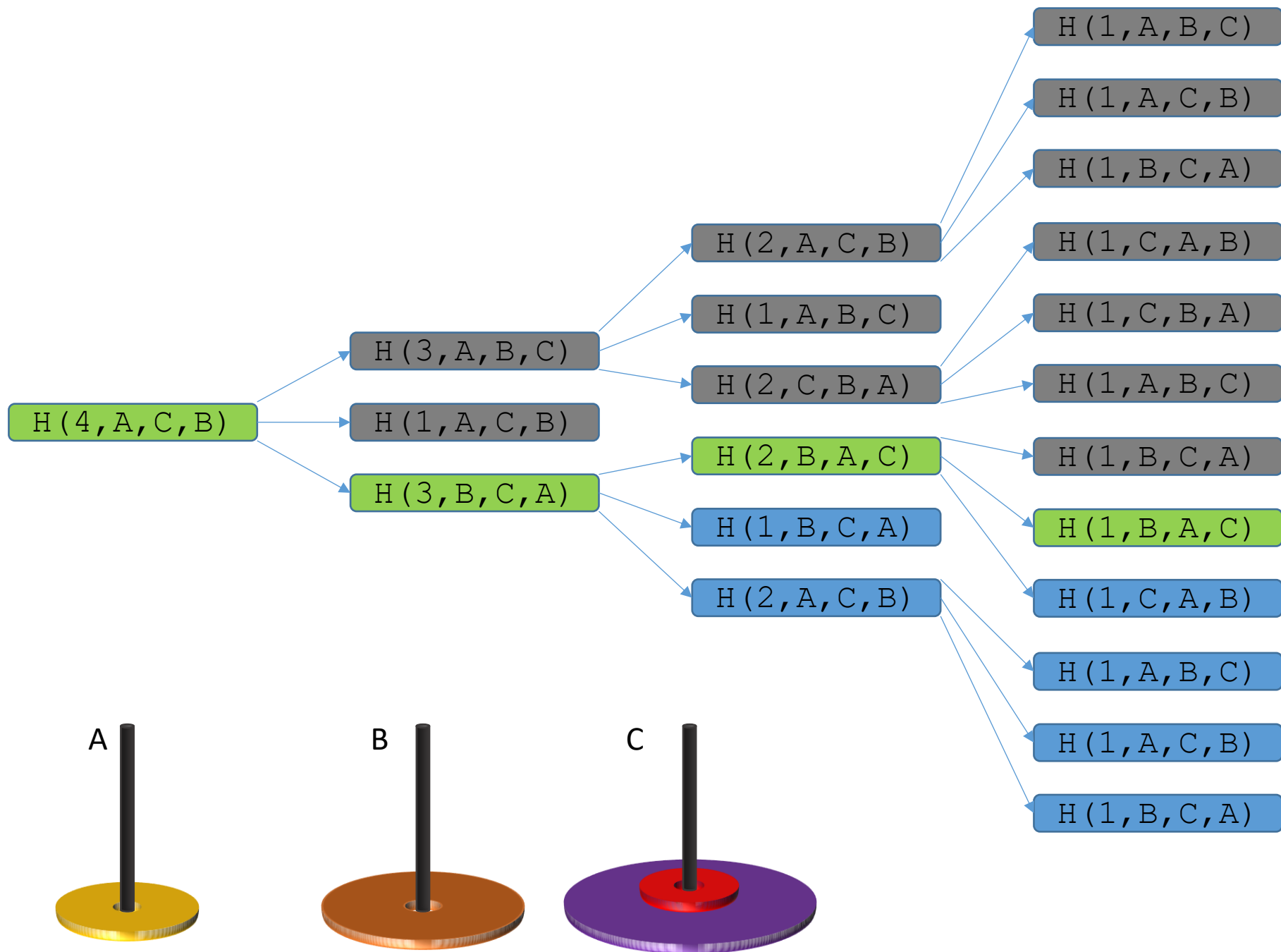


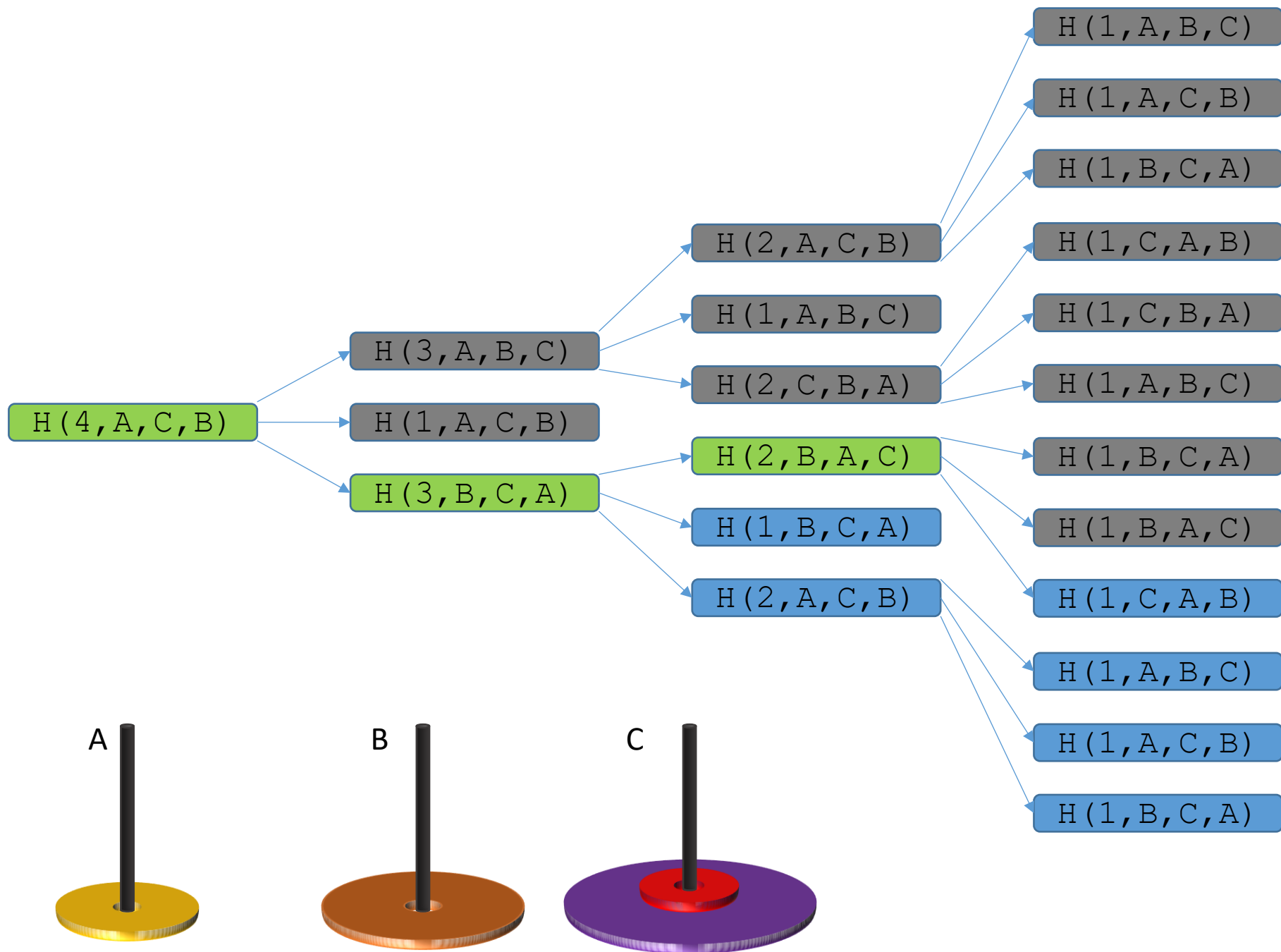


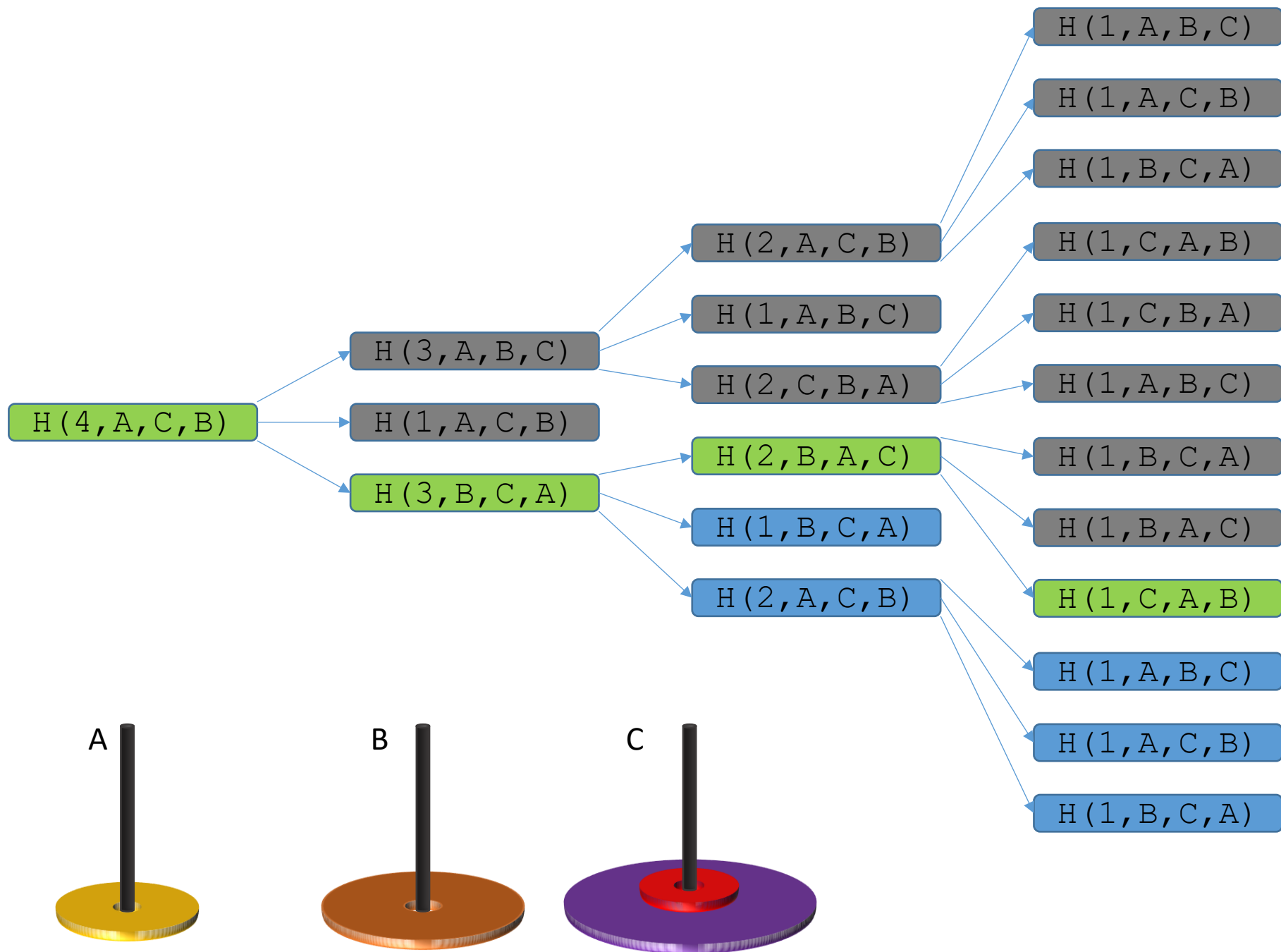


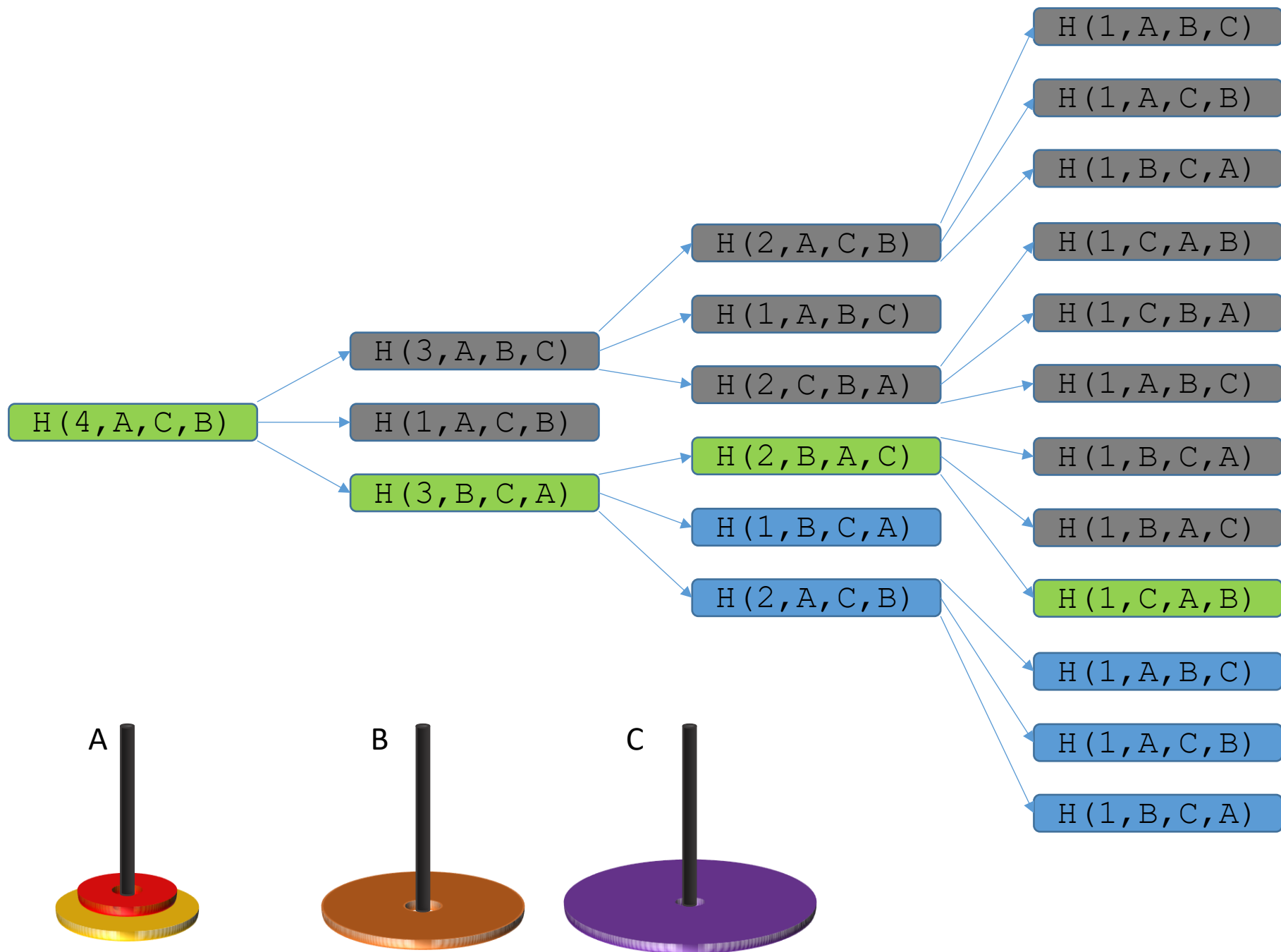


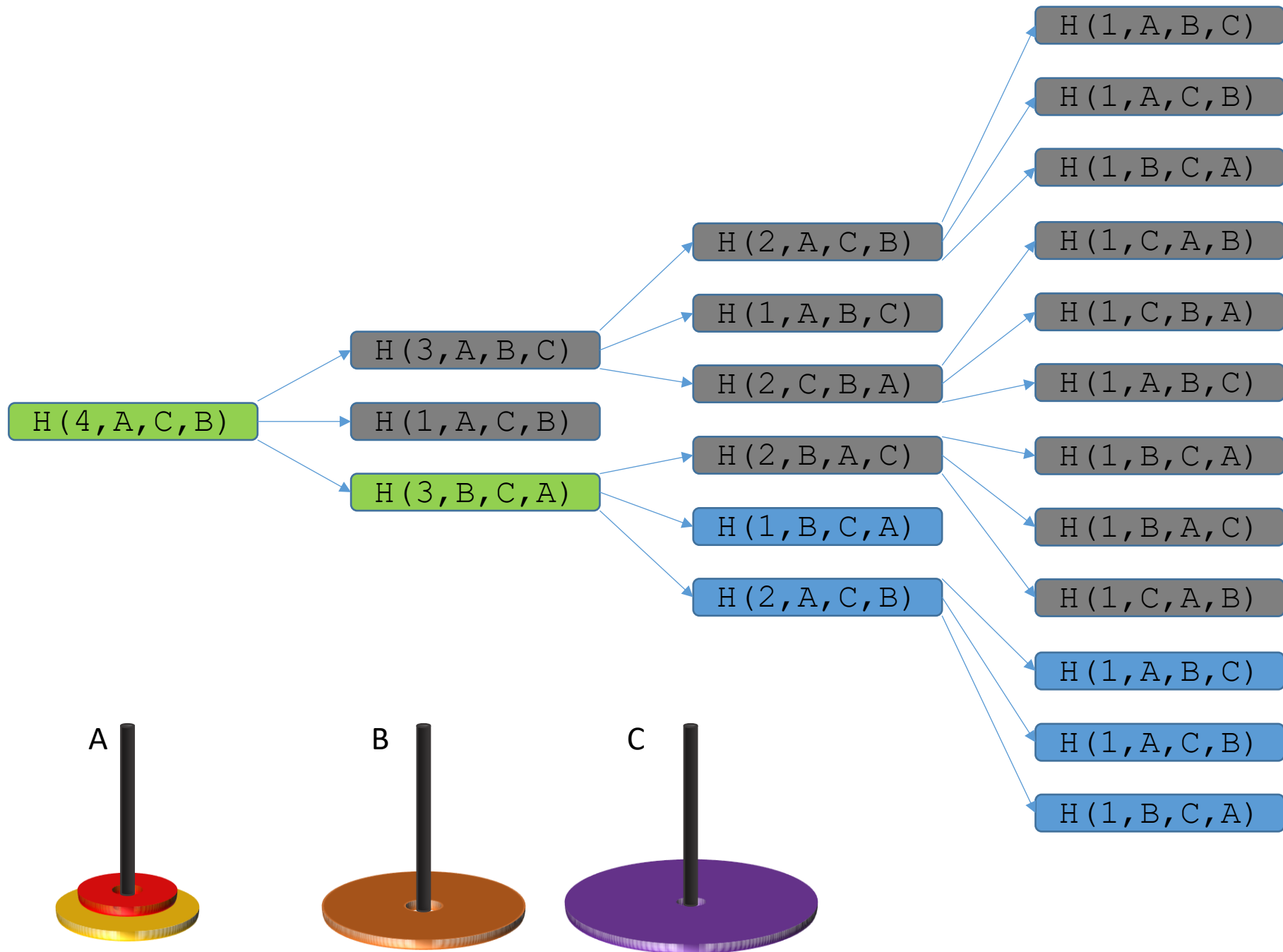


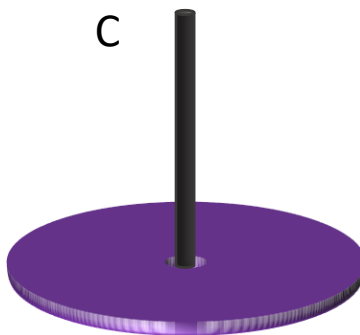
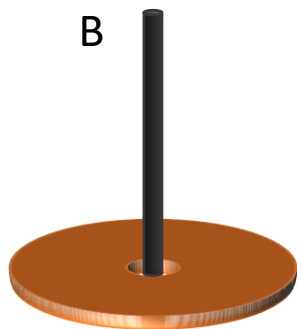
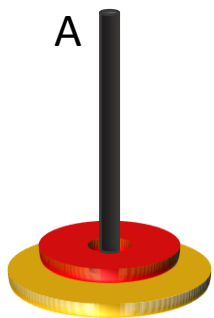
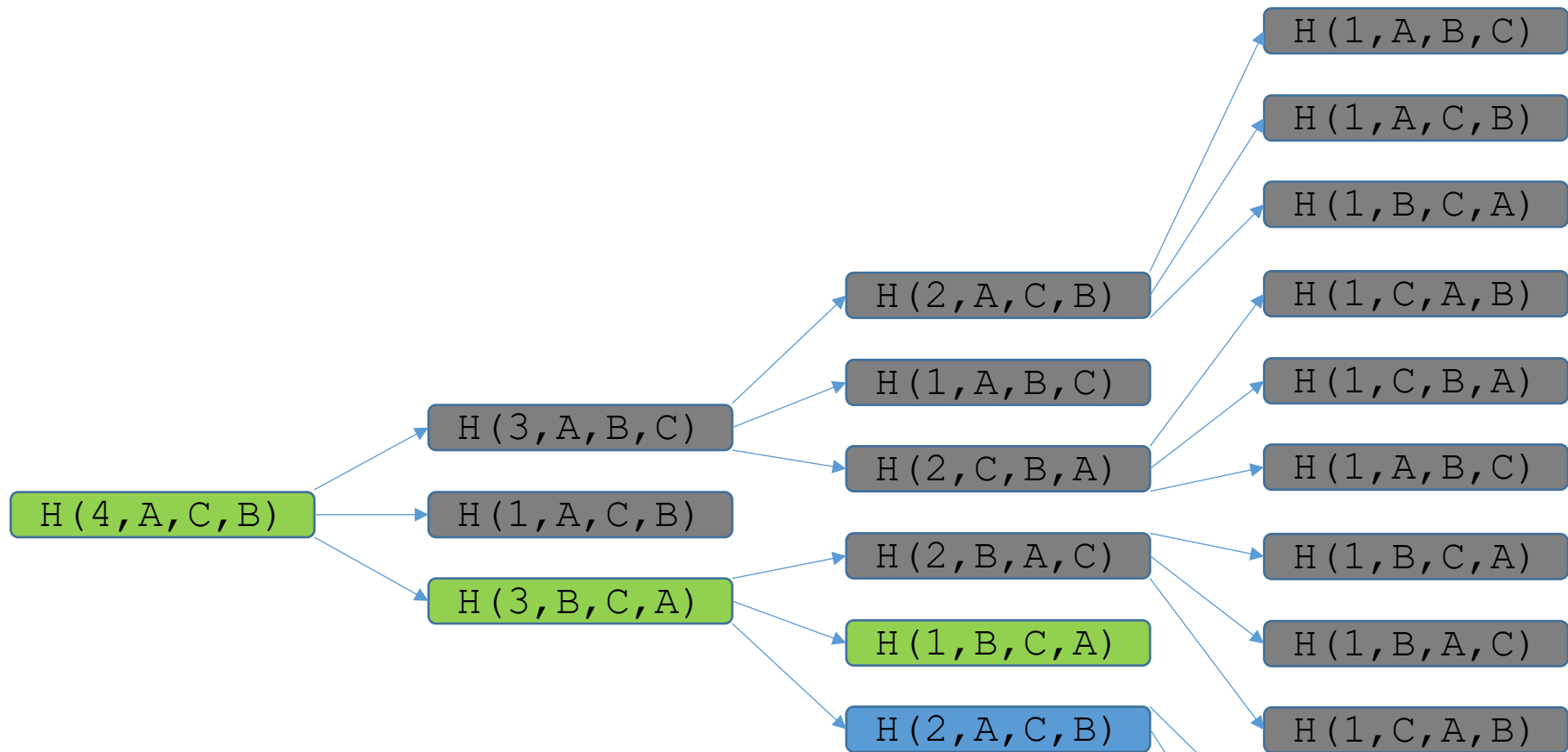


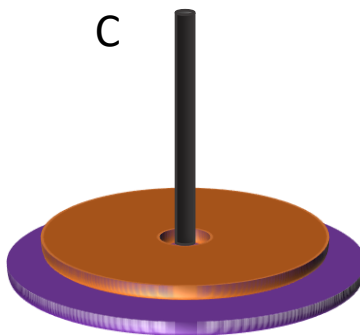
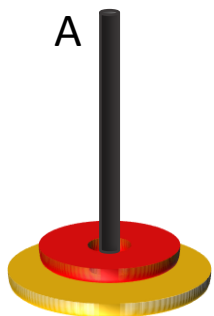
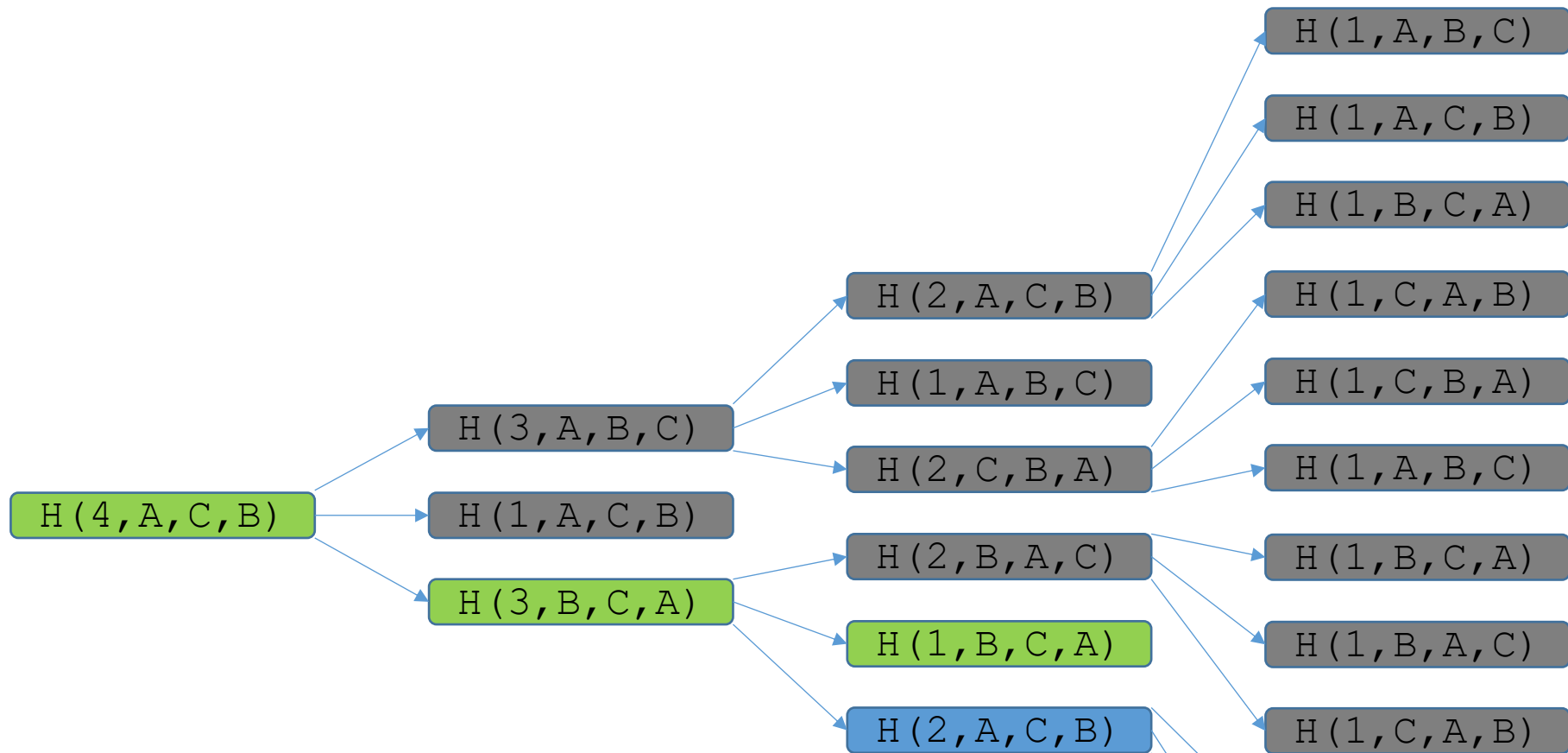


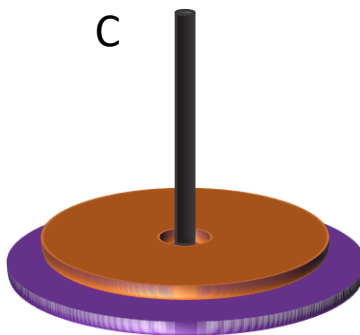
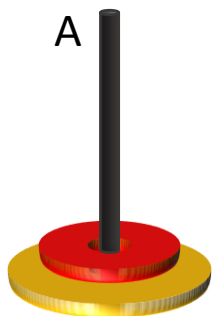
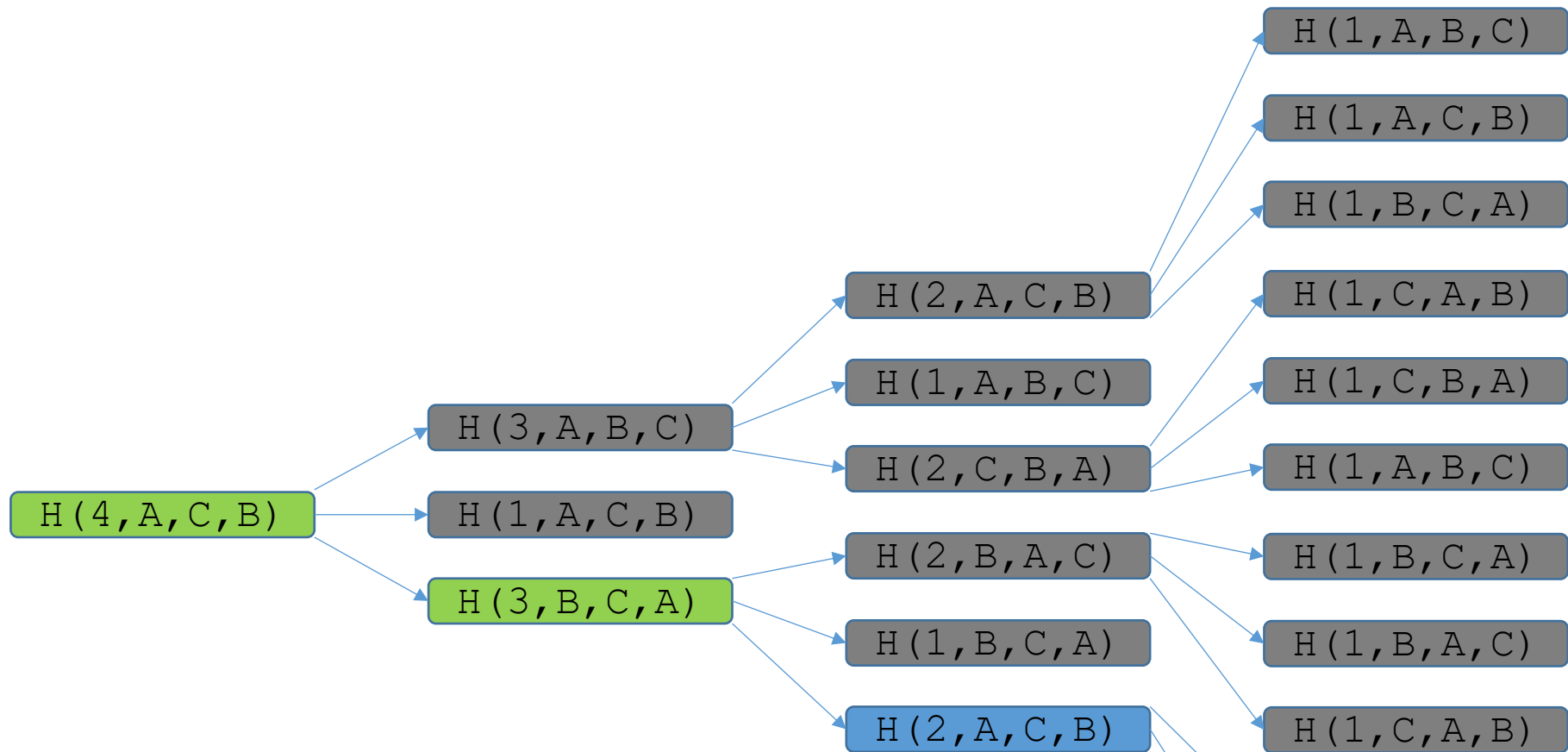


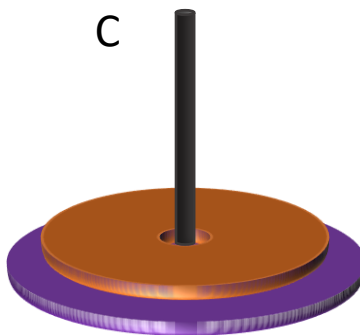
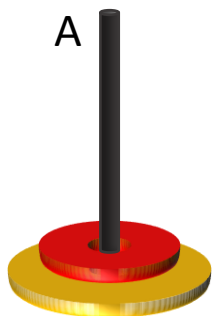
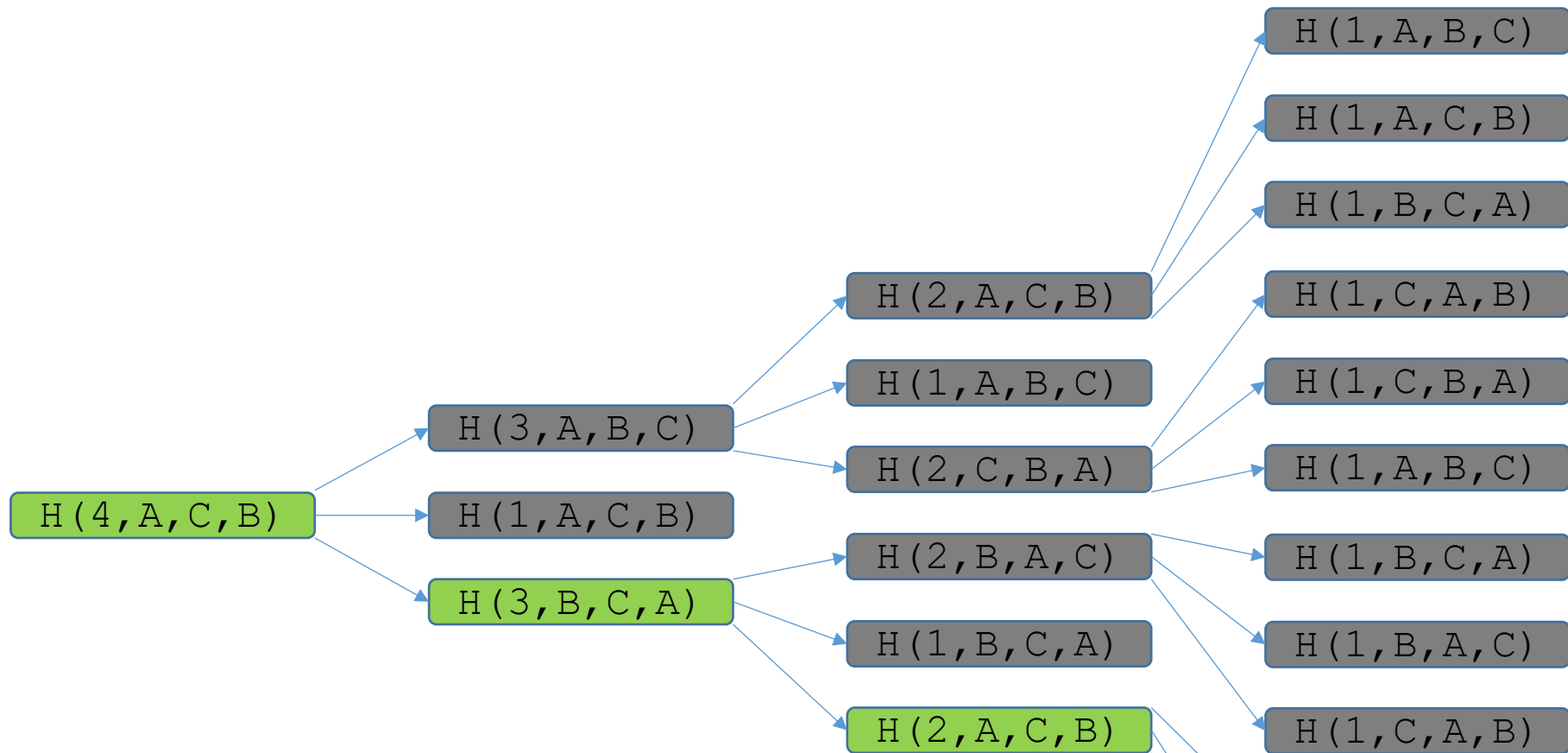


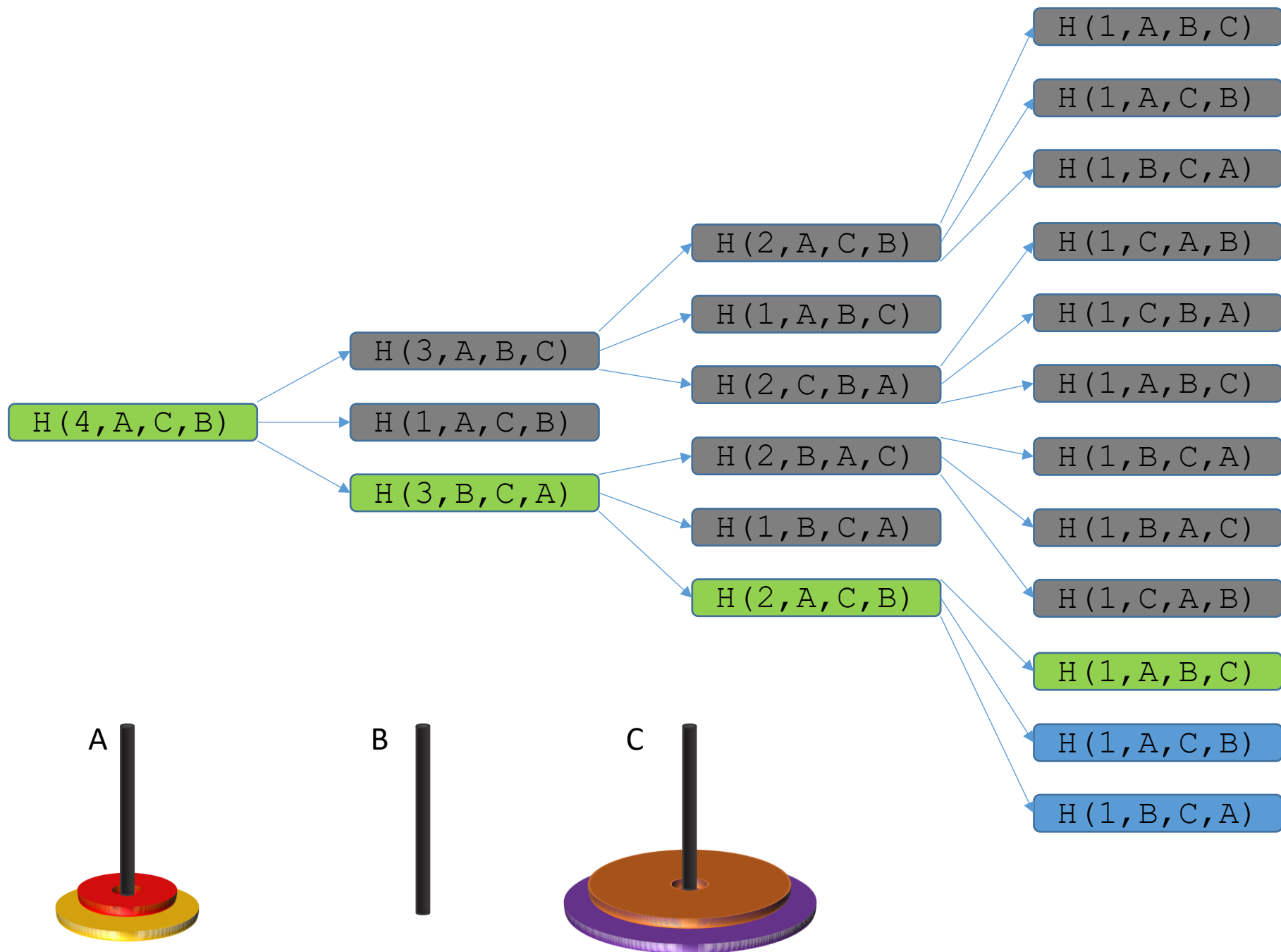


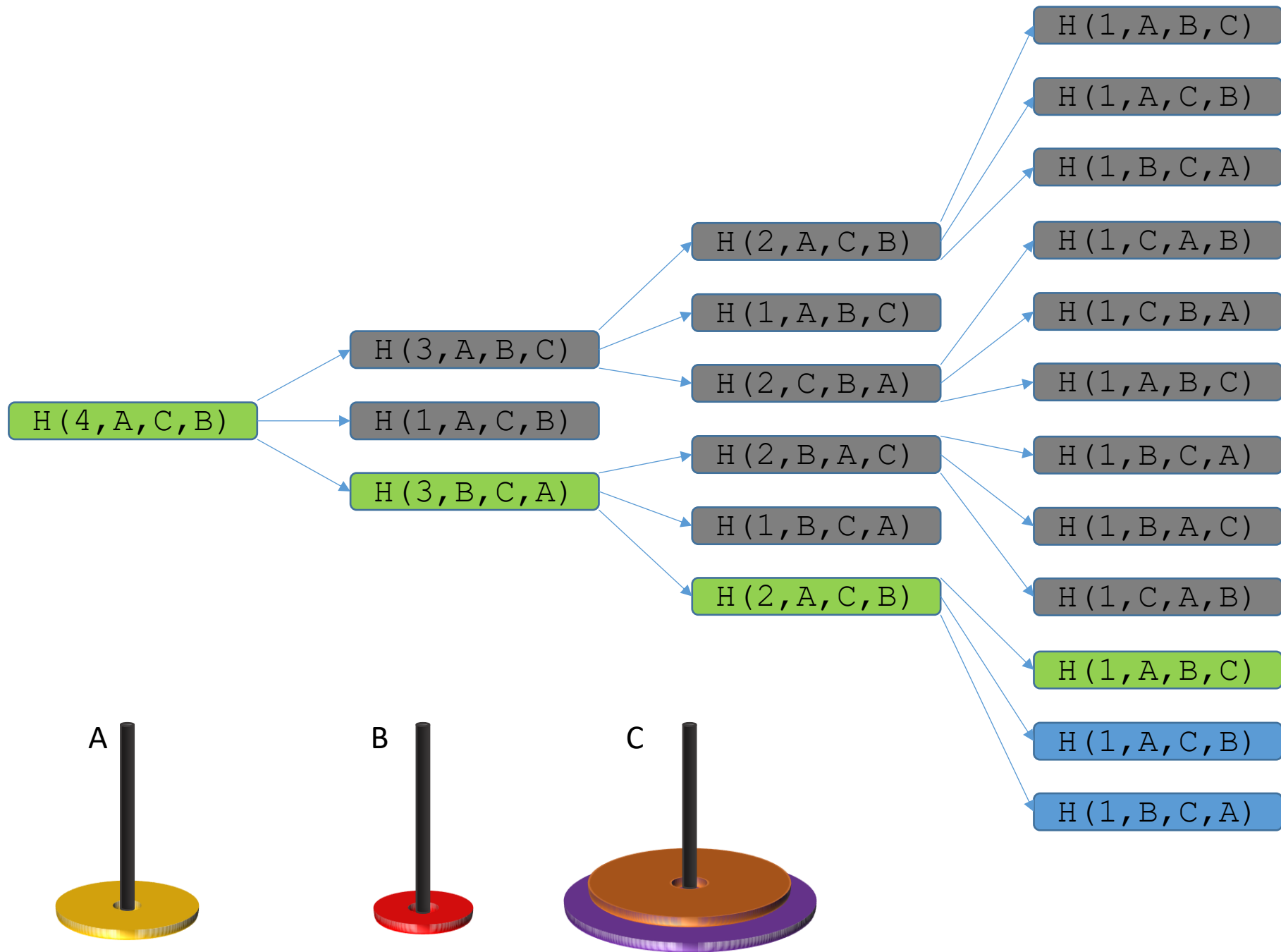


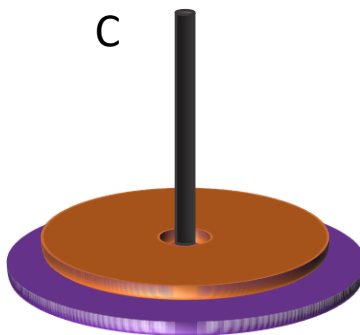
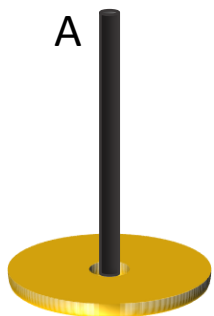
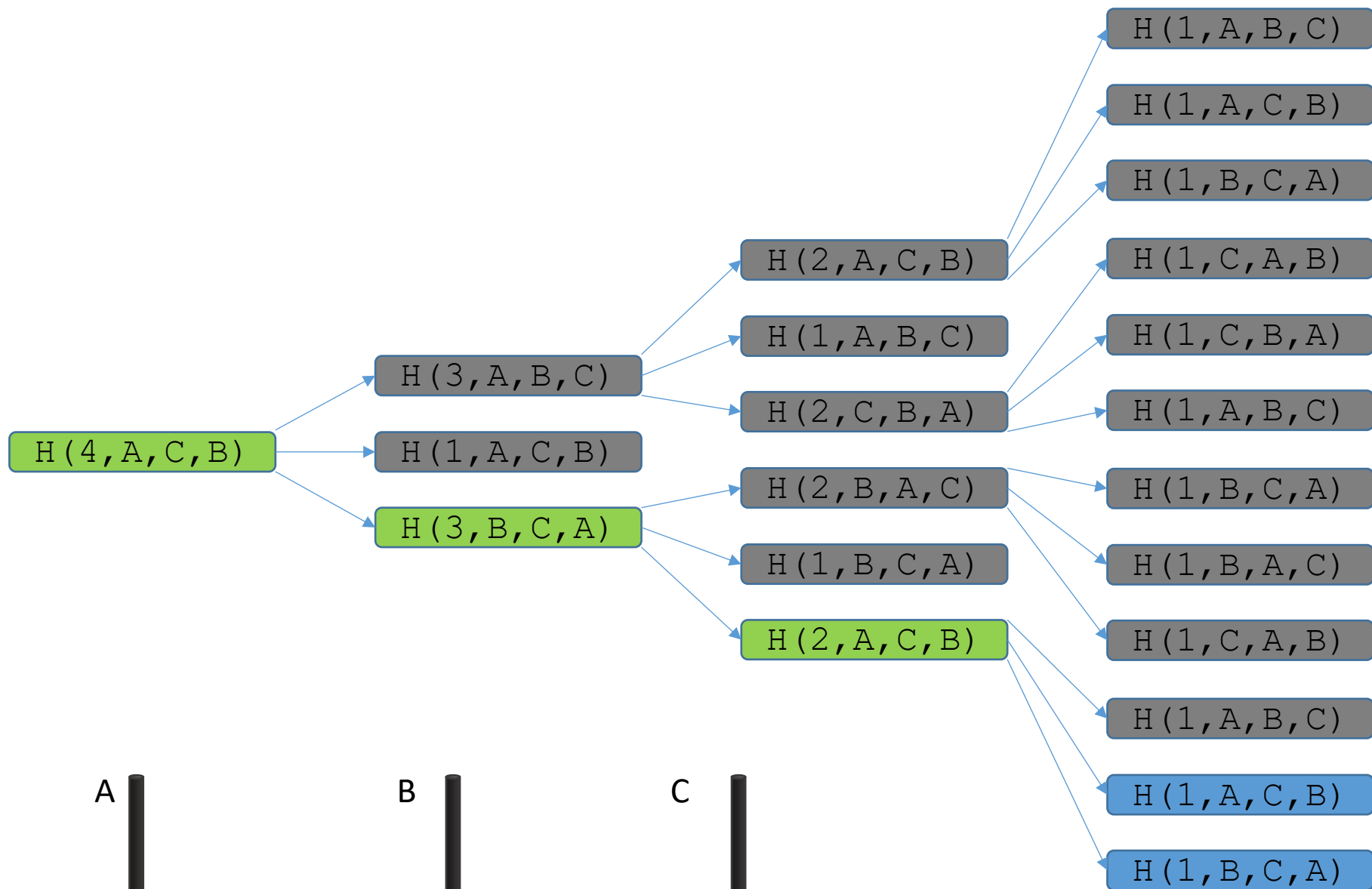


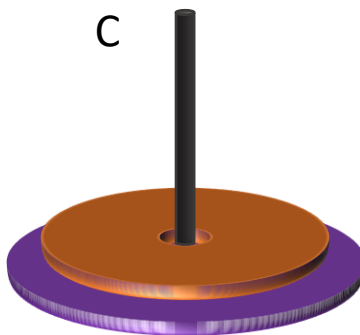
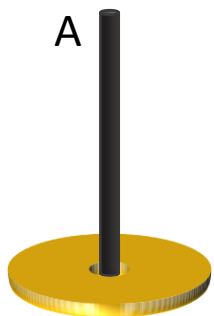
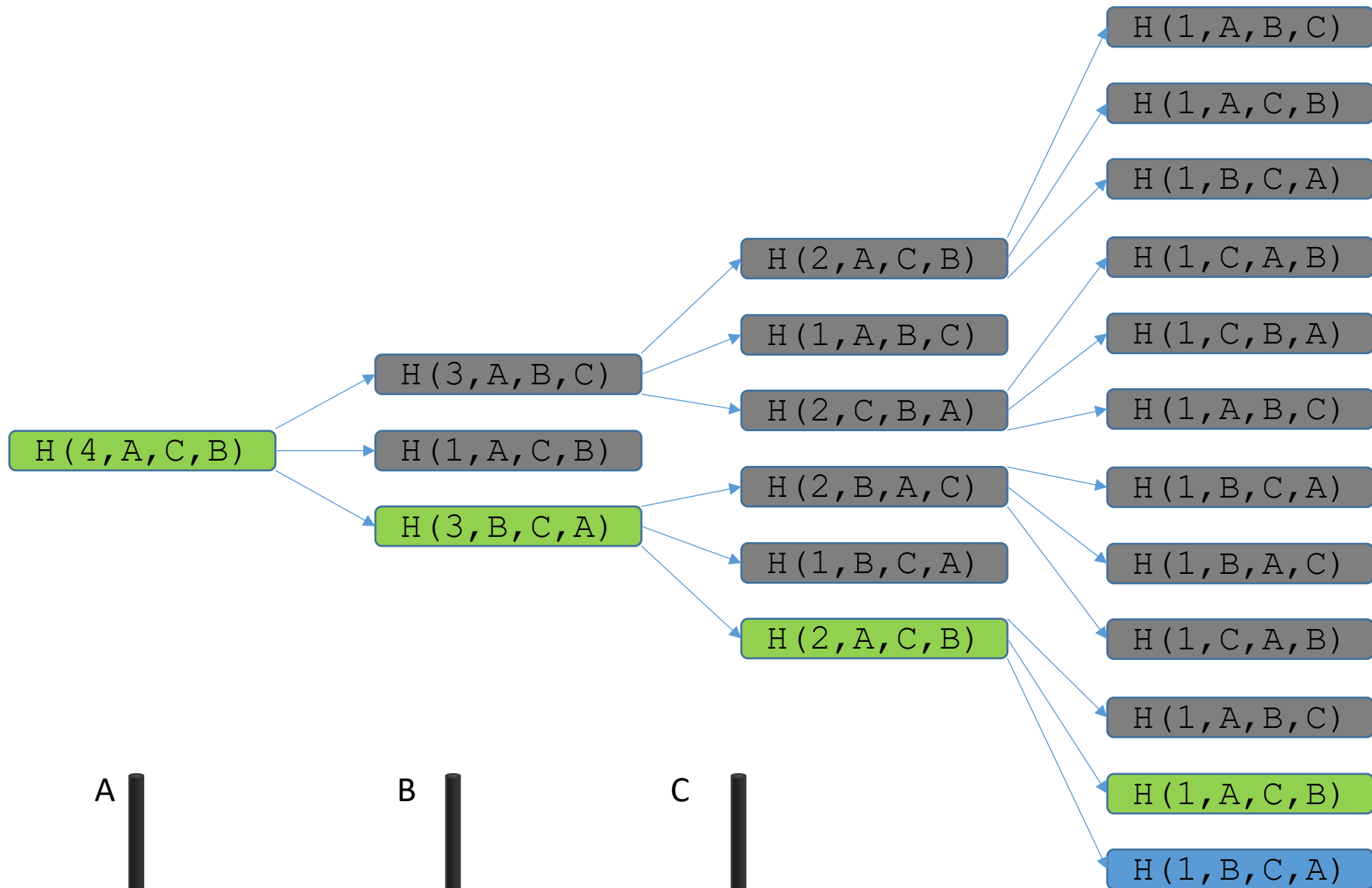


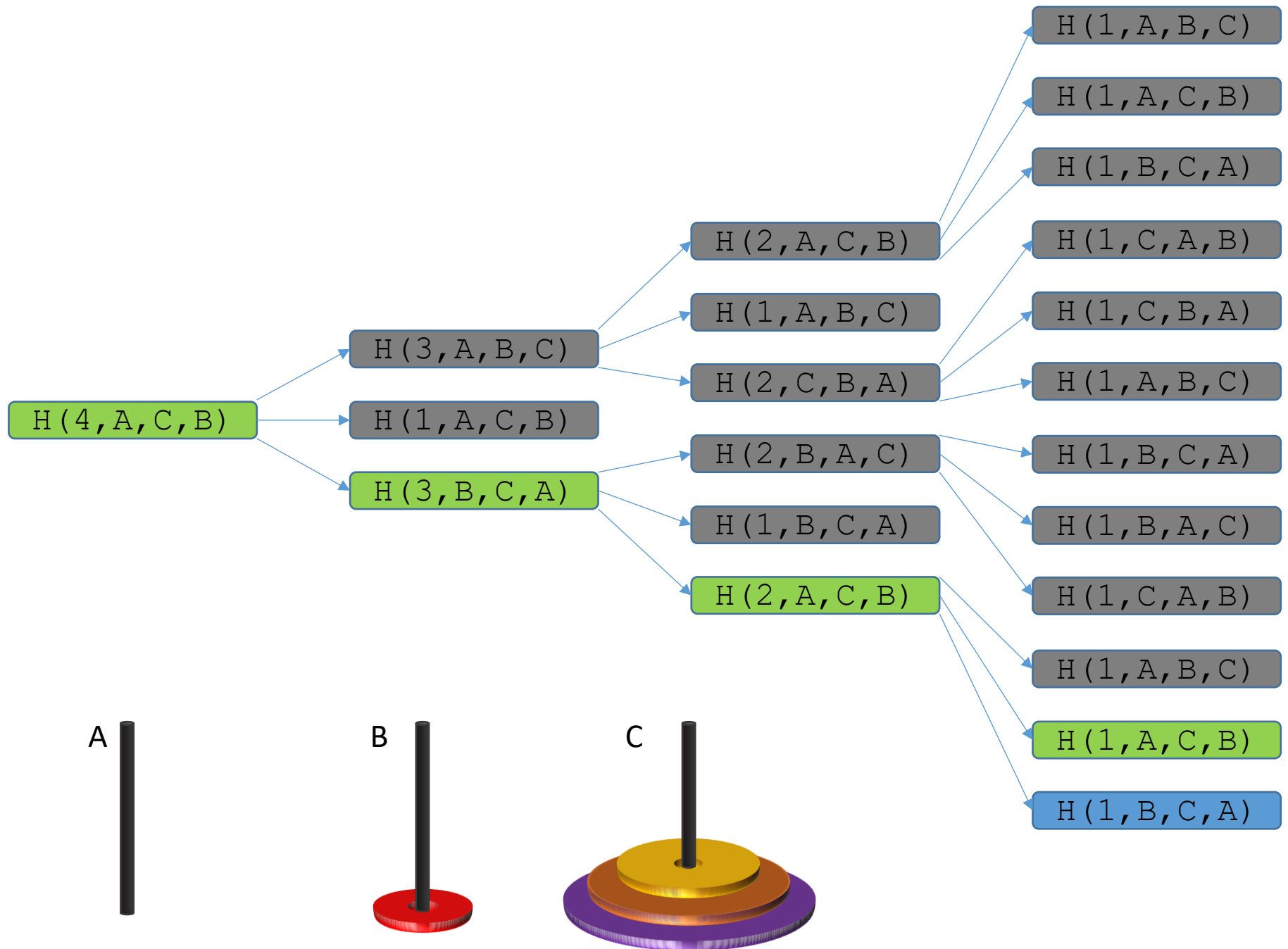


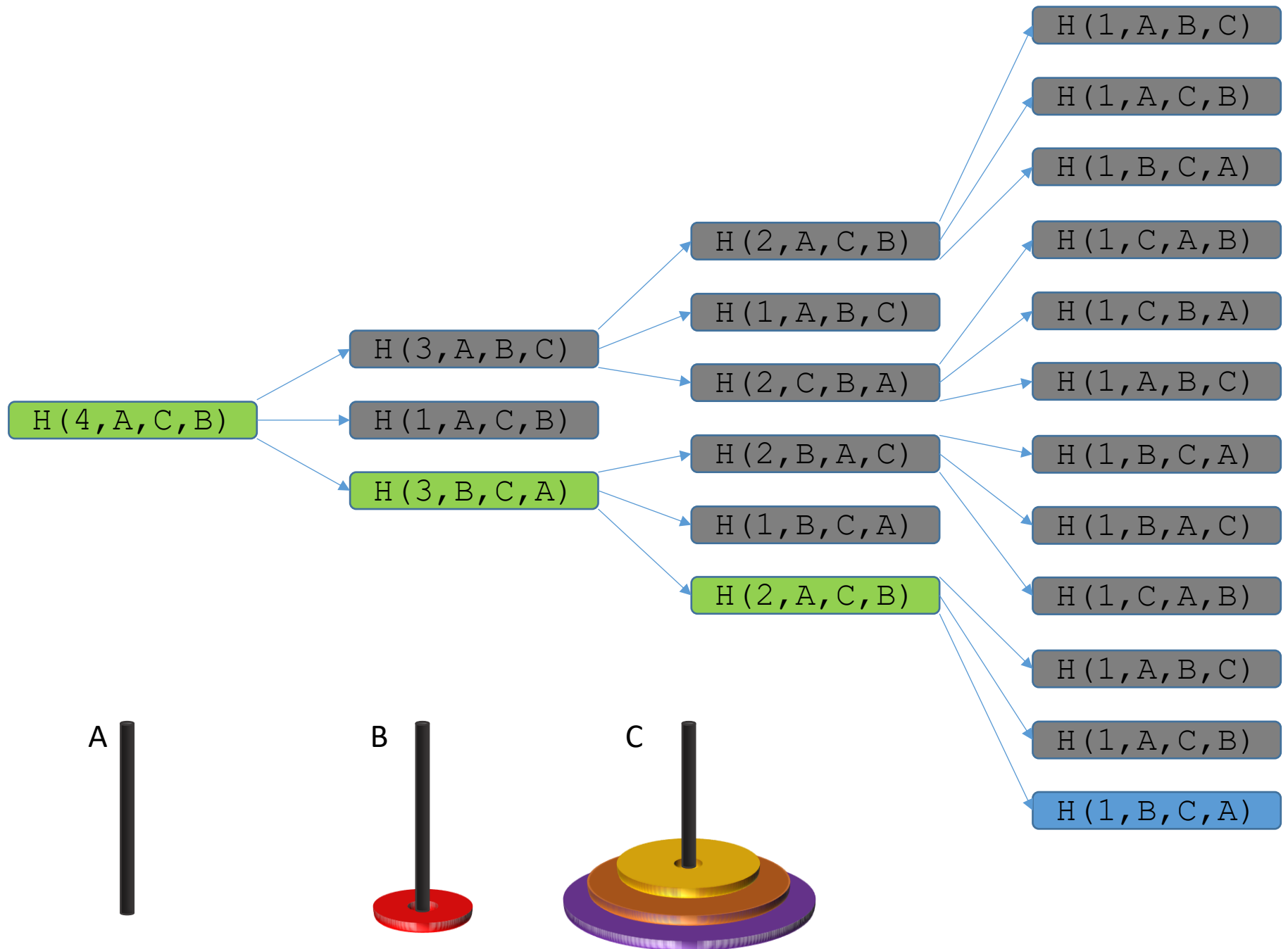


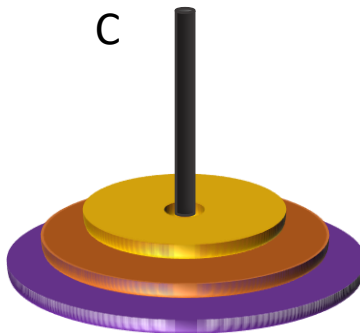
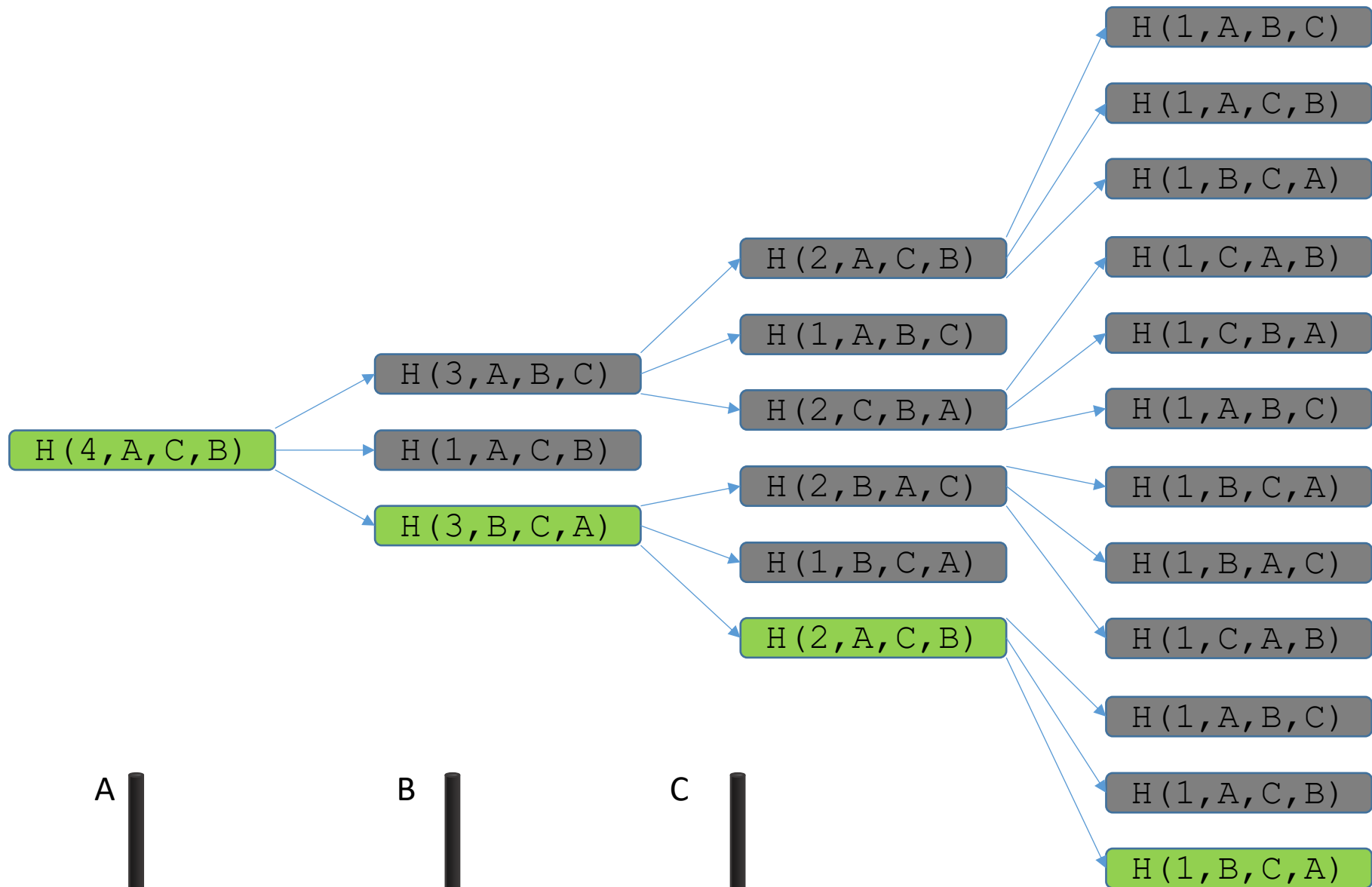


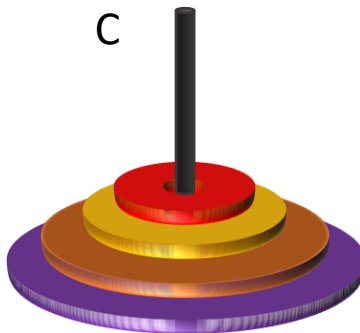
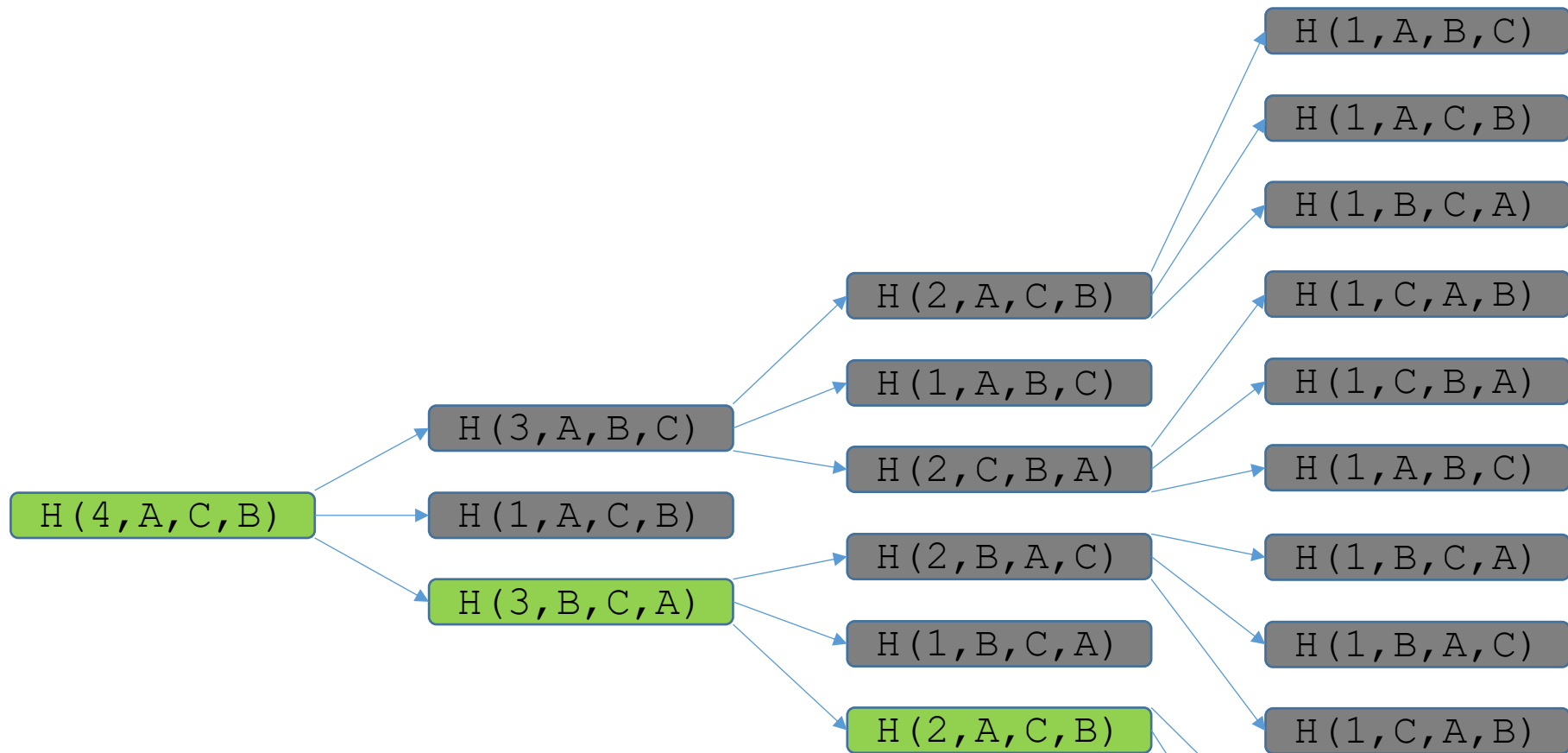


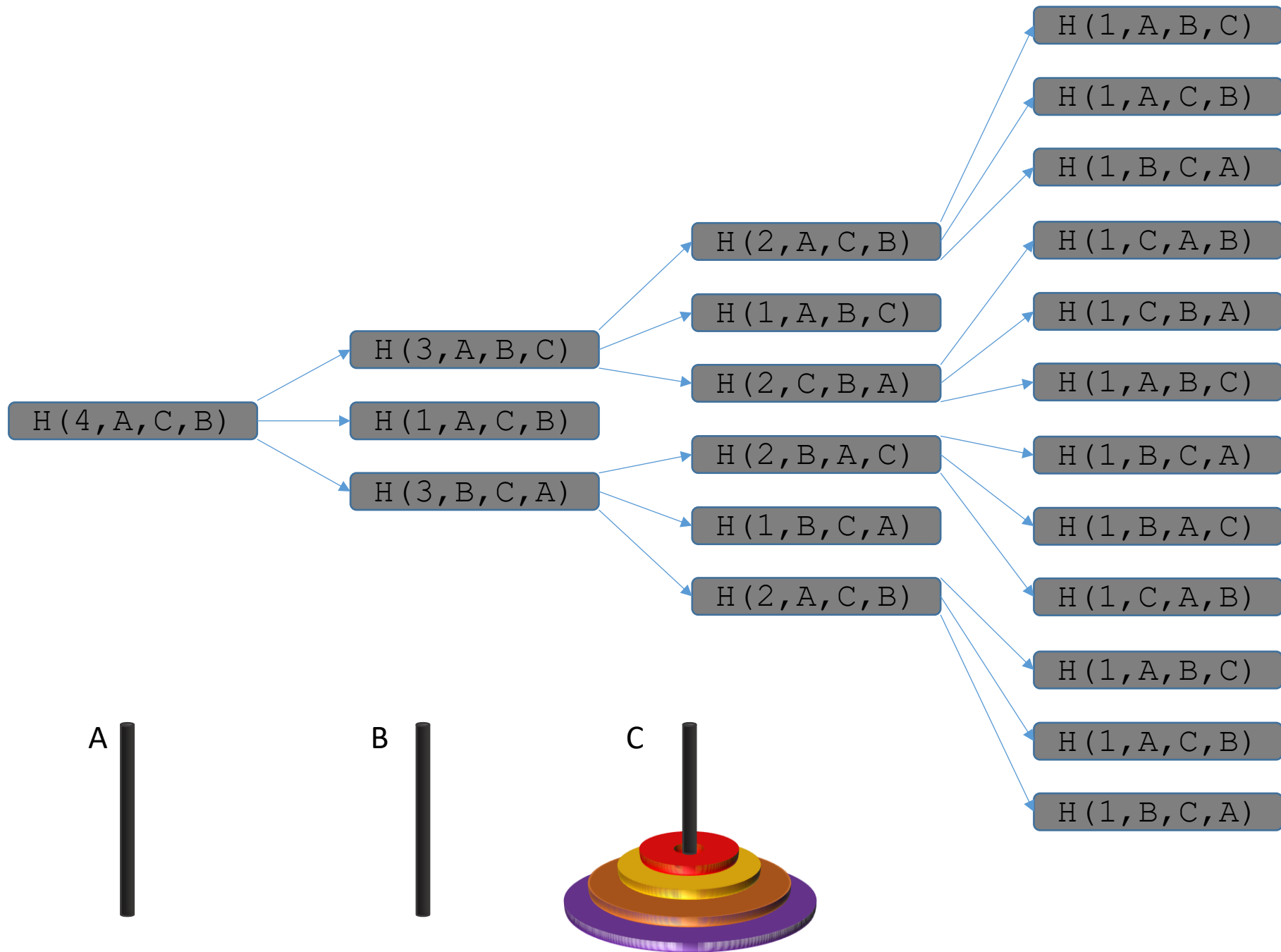












Recursive Function for Solving Hanoi

Output:

```
Move disc from peg A to peg B
Move disc from peg A to peg C
Move disc from peg B to peg C
Move disc from peg A to peg B
Move disc from peg C to peg A
Move disc from peg C to peg B
Move disc from peg A to peg B
Move disc from peg A to peg C
Move disc from peg B to peg C
Move disc from peg B to peg A
Move disc from peg C to peg A
Move disc from peg B to peg C
Move disc from peg A to peg B
Move disc from peg A to peg C
Move disc from peg B to peg C
```

Home-work

Modify the Hanoi function so that it produces the following output

```
Move disc 1 from peg A to peg B
Move disc 2 from peg A to peg C
Move disc 1 from peg B to peg C
Move disc 3 from peg A to peg B
Move disc 1 from peg C to peg A
Move disc 2 from peg C to peg B
Move disc 1 from peg A to peg B
Move disc 4 from peg A to peg C
Move disc 1 from peg B to peg C
Move disc 2 from peg B to peg A
Move disc 1 from peg C to peg A
Move disc 3 from peg B to peg C
Move disc 1 from peg A to peg B
Move disc 2 from peg A to peg C
Move disc 1 from peg B to peg C
```